Chapter 3

Memory Hierarchy

- Technology Trend and Memory Hierarchy
- Four Questions for Cache Designers
- Memory System Performance
  - Improve Cache Performance
- CPU vulnerabilities

Memory

- Register
- Cache
- Memory
- Storage

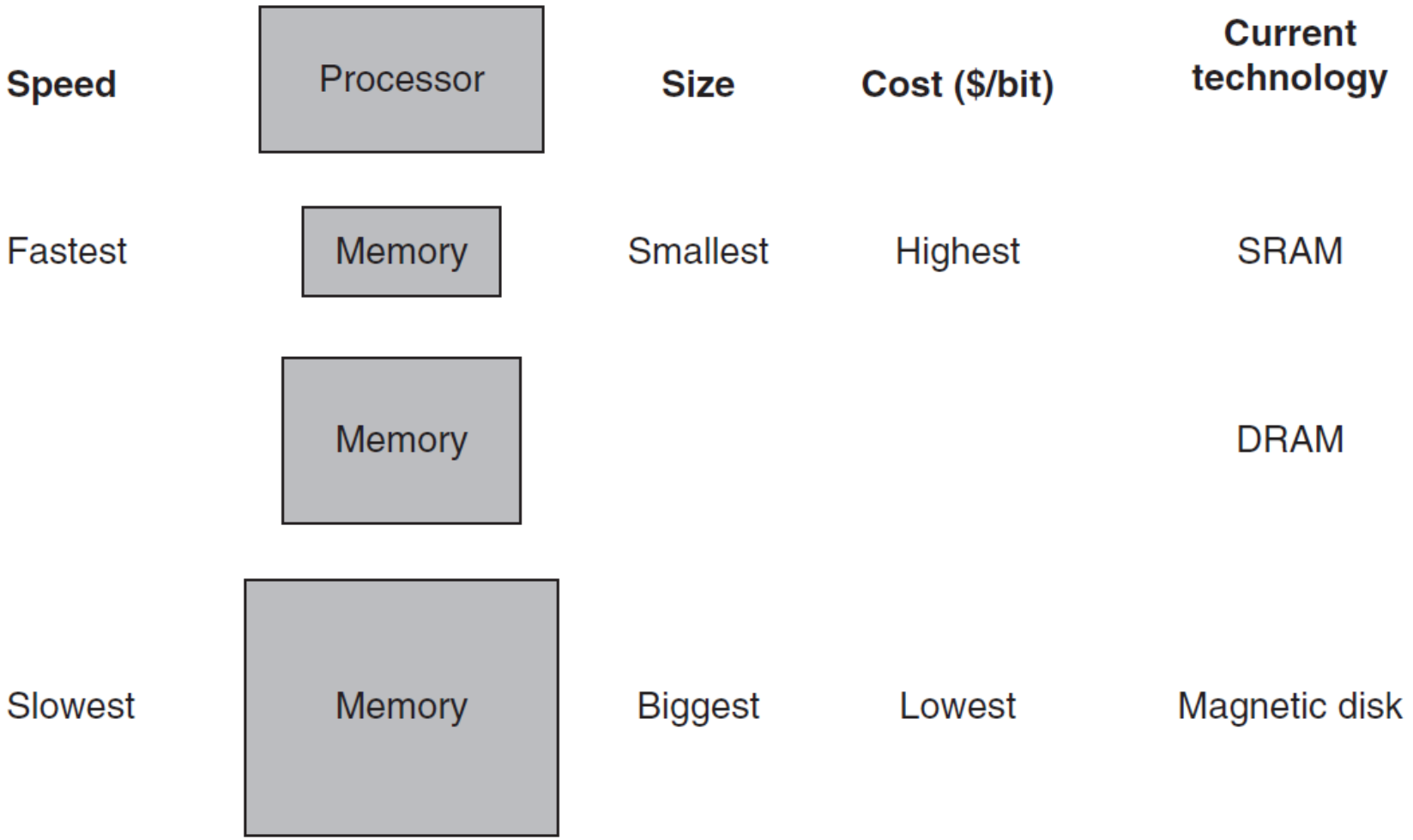
- Mechanical memory
  - Acoustic wave/torque wave delay line memory
  - Magnetic Drum Memory
  - Magnetic core memory
- Electronic memory
  - SRAM
  - DRAM
    - SDRAM
  - Flash
  - ROM
    - PROM
    - EPROM
- Optical memory

Temporal locality (locality in time): if an item is referenced, it will tend to be referenced again soon. If you recently brought a book to your desk to look at, you will probably need to look at it again soon.

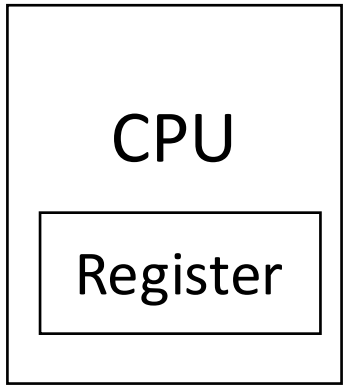
Spatial locality (locality in space): if an item is referenced, items whose addresses are close by will tend to be referenced soon.

Memory?

- Load R2, 0(R1)
- Store R2, 0(R1)

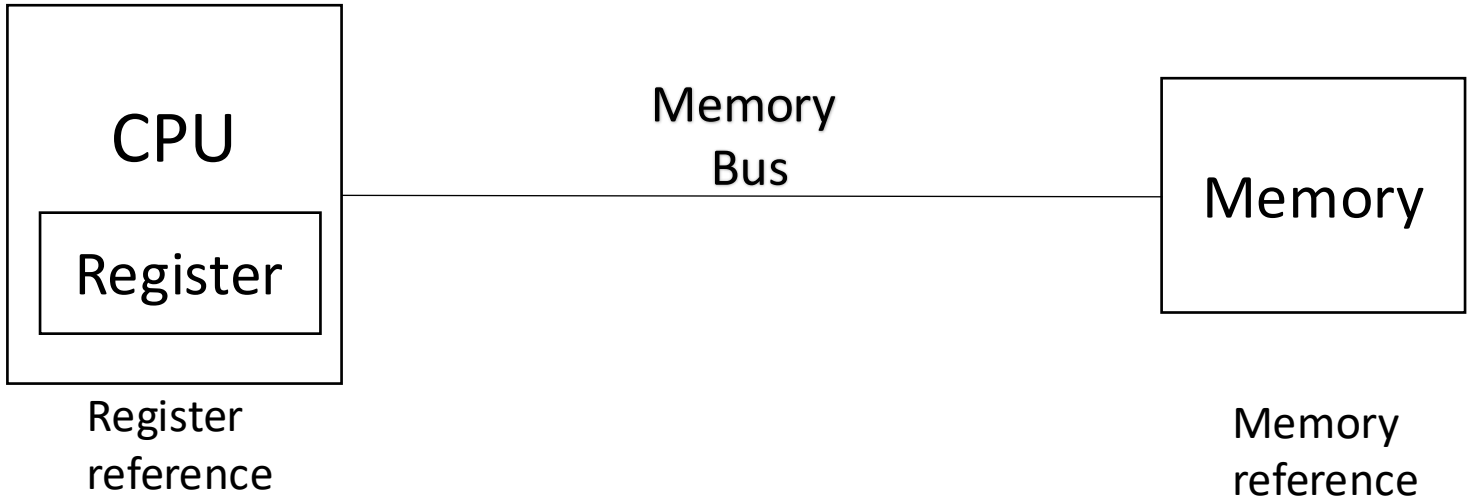

Speed
Processor
Size
Cost ($/bit)
Current technology
Fastest
Memory
Smallest
Highest
SRAM
Memory
DRAM
Slowest
Memory
Biggest
Lowest
Magnetic disk

Data processing & temporary storage

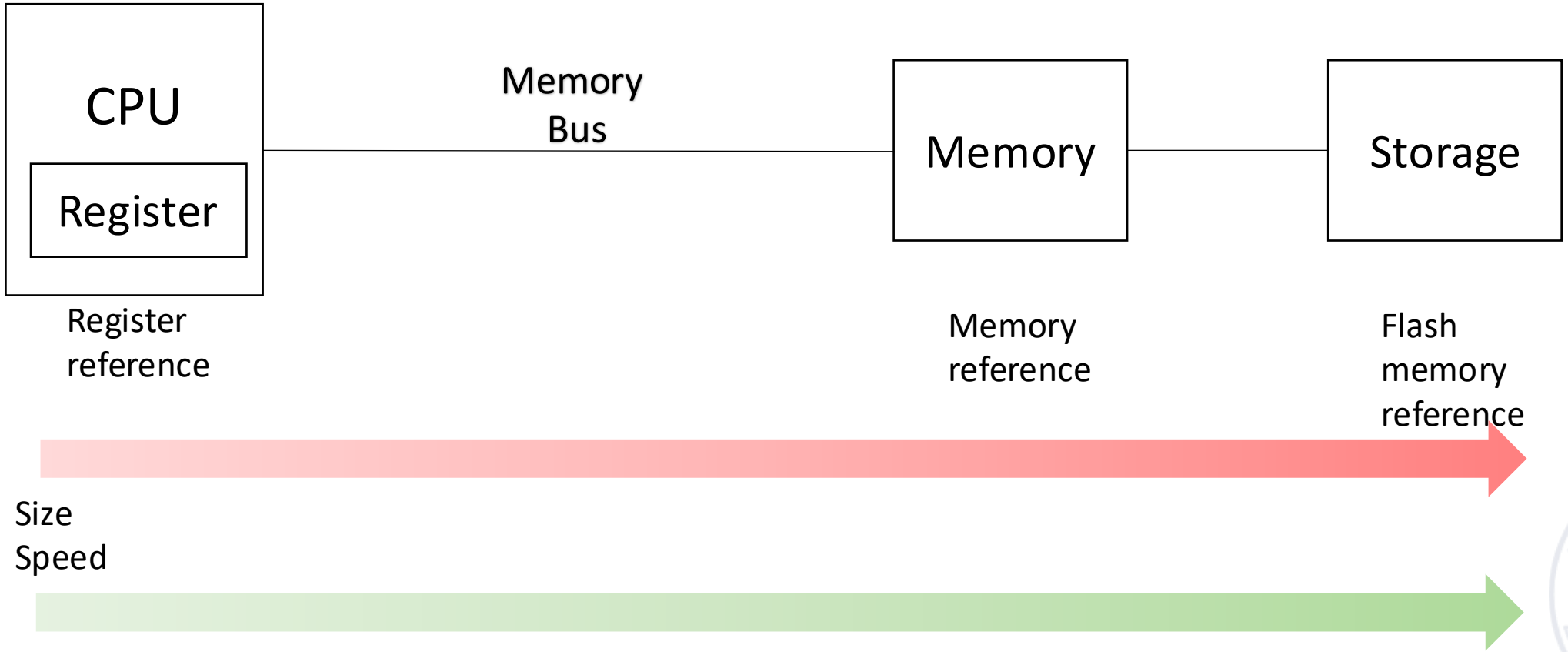


Register reference

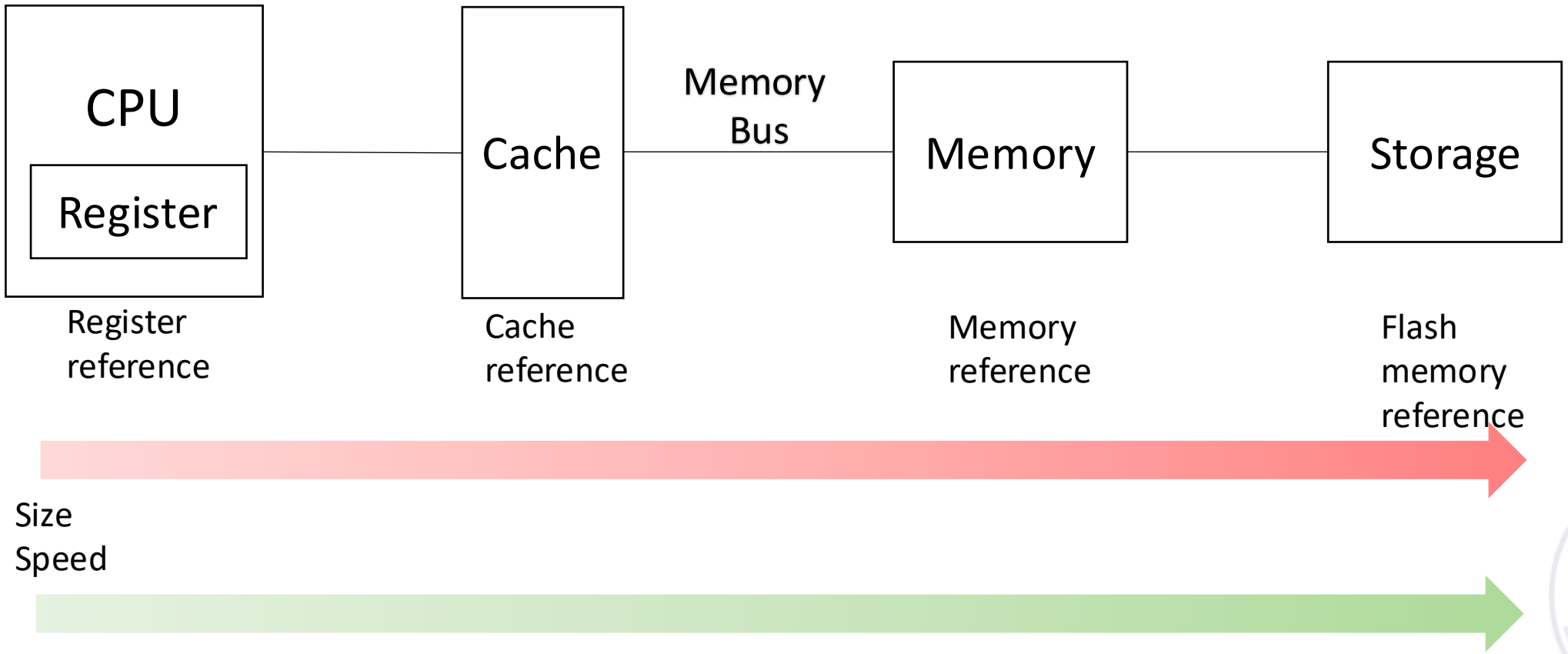
Temporary storage



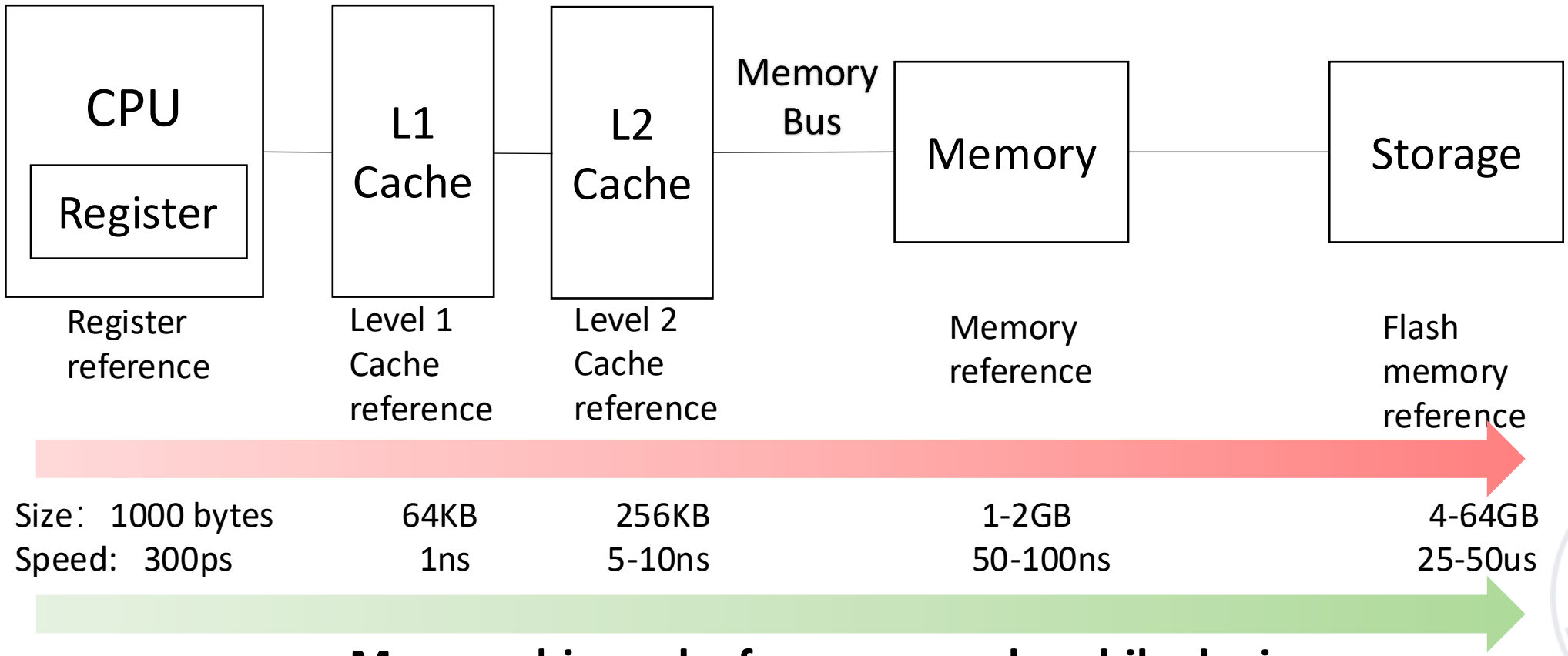
Faster temporary storage



Faster temporary storage



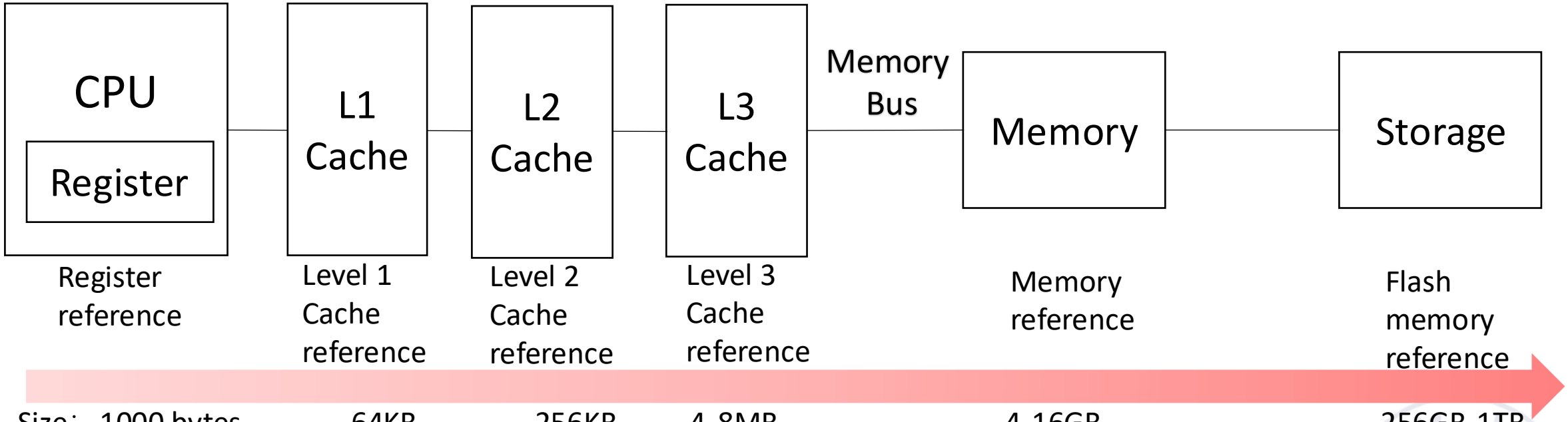
Memory Hierarchy



Memory hierarchy for a personal mobile device

*1000 picoseconds = 1 nanosecond = 10^{-6} millisecond

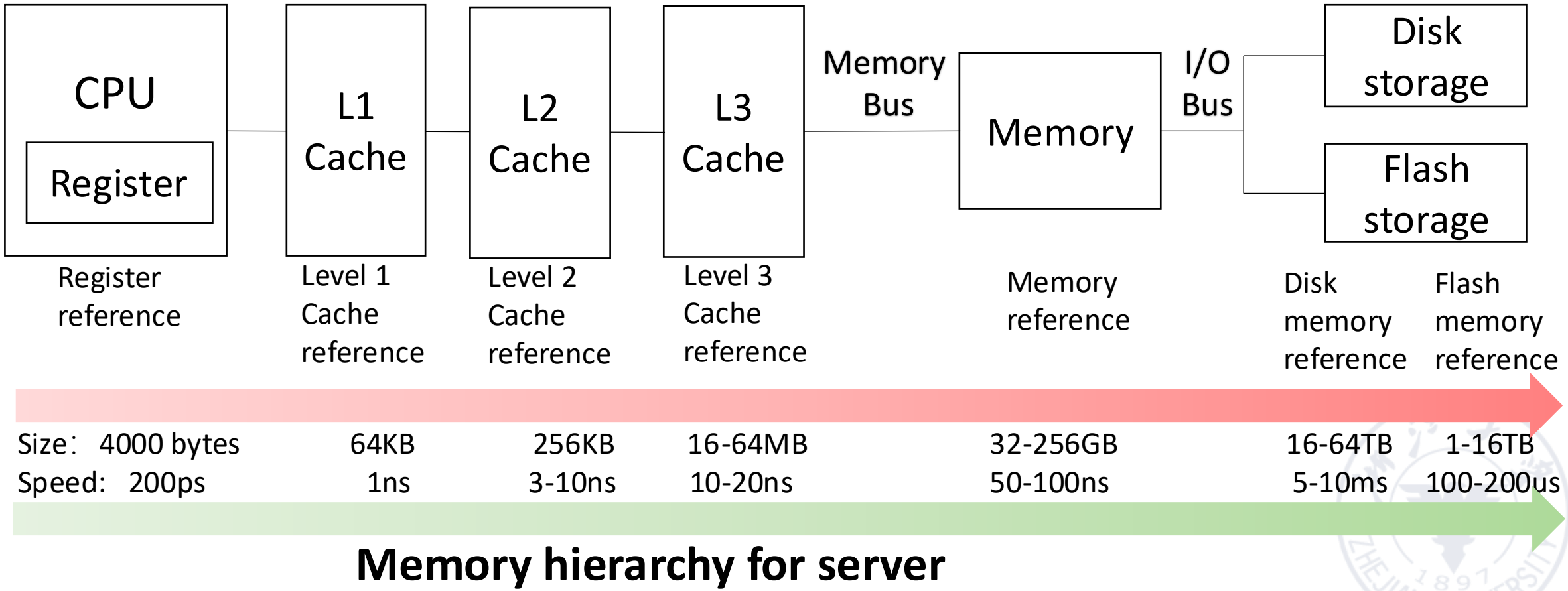
Memory Hierarchy



	Register reference	Level 1 Cache reference	Level 2 Cache reference	Level 3 Cache reference	Memory reference	Flash memory reference
Laptop	Size: 1000 bytes	64KB	256KB	4-8MB	4-16GB	256GB-1TB
	Speed: 300ps	1ns	5-10ns	10-20ns	50-100ns	50-100us
Desktop	Size: 2000 bytes	64KB	256KB	8-32MB	8-64GB	256GB-2TB
	Speed: 300ps	1ns	5-10ns	10-20ns	50-100ns	50-100us

Memory hierarchy for a laptop or a desktop

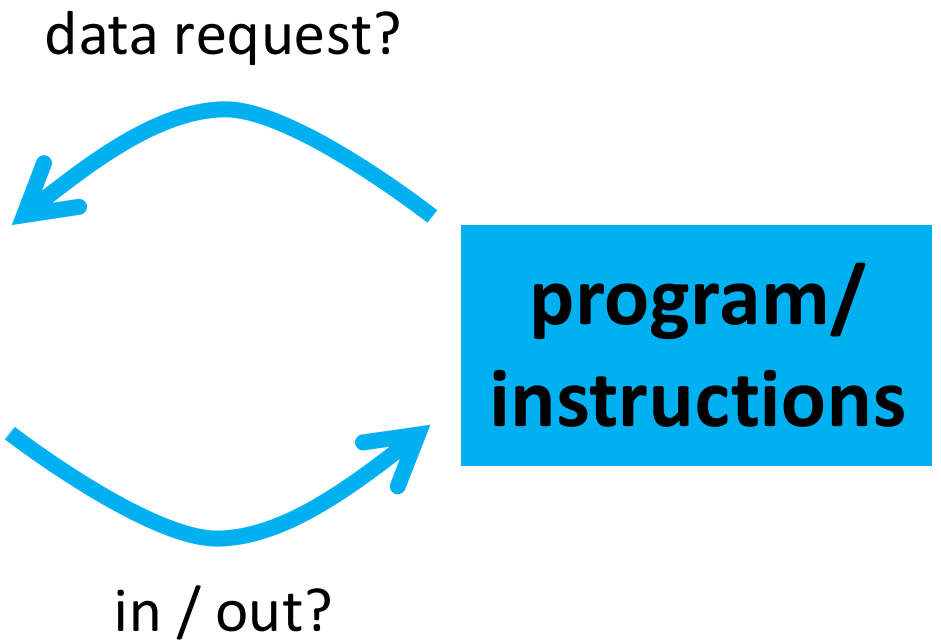
Memory Hierarchy



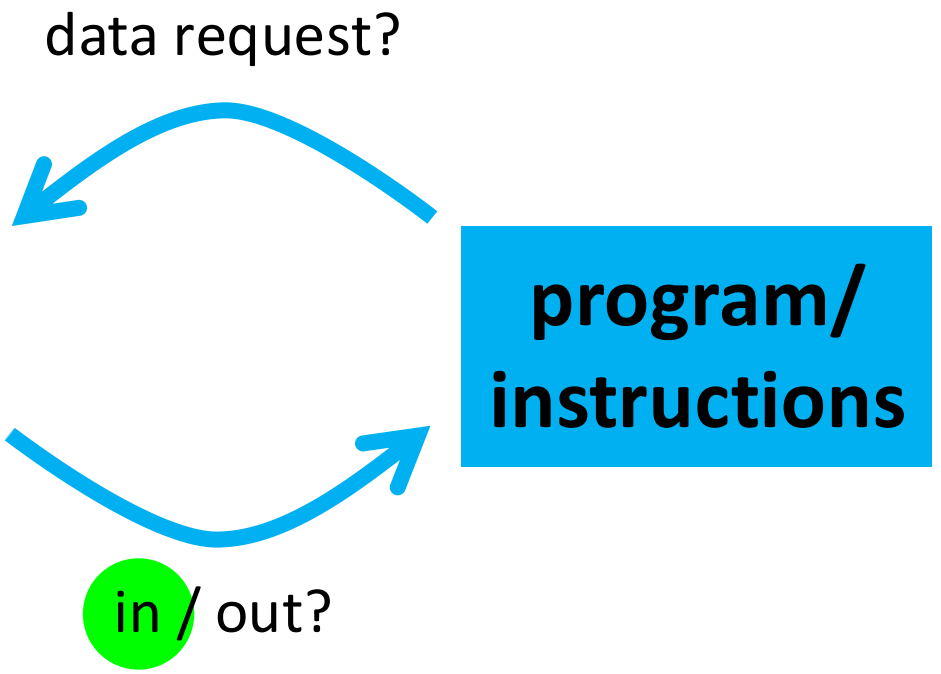
Memory hierarchy for server

Wait, but what's cache?

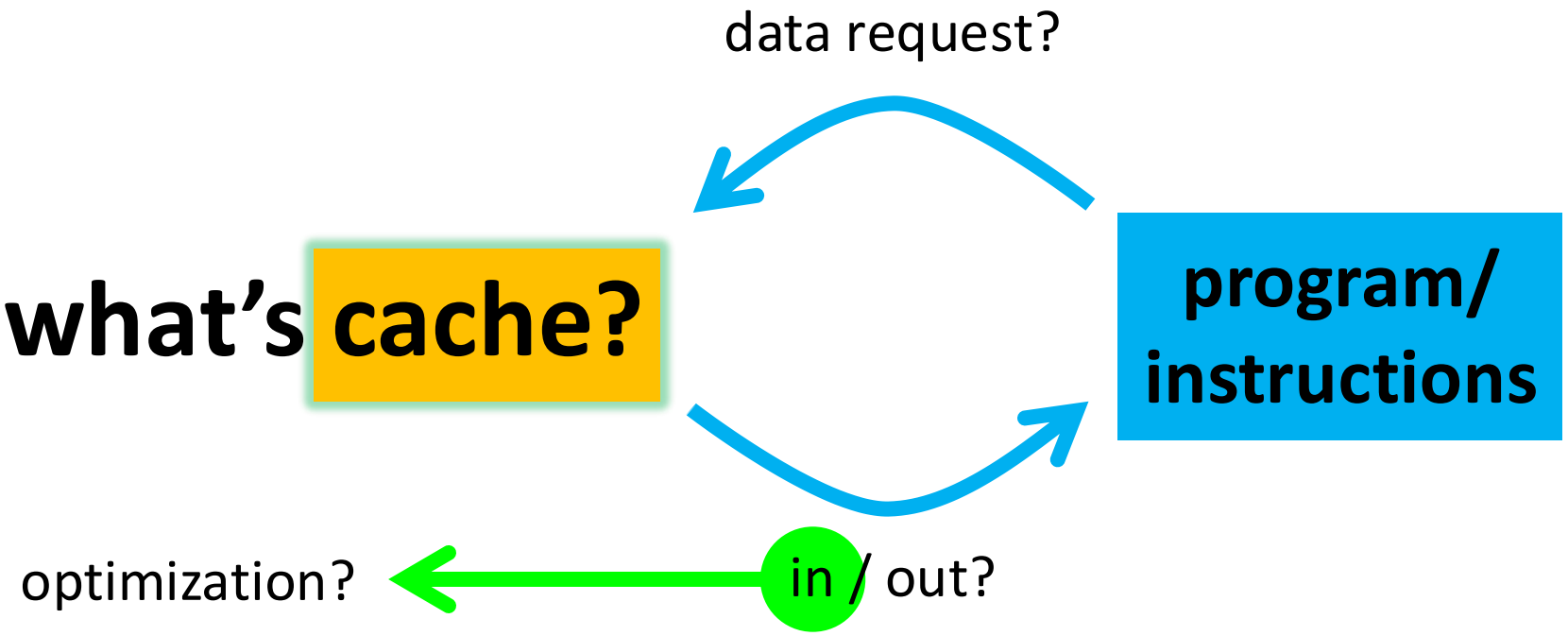
Wait, but what's cache?



Wait, but what's cache?



Wait, but what's cache?



So, what’s cache?

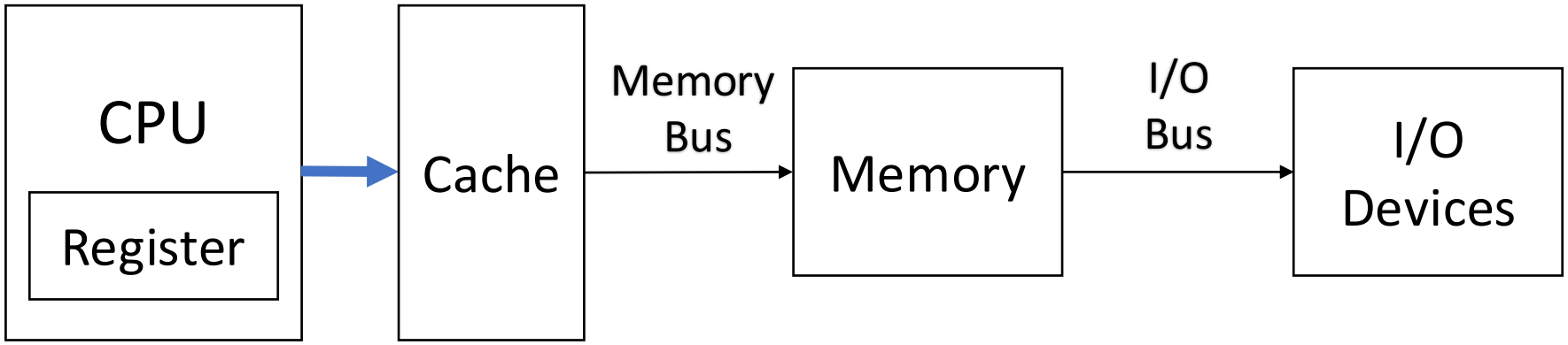
So, what's cache?

- **Cache: a safe place for hiding or storing things.**

Webster's New World Dictionary of the American Language

Second College Edition (1976)

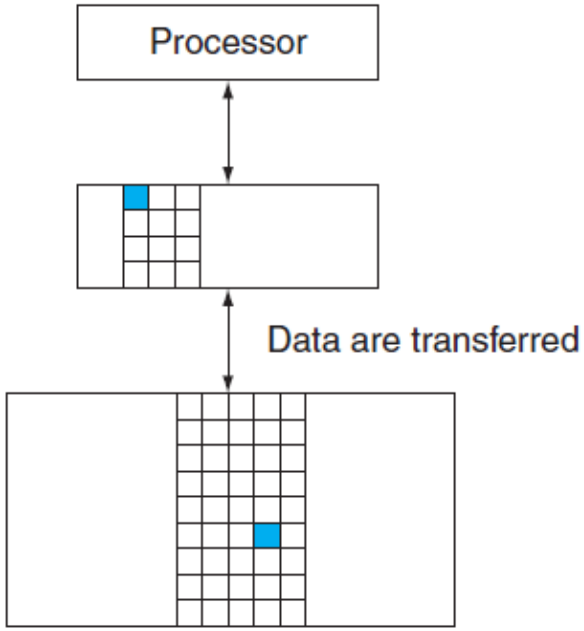
Cache



- The highest or first level of the memory hierarchy encountered once the *addr* leaves the processor
- Employ buffering to reuse commonly occurring items

Cache Hit/Miss

- When the processor can/cannot find a requested data item in the cache

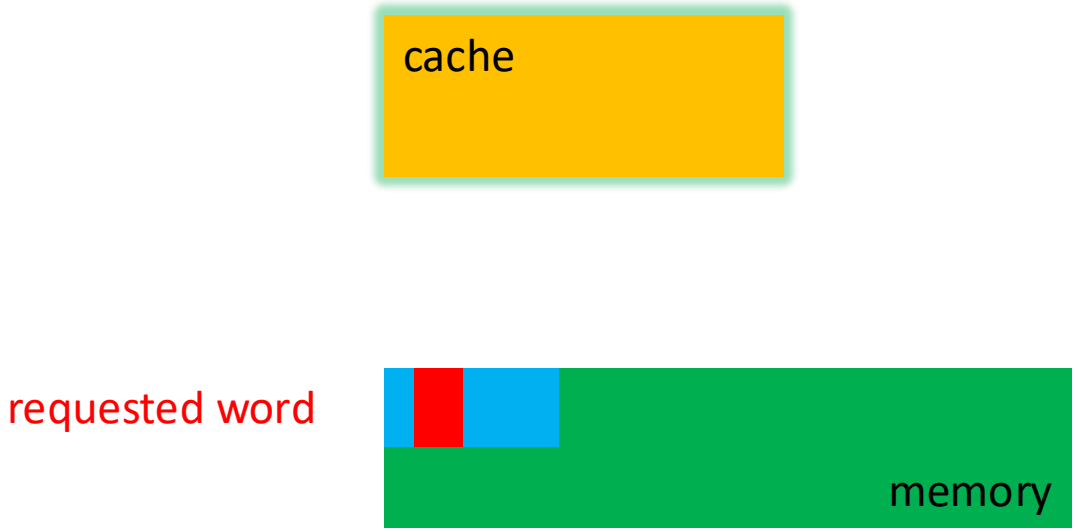


hit rate **miss rate**

hit time **miss penalty**

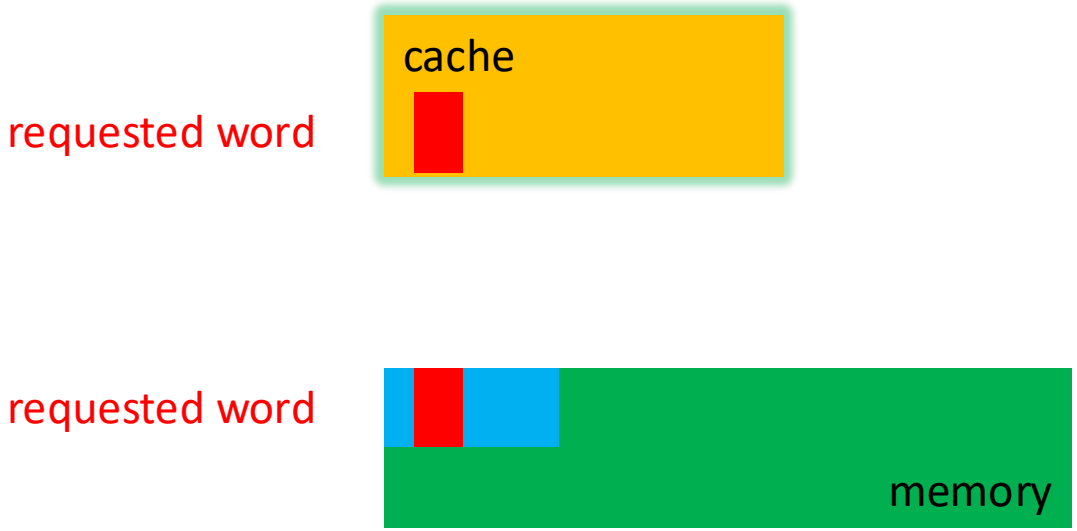
Block/Line Run

- A fixed-size collection of data containing the requested word, retrieved from the main memory and placed into the cache



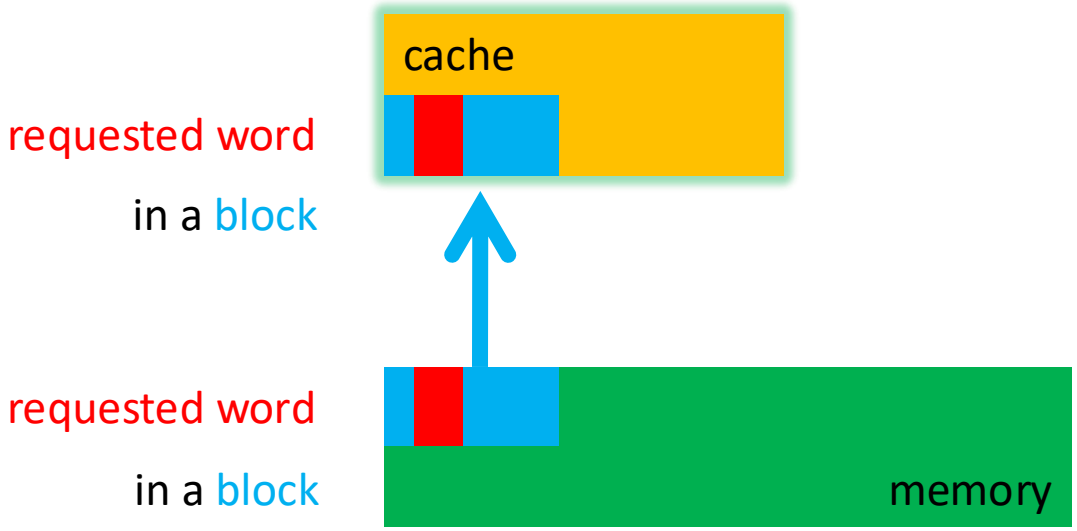
Block/Line Run

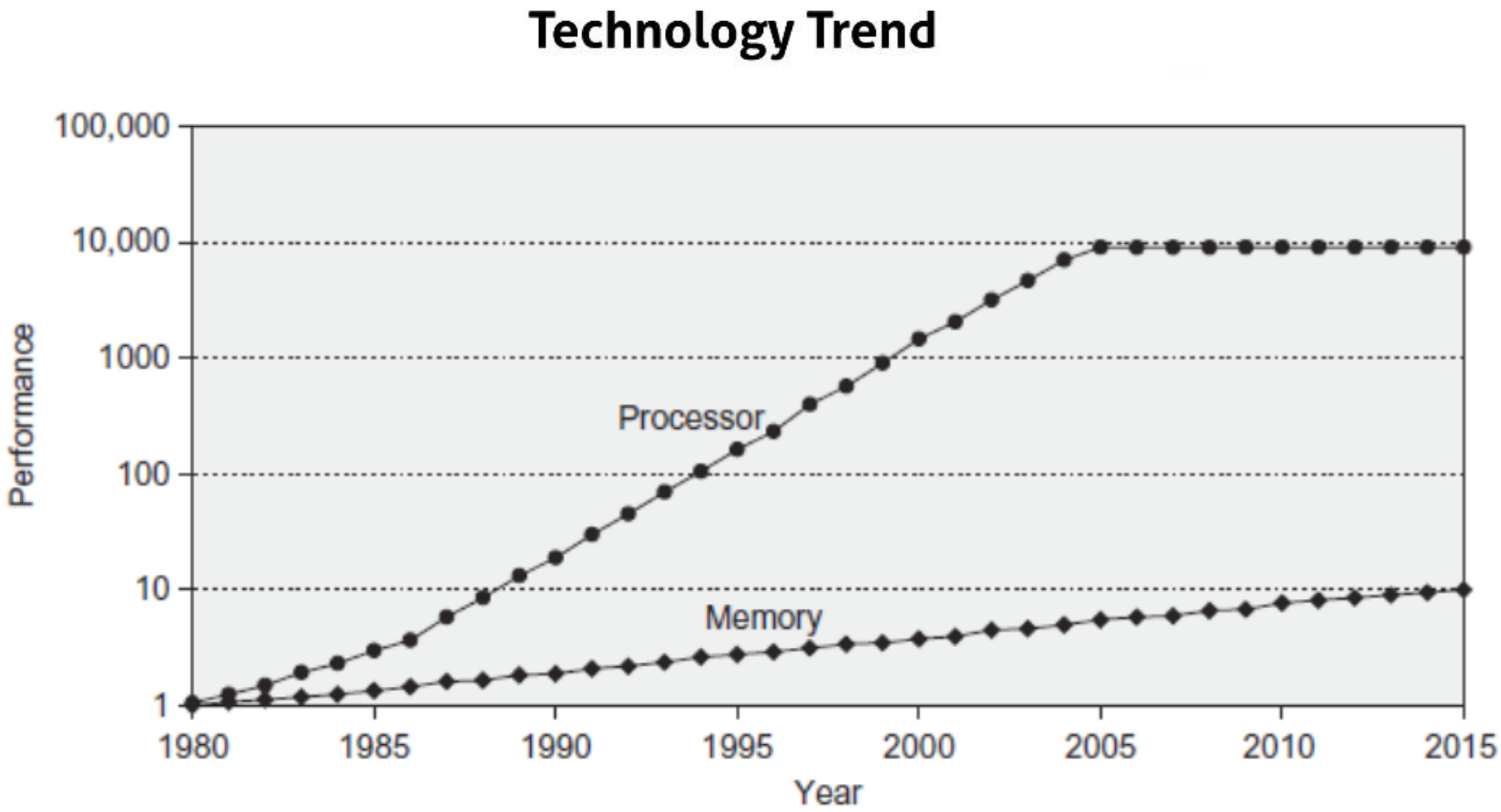
- A fixed-size collection of data containing the requested word, retrieved from the main memory and placed into the cache



Block/Line Run

- A fixed-size collection of data containing the requested word, retrieved from the main memory and placed into the cache




Technology Trend
Performance
100,000
10,000
1000
100
10
1
Processor
Memory
1980
1985
1990
1995
2000
2005
2010
2015
Year

§ 3.2 Technology Trend and Memory Hierarchy

Memory technology	Typical access time	$ per GiB in 2012
SRAM semiconductor memory	0.5–2.5 ns	$500–$1000
DRAM semiconductor memory	50–70 ns	$10–$20
Flash semiconductor memory	5,000–50,000 ns	$0.75–$1.00
Magnetic disk	5,000,000–20,000,000 ns	$0.05–$0.10

SRAM (*static random access memory*)

DRAM (*dynamic random access memory*)

SDRAM (Synchronous DRAM)

Double Data Rate (DDR) SDRAM

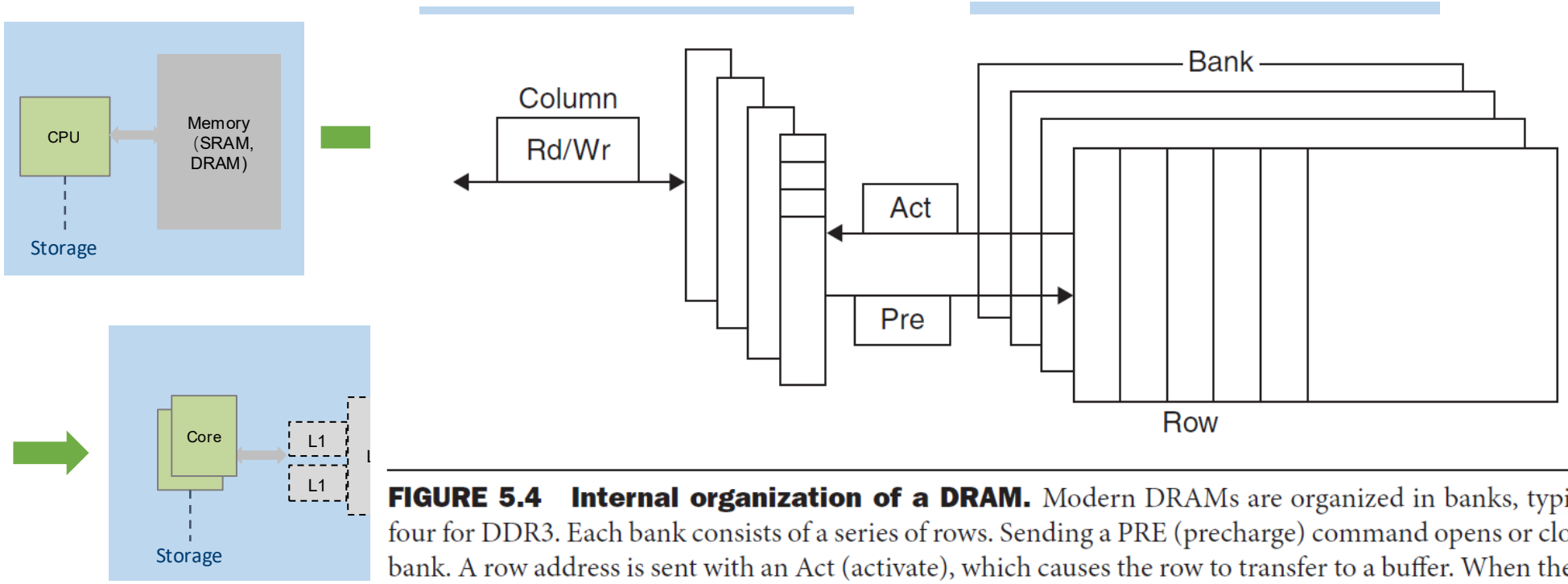
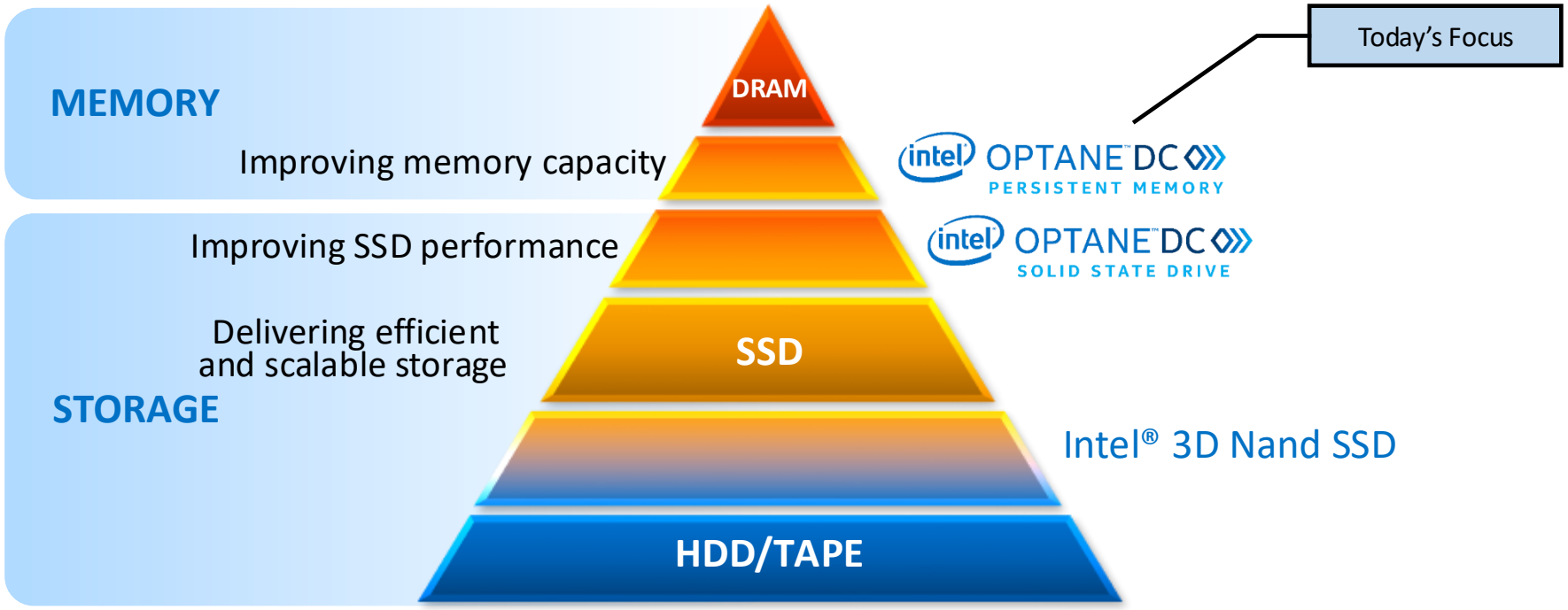


FIGURE 5.4 Internal organization of a DRAM. Modern DRAMs are organized in banks, typically four for DDR3. Each bank consists of a series of rows. Sending a PRE (precharge) command opens or closes a bank. A row address is sent with an Act (activate), which causes the row to transfer to a buffer. When the row is in the buffer, it can be transferred by successive column addresses at whatever the width of the DRAM is (typically 4, 8, or 16 bits in DDR3) or by specifying a block transfer and the starting address. Each command, as well as block transfers, is synchronized with a clock.

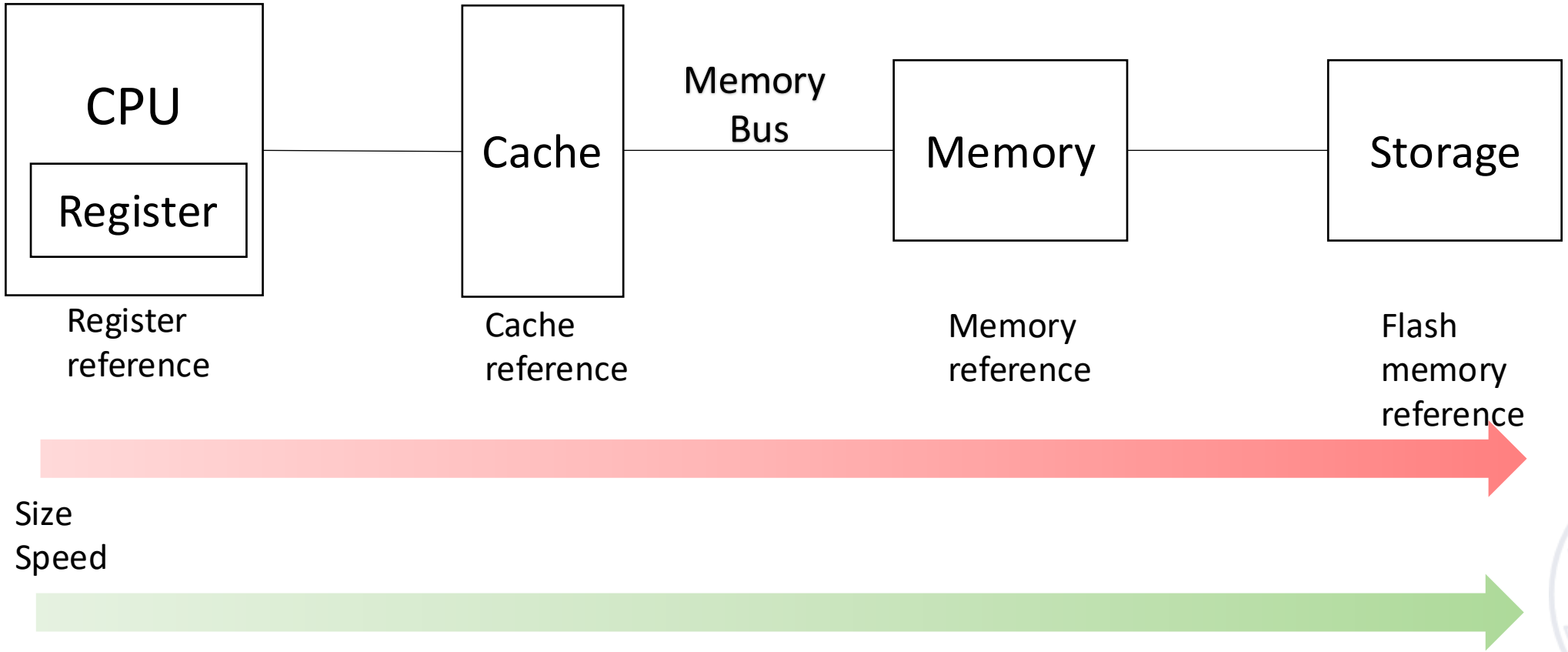
HBM (*High Bandwidth Memory)*

Memory/Storage



Flash Memory: electrically erasable programmable read-only memory (EEPROM).

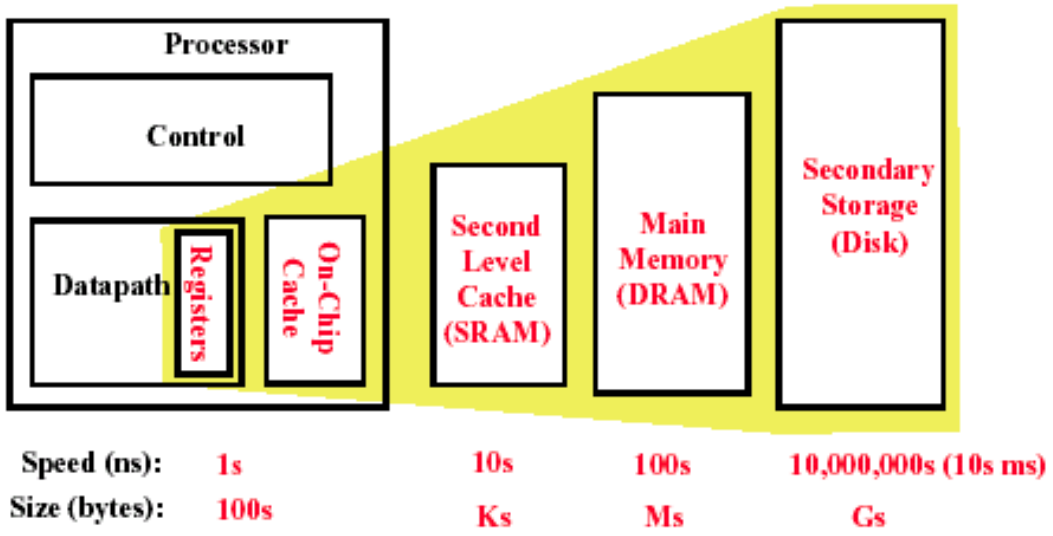
Processor-Memory Performance Gap



Memory Hierarchy of a Modern Computer System

By taking advantage of the principle of locality:

- Present the user with as much memory as is available in the cheapest technology.
- Provide access at the speed offered by the fastest technology.



Cache Locality

- **Temporal locality**

  need the requested word again soon

- **Spatial locality**

  likely need other data within the same block soon

Cache Miss

- Time required for cache miss depends on:
  - **Latency**: the time to retrieve the first word of the block
  - **Bandwidth**: the time to retrieve the rest of this block
- Causes of Cache misses
  - **Compulsory:** first reference to a block
  - **Capacity:** blocks discarded and later retrieved
  - **Conflict:** program makes repeated references to multiple addresses from different blocks that map to the same location in the cache

Enhance performance

By taking advantage of the principle of locality:

- **most programs do not access all code or data uniformly**
- Temporal Locality (Locality in Time):
- If an item is referenced, the same item will tend to be referenced again soon
  - Keep most recently accessed data items closer to the processor
- Spatial Locality (Locality in Space):
- If an item is referenced, nearby items will tend to be referenced soon
  - Move recently accessed groups of contiguous words(block) closer to processor.

Three classes of computers with different concerns in memory hierarchy

- **Desktop computers:**
  - Are primarily running one application for single user
  - Are concerned more with average latency from the memory hierarchy.

- **Server computers:**
  - May typically have hundreds of users running potentially dozens of applications simultaneously.
  - Are concerned about memory bandwidth.

Three classes of computers with different concerns in memory hierarchy

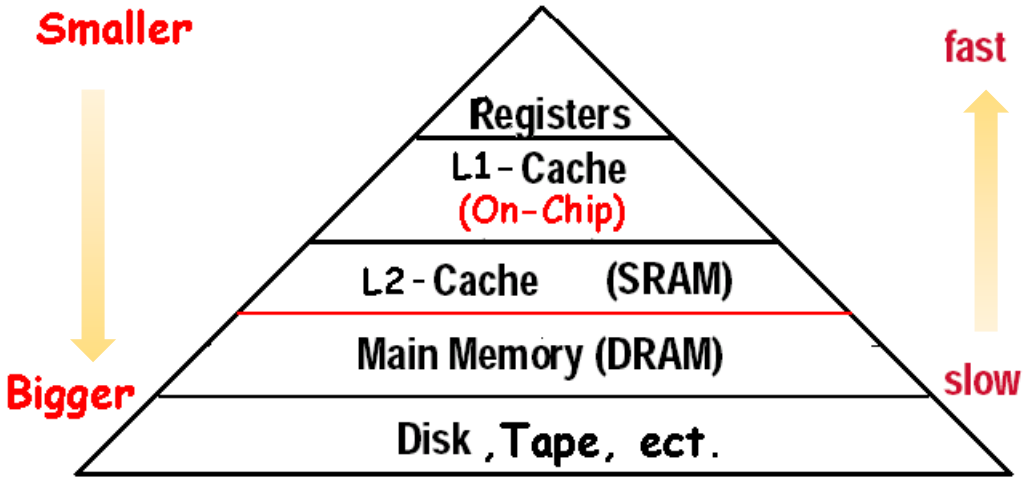
- **Embedded computers:**
  - Real-time applications.
    - Worst-case performance vs Best case performance
  - Are concerned more about power and battery life.
    - Hardware vs software
  - Running single app & use simple OS
    - The protection role of the memory hierarchy is often diminished.
  - Main memory is very small
    - often no disk storage

36 terms of Cache

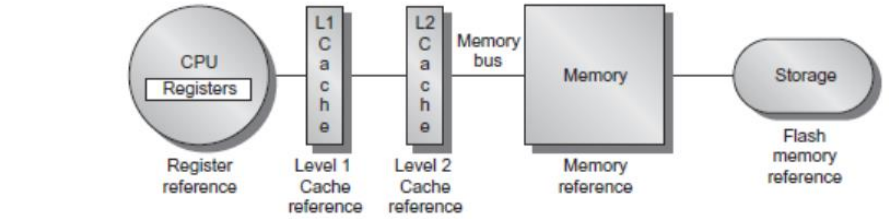
Cache	full associative	write allocate
Virtual memory	dirty bit	unified cache
Memory stall cycles	block	block offset
misses per instruction	direct mapped	write back
Valid bit	data cache	locality
Block address	hit time	address trace
Write through	cache miss	set
Instruction cache	page fault	miss rate
random replacement	index field	cache hit
Average memory access time	page	tag field
n-way set associative	no-write allocate	miss penalty
Least-recently used	write buffer	write stall

What is a cache?

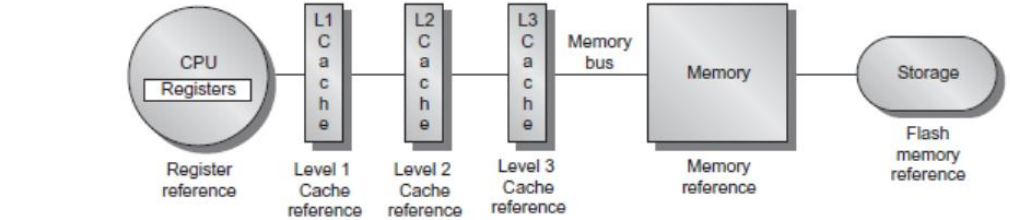
- Small, fast storage used to improve average access time to slow memory.
- In computer architecture, almost everything is a cache!
  - Registers “a cache” on variables – software managed
  - First-level cache a cache on second-level cache
  - Second-level cache a cache on memory
  - Memory a cache on disk (virtual memory)
  - TLB a cache on page table
  - Branch-prediction a cache on prediction information?



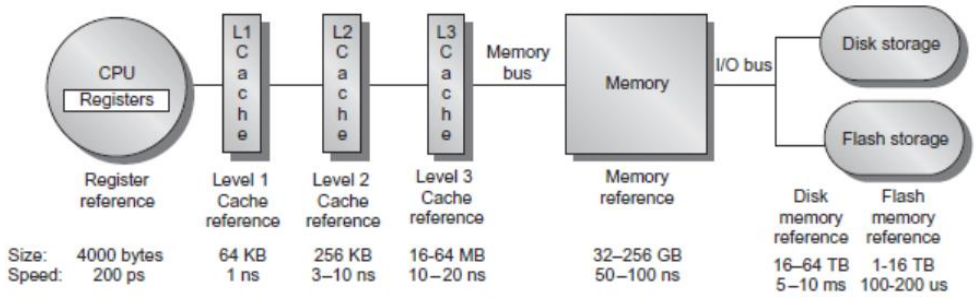
§ 3.2 Technology Trend and Memory Hierarchy



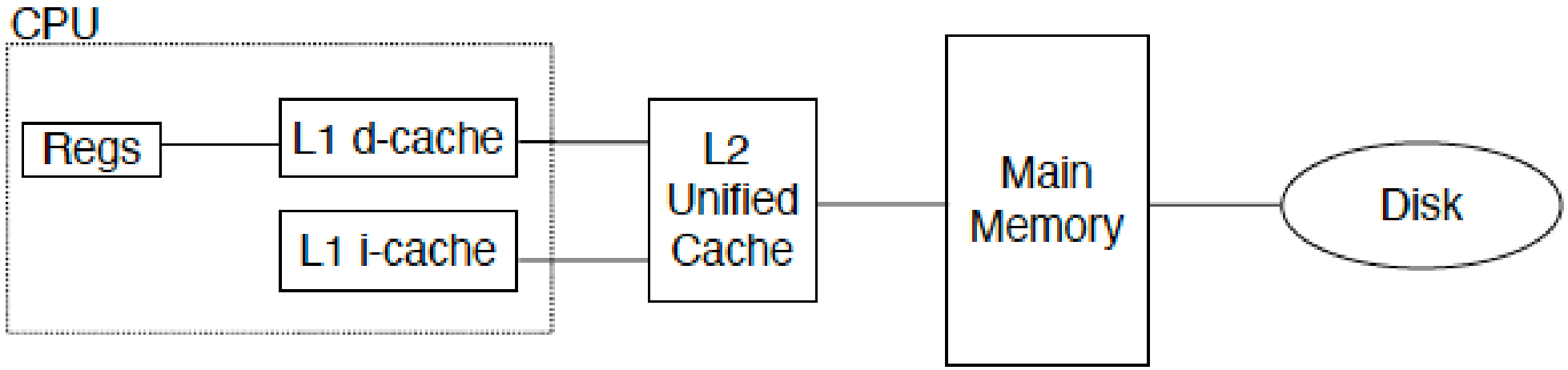
(A) Memory hierarchy for a personal mobile device



(B) Memory hierarchy for a laptop or a desktop



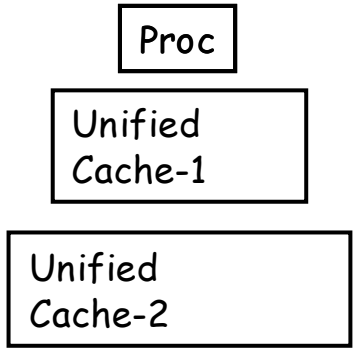
(C) Memory hierarchy for server

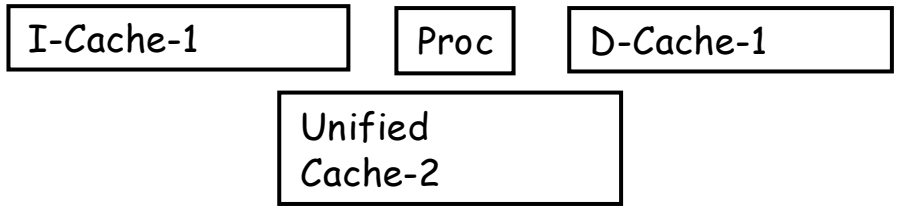


A typical multi-level cache organization

Split vs. unified caches

- Unified cache
  - All memory requests go through a single cache.
  - This requires less hardware, but also has lower performance
- Split I & D cache
  - A separate cache is used for instructions and data.
  - This uses additional hardware, though there are some simplifications (the I cache is read-only).

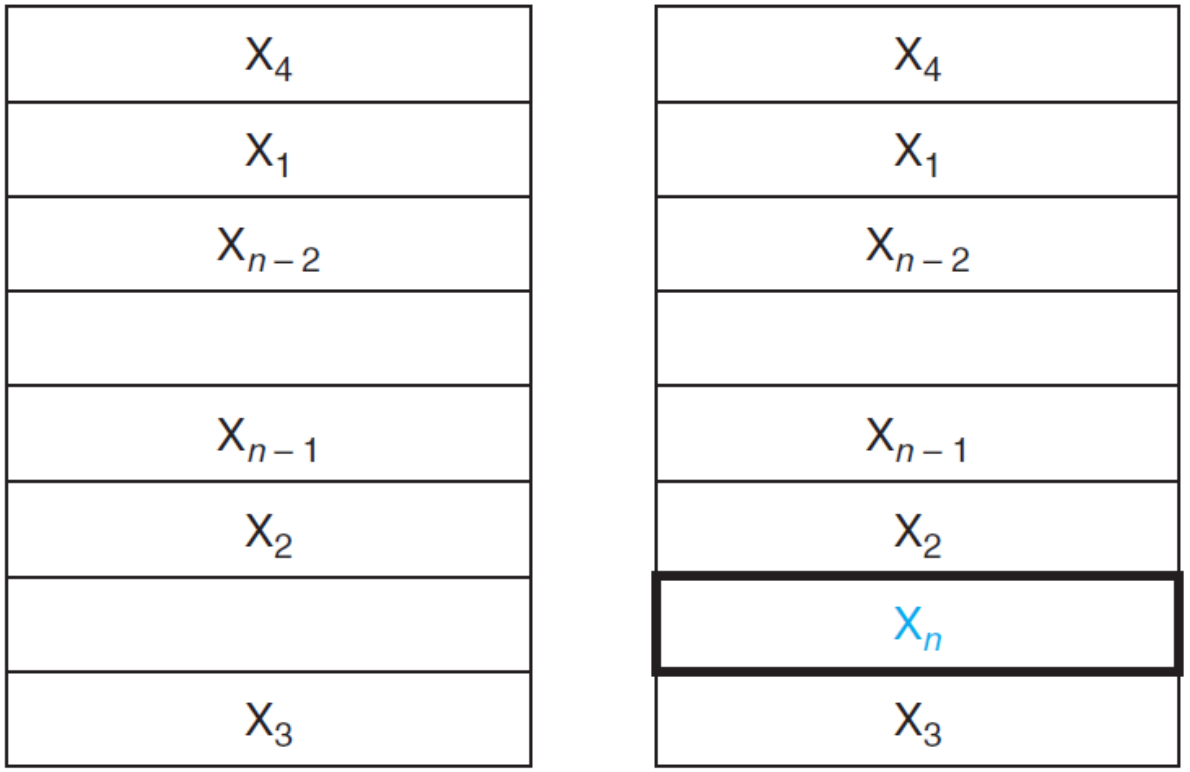




Cache type	Associativity (E)	Block size (B)	Sets (S)	Cache size (C)
on-chip L1 i-cache	4	32 B	128	16 KB
on-chip L1 d-cache	4	32 B	128	16 KB
off-chip L2 unified cache	4	32 B	1024–16384	128 KB–2 MB

Intel Pentium cache organization

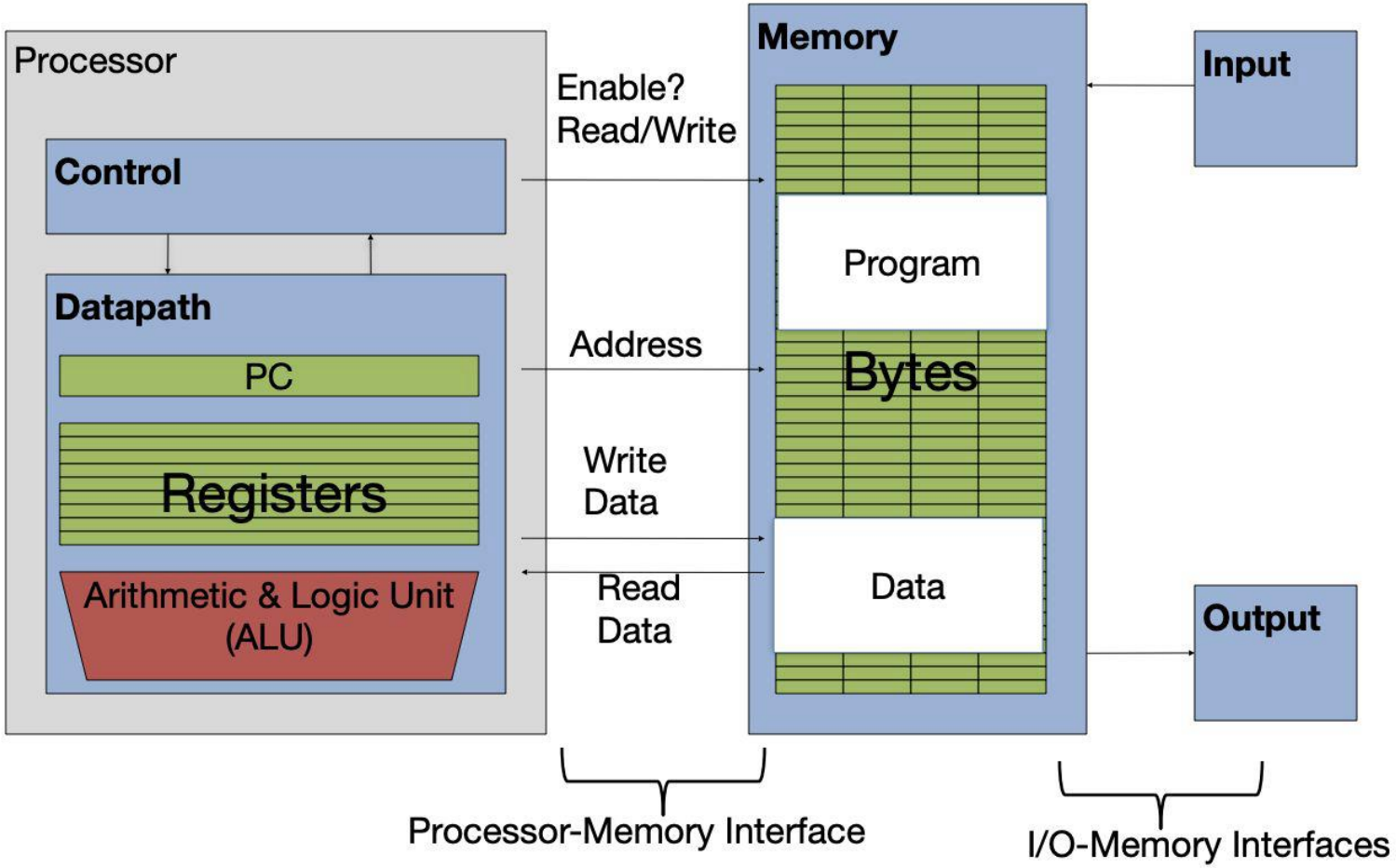
The cache just before and just after a reference to a word Xn that is not initially in the cache.

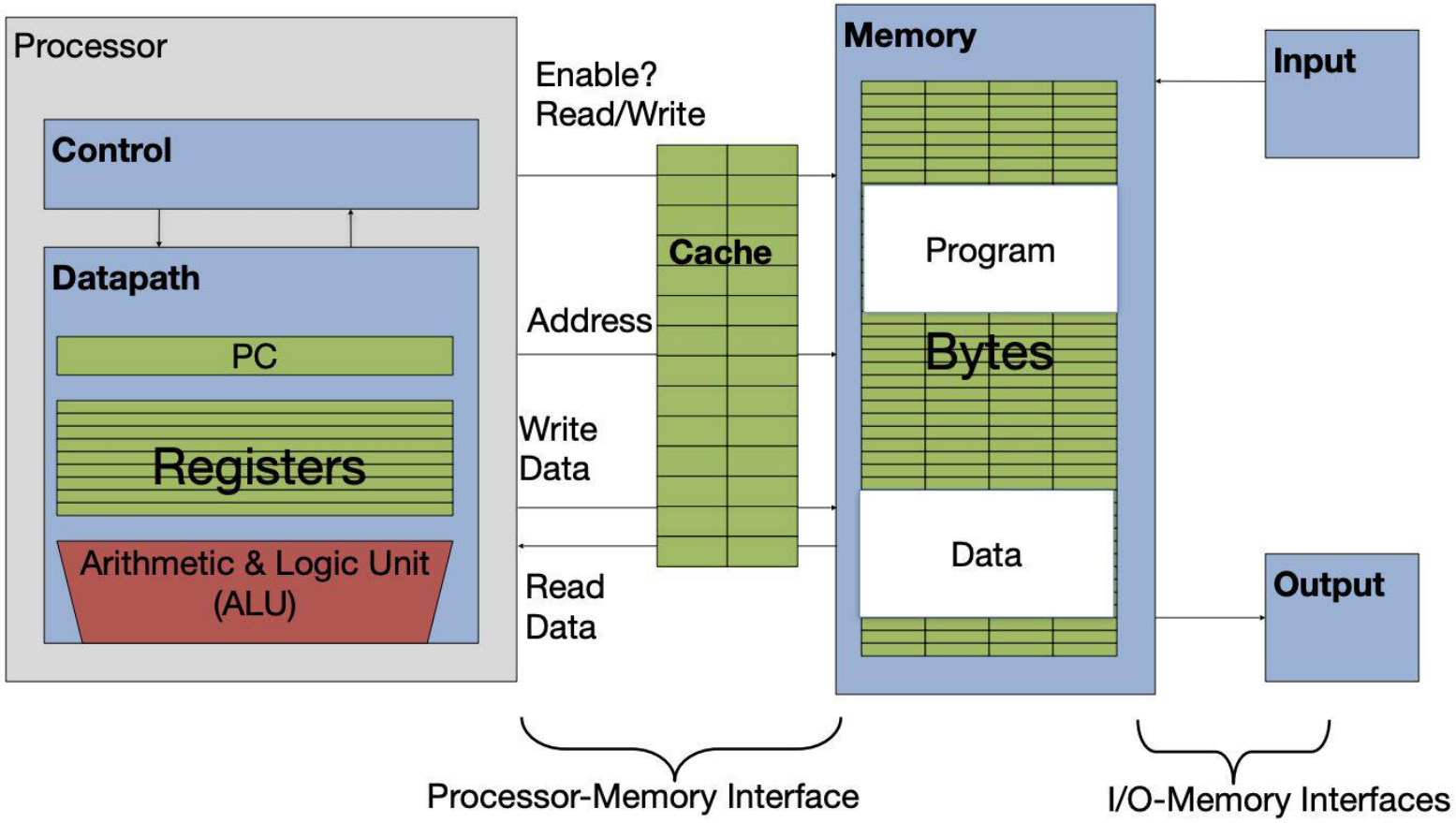


a. Before the reference to X_n b. After the reference to X_n

This reference causes a miss that forces the cache to fetch Xn from memory and insert into the cache.

How do we know if a data item is in the cache? Moreover, if it is, how do we find it?


Processor
Control
Datapath
PC
Registers
Arithmetic & Logic Unit (ALU)
Enable?
Read/Write
Address
Write Data
Read Data
Memory
Program
Bytes
Data
Input
Output
Processor-Memory Interface
I/O-Memory Interfaces



Lines (i.e., blocks) of data are copied from memory to the cache.

Memory Access without Cache

- Load word instruction: lw t0, 0(t1)
- t1 contains 1022_{10}, Memory[1022]=99
  - Processor issues address 1022_{10} to Memory
  - Memory reads word at address 1022_{10}(99)
  - Memory sends 99 to Processor
  - Processor loads 99 into register t0

Memory Access with Cache

- Load word instruction: lw t0, 0(t1)
- t1 contains 1022_{10}, Memory[1022]=99
- With cache (similar to a hash)
  - Processor issues address 1022_{10} to Cache
  - Cache checks to see if has copy of data at address 1022_{10}
    - If finds a match (cache hit): cache reads 99, sends to process
    - Not match(cache miss):cache sends address to 1022_{10} Memory
      - Memory reads 99 at address 1022_{10}
      - Memory sends 99 to Cache
      - Cache replaces word with new 99
      - Cache sends 99 into process
  - Processor loads 99 into register t0

Memory Access with Cache

- When reading memory, three things can happen:
- Cache hit
  - Cache block is valid and contains proper address, so read desired word
- Cache miss
  - Nothing in cache in appropriate block, so fetch from memory
- Cache miss, block replacement
  - Wrong data is in cache at appropriate block, so discard it and fetch desired data from memory (cache always copy)

Four Questions for Cache Designers

Caching is a general concept used in processors, operating systems, file systems, and applications.

There are **Four Questions** for Cache/Memory Hierarchy Designers

- Q1: Where can a block be placed in the upper level/main memory?

  (Block placement)

  - Fully Associative, Set Associative, Direct Mapped
- Q2: How is a block found if it is in the upper level/main memory?

  (Block identification)

  - Tag/Block
- Q3: Which block should be replaced on a Cache/main memory miss?

  (Block replacement)

  - Random, LRU,FIFO
- Q4: What happens on a write?

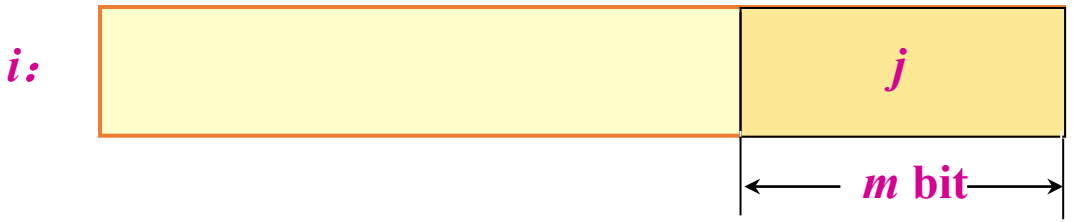
  (Write strategy)

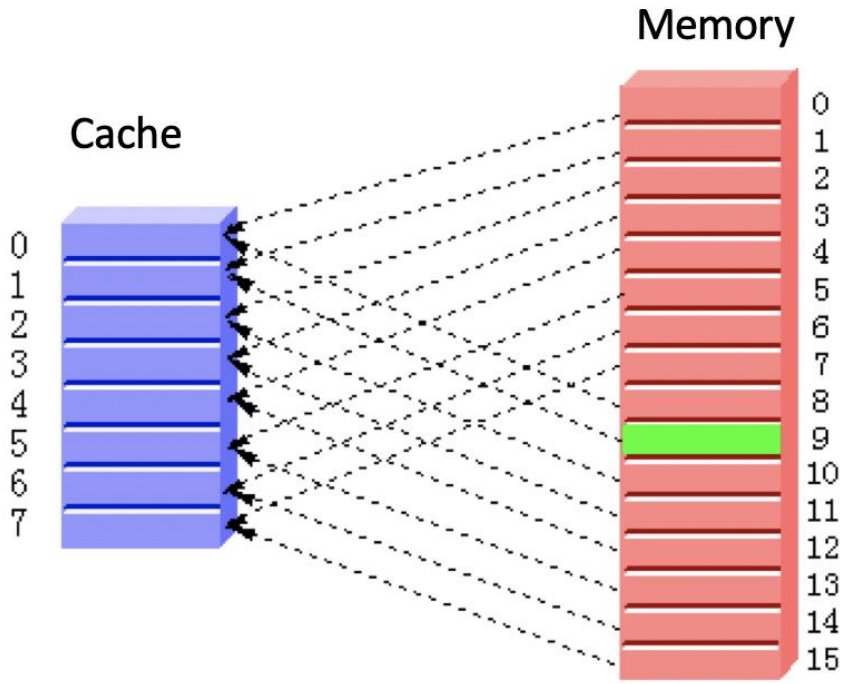
  - Write Back or Write Through (with Write Buffer)

Q1: Block Placement

Direct mapped

(Block address) modulo (Number of blocks in the cache)



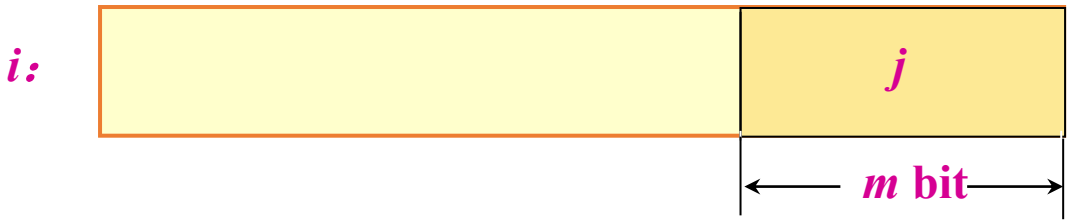


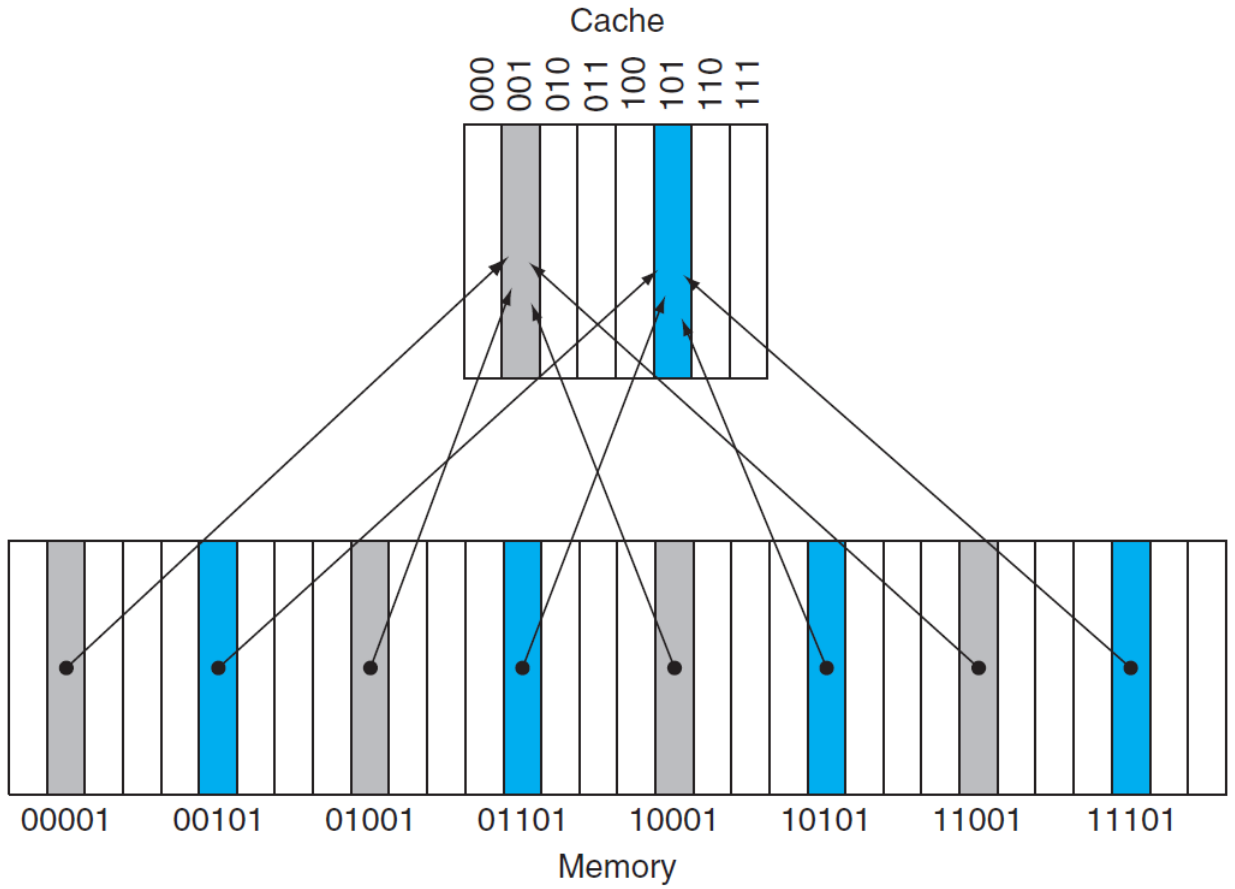
A cache structure in which each memory location is mapped to exactly one location in the cache.

Q1: Block Placement

Direct mapped

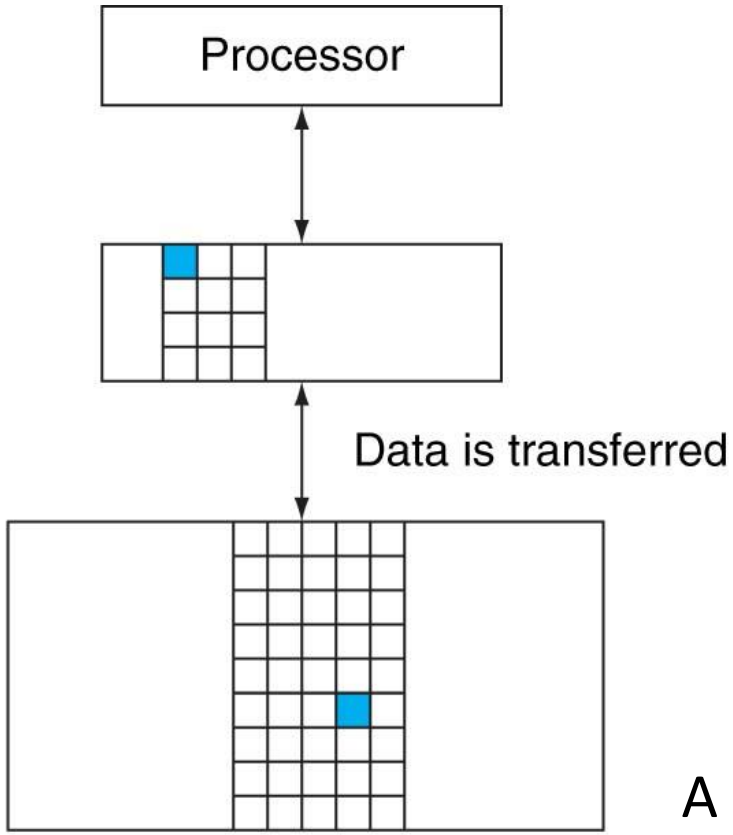
(Block address) modulo (Number of blocks in the cache)

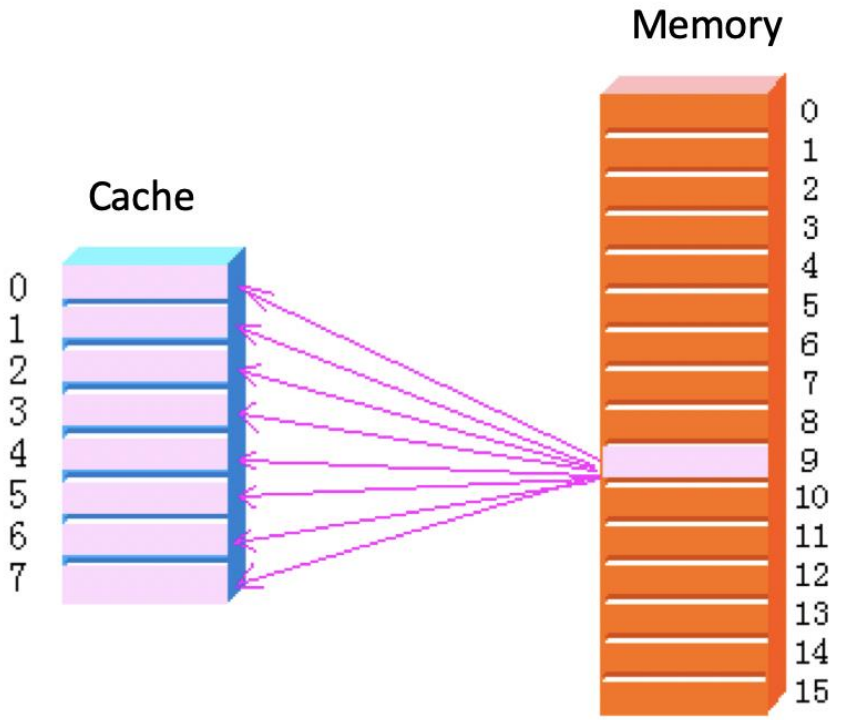




A cache structure in which each memory location is mapped to exactly one location in the cache.

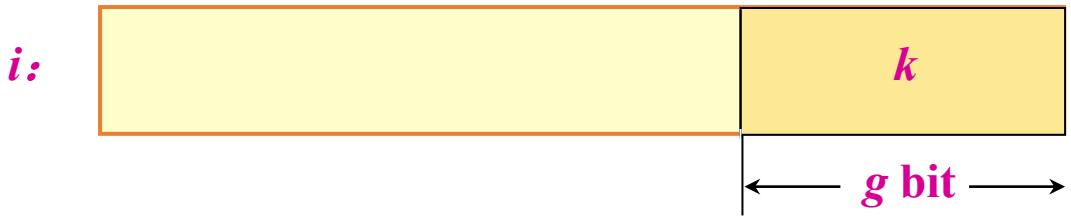
Fully-associative

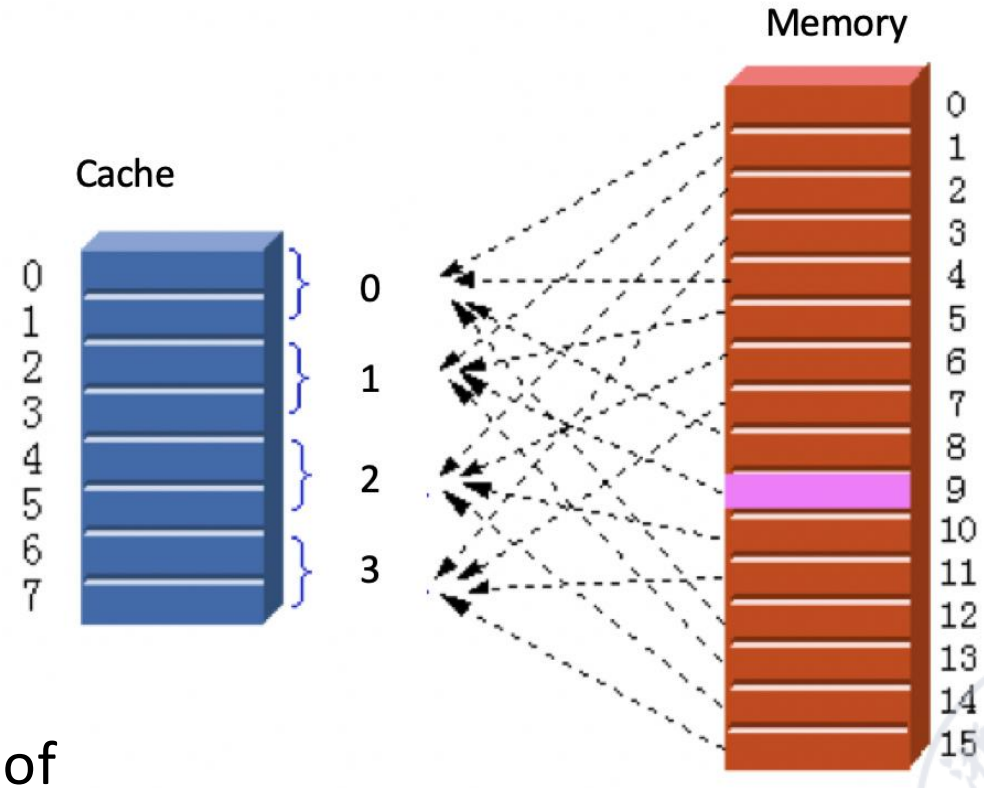




A cache structure in which a block can be placed in any location in the cache.

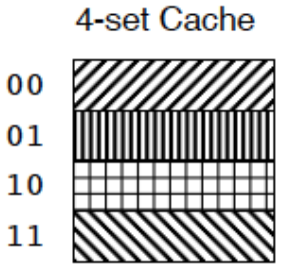
2-way Set-associative





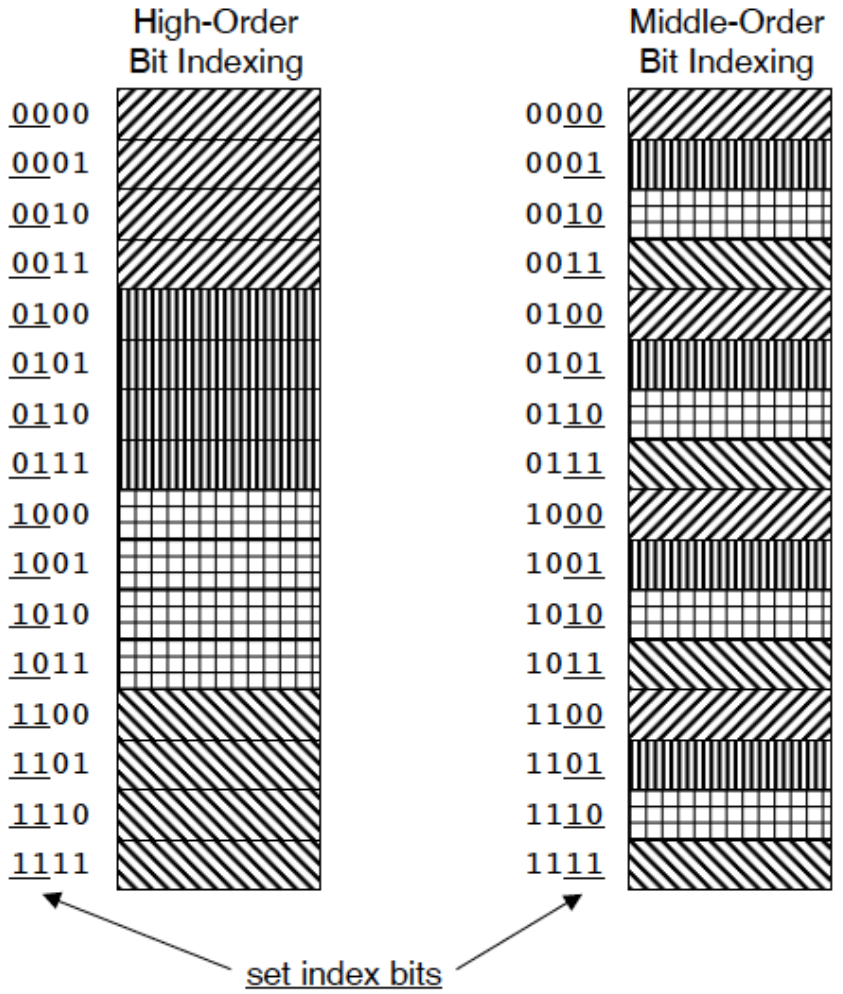
A cache that has a fixed number of locations (at least two) where each block can be placed.

4-way Set-associative



Why Index With the Middle Bits?

If the high-order bits are used as an index, then some contiguous memory blocks will map to the same cache set.



Q1: Block Placement

- Direct mapped
  - Block
- Fully ass
- Set associative
  - Block can go in one of a set of places in the cache.
  - A set is a group of blocks in the cache.

    Block address MOD Number of *sets* in the cache
  - If sets have n blocks, the cache is said to be n-way set associative.

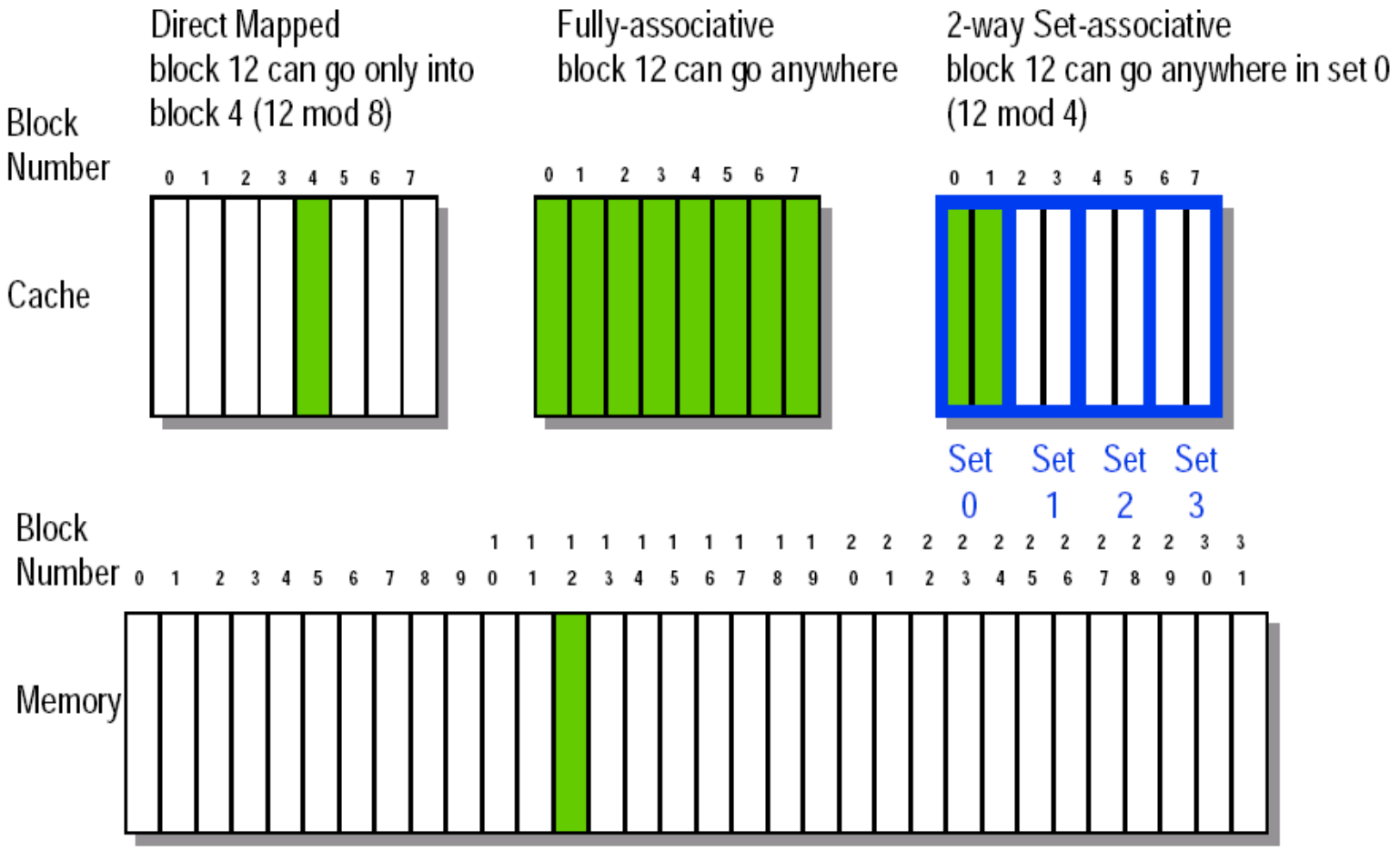
- *Note that direct mapped is the same as 1-way set associative, and fully associative is m-way set-associative (for a cache with m blocks).*

n sets => *n-way set associative*

Direct-mapped cache => one block per set

Fully associative => one set

8-32 Block Placement



N-way Set-associative

- The higher the degree of association, the higher the utilization of cache space, the lower the probability of block collision and the lower the failure rate.

	n	G
Full-associative	***M***	***1***
Direct mapped	***1***	***M***
Set-associative	***1 < n < M***	***1 < G < M***

- Most Cache: $n \leq 4$
- Question： Is the greater the number n, the better?

Four Questions for Cache Designers

Caching is a general concept used in processors, operating systems, file systems, and applications.

There are **Four Questions** for Cache/Memory Hierarchy Designers

- Q1: Where can a block be placed in the upper level/main memory?

  (Block placement)

  - Fully Associative, Set Associative, Direct Mapped
- Q2: How is a block found if it is in the upper level/main memory?

  (Block identification)

  - Tag/Block
- Q3: Which block should be replaced on a Cache/main memory miss?

  (Block replacement)

  - Random, LRU,FIFO
- Q4: What happens on a write?

  (Write strategy)

  - Write Back or Write Through (with Write Buffer)

Q2: Block Identification

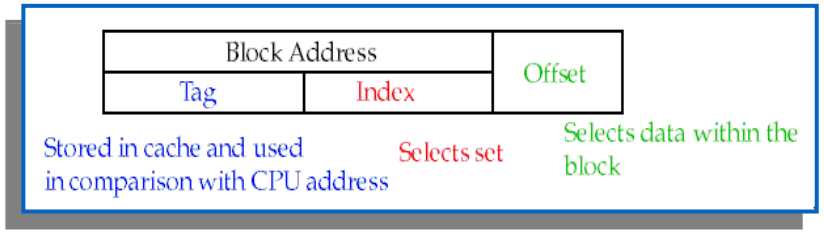
- Every block has an **address tag** that stores the main memory address of the data stored in the block.

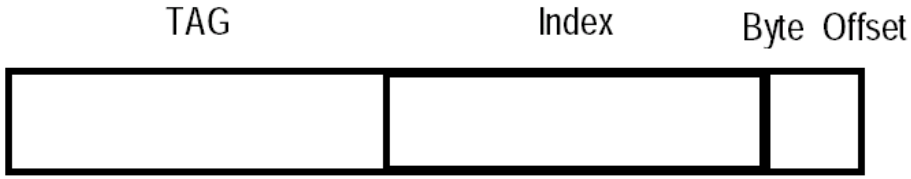
- When checking the cache, the processor will **compare** the requested **memory address to the cache tag** -- if the two are equal, then there is a cache hit and the data is present in the cache

- Often, each cache block also has a **valid bit** that tells if the contents of the cache block are valid

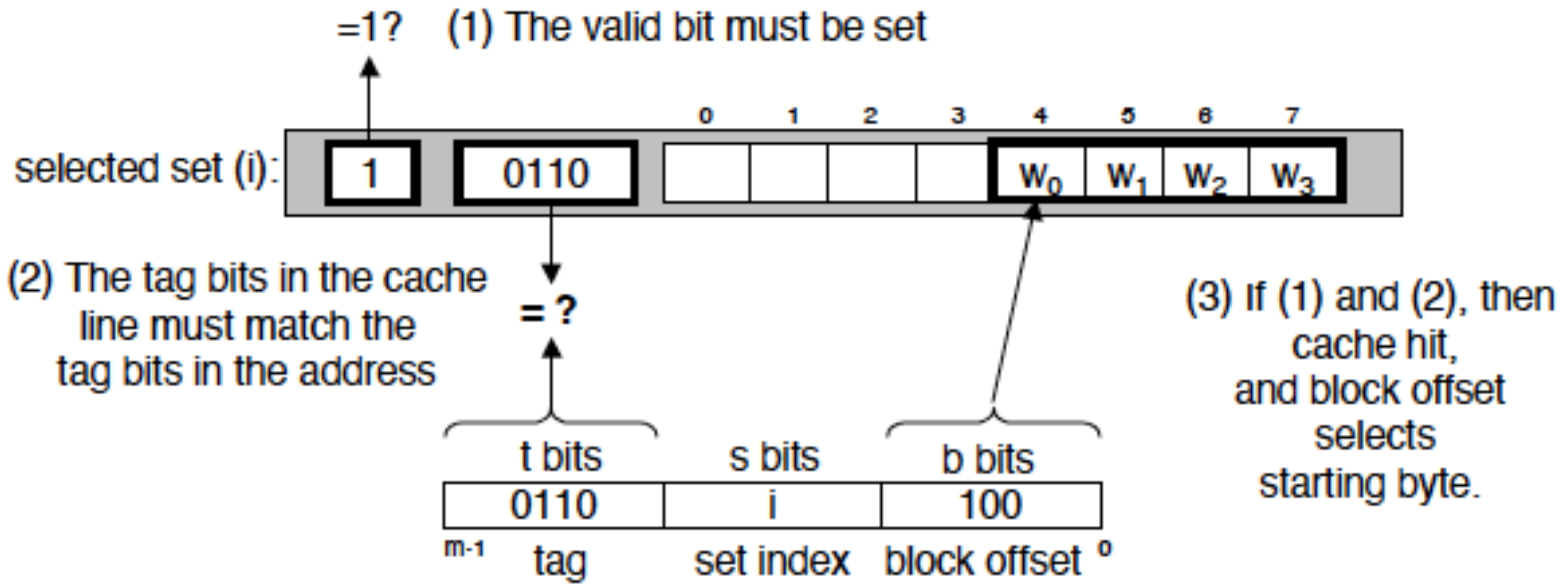
The Format of the Physical Address

- The Index field selects
  - The set, in case of a set-associative cache
  - The block, in case of a direct-mapped cache
  - Has as many bits as log2(#sets) for set-associative caches, or log2(#blocks) for direct-mapped caches
- The Byte Offset field selects
  - The byte within the block
  - Has as many bits as log2(size of block)
- The Tag is used to find the matching block within a set or in the cache
  - Has as many bits as Address_size – Index_size – Byte_Offset_Size





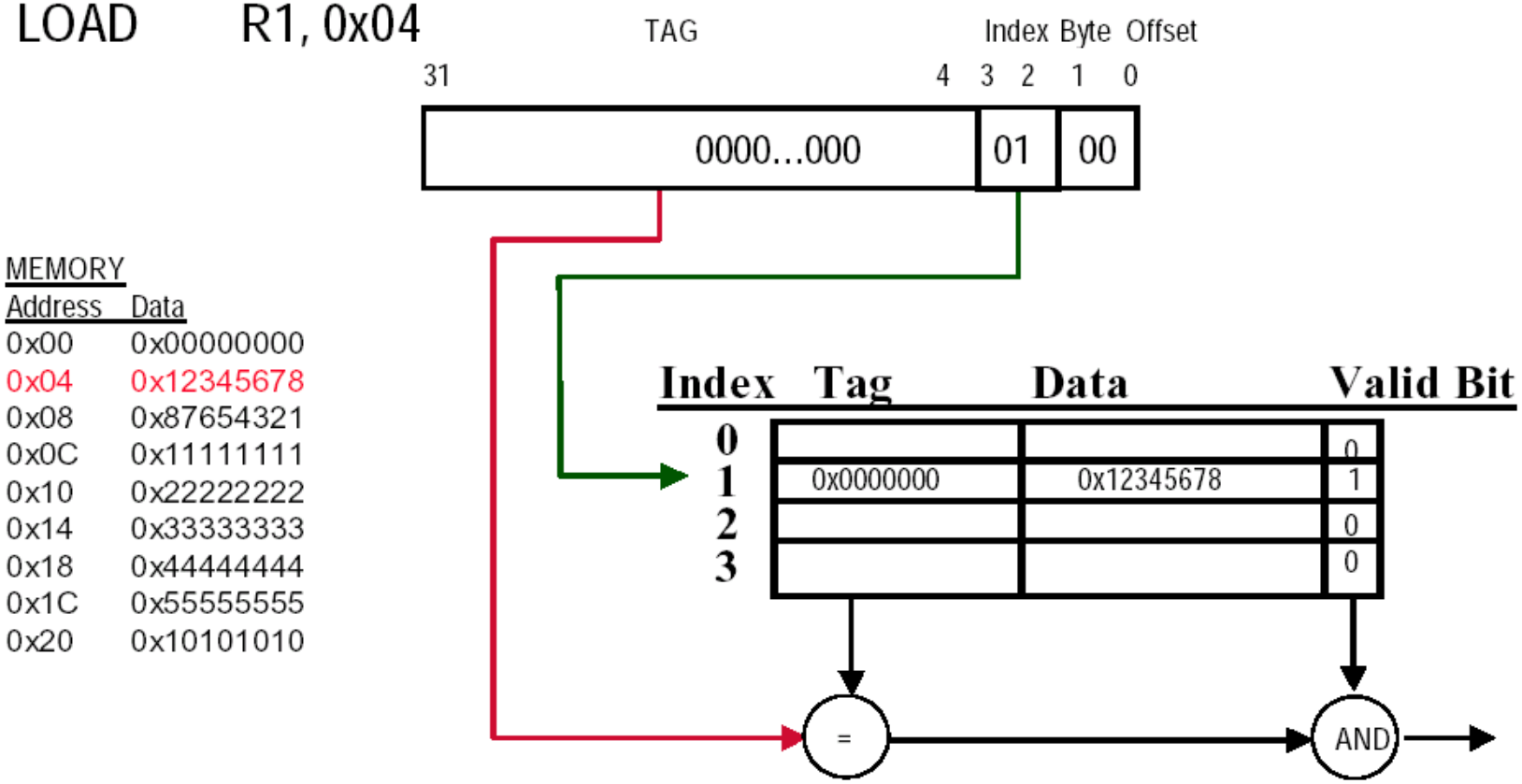
Line Matching and Word Selection



direct-mapped cache

Direct-mapped Cache Example (1-word Blocks)

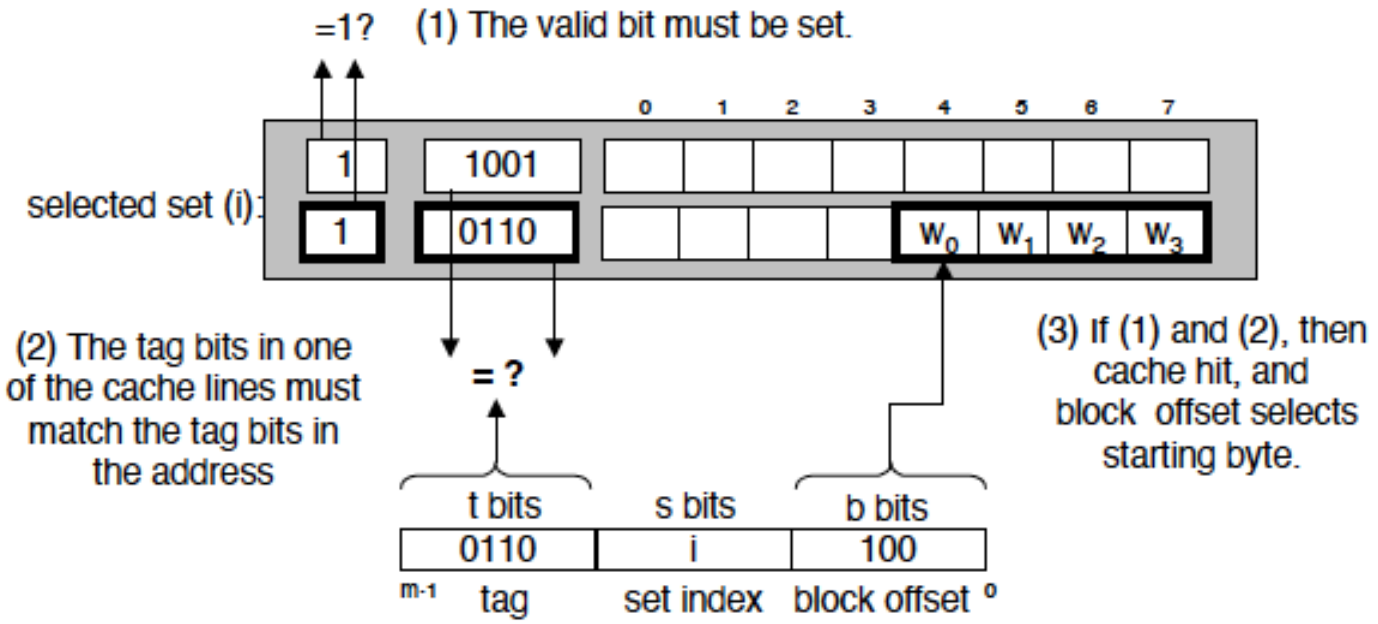
LOAD R1, 0x04



MEMORY

Address	Data
0x00	0x00000000
0x04	0x12345678
0x08	0x87654321
0x0C	0x11111111
0x10	0x22222222
0x14	0x33333333
0x18	0x44444444
0x1C	0x55555555
0x20	0x10101010

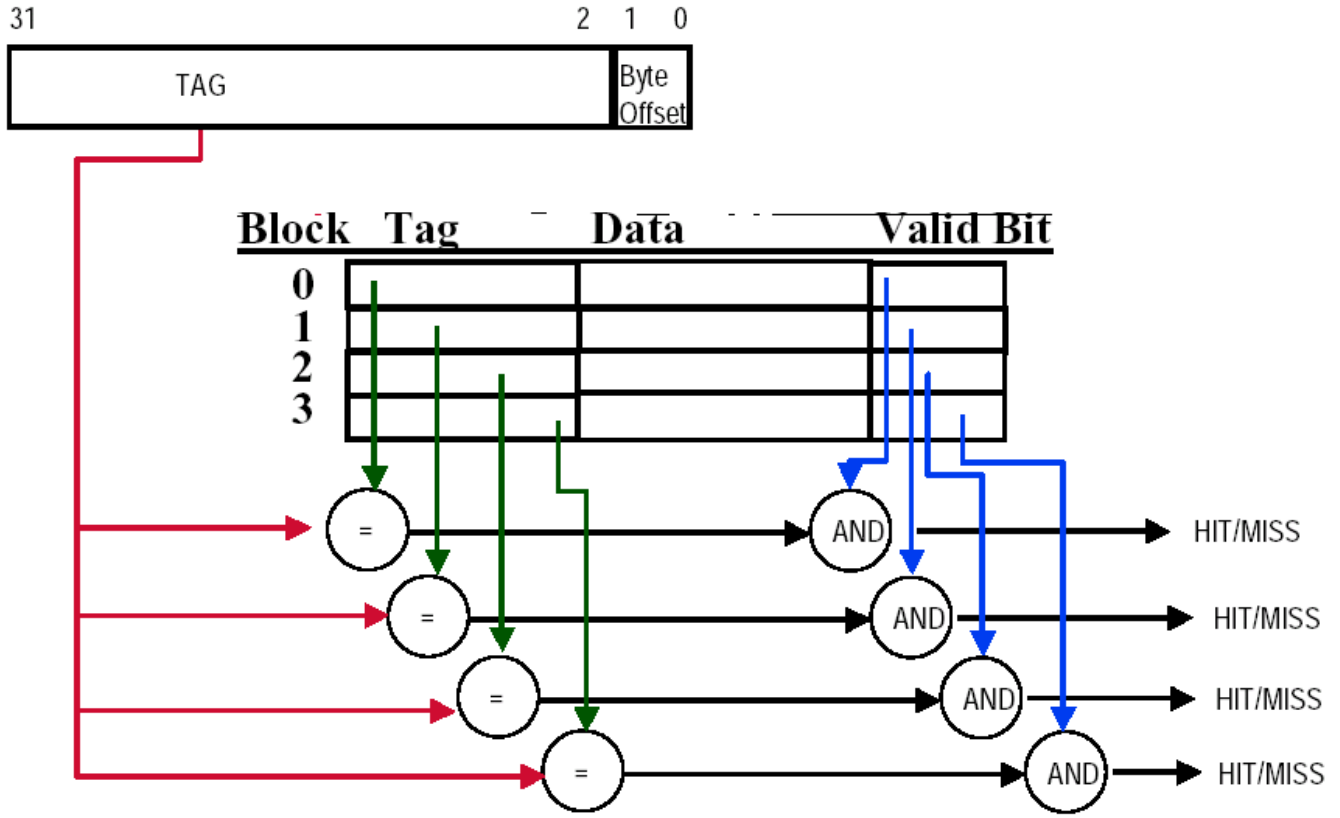
Line Matching and Word Selection



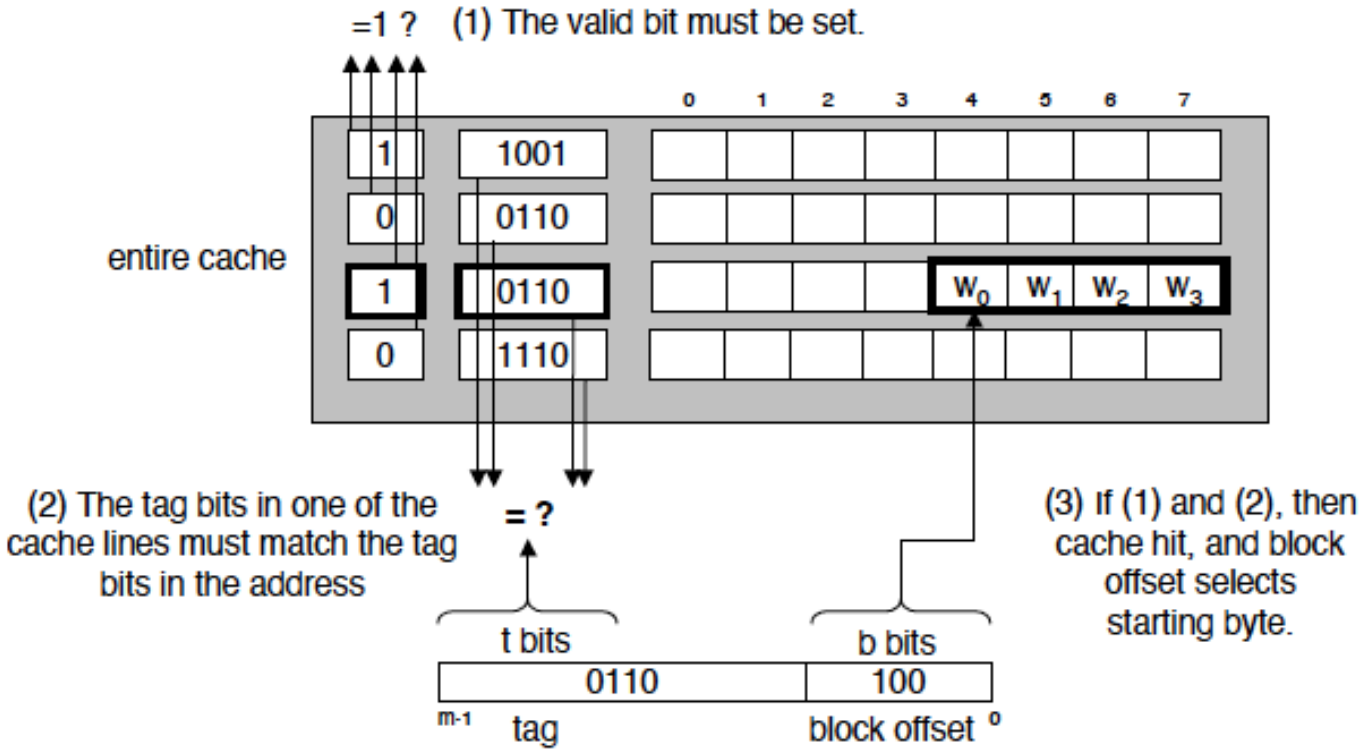
set associative cache

Fully-Associative Cache example (1-word Blocks)

- Assume cache has 4 blocks



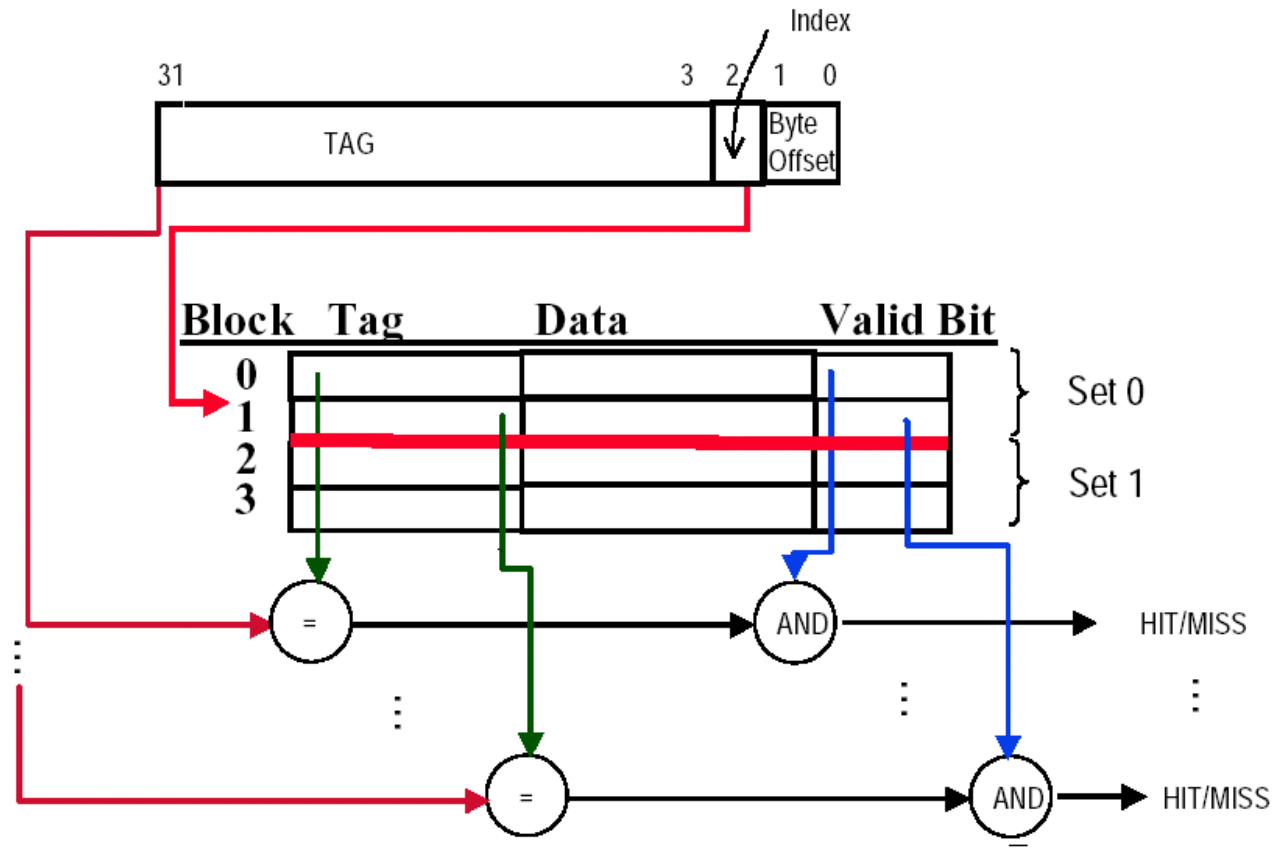
Line Matching and Word Selection



fully associative cache

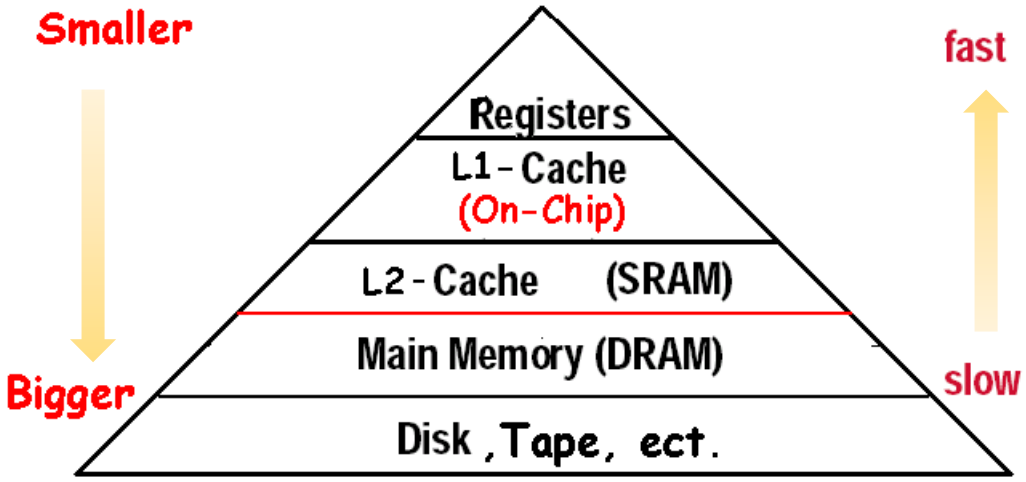
2-Way Set-Associative Cache

- Assume cache has 4 blocks and each block is 1 word
- 2 blocks per set, hence 2 sets per cache



What is a cache?

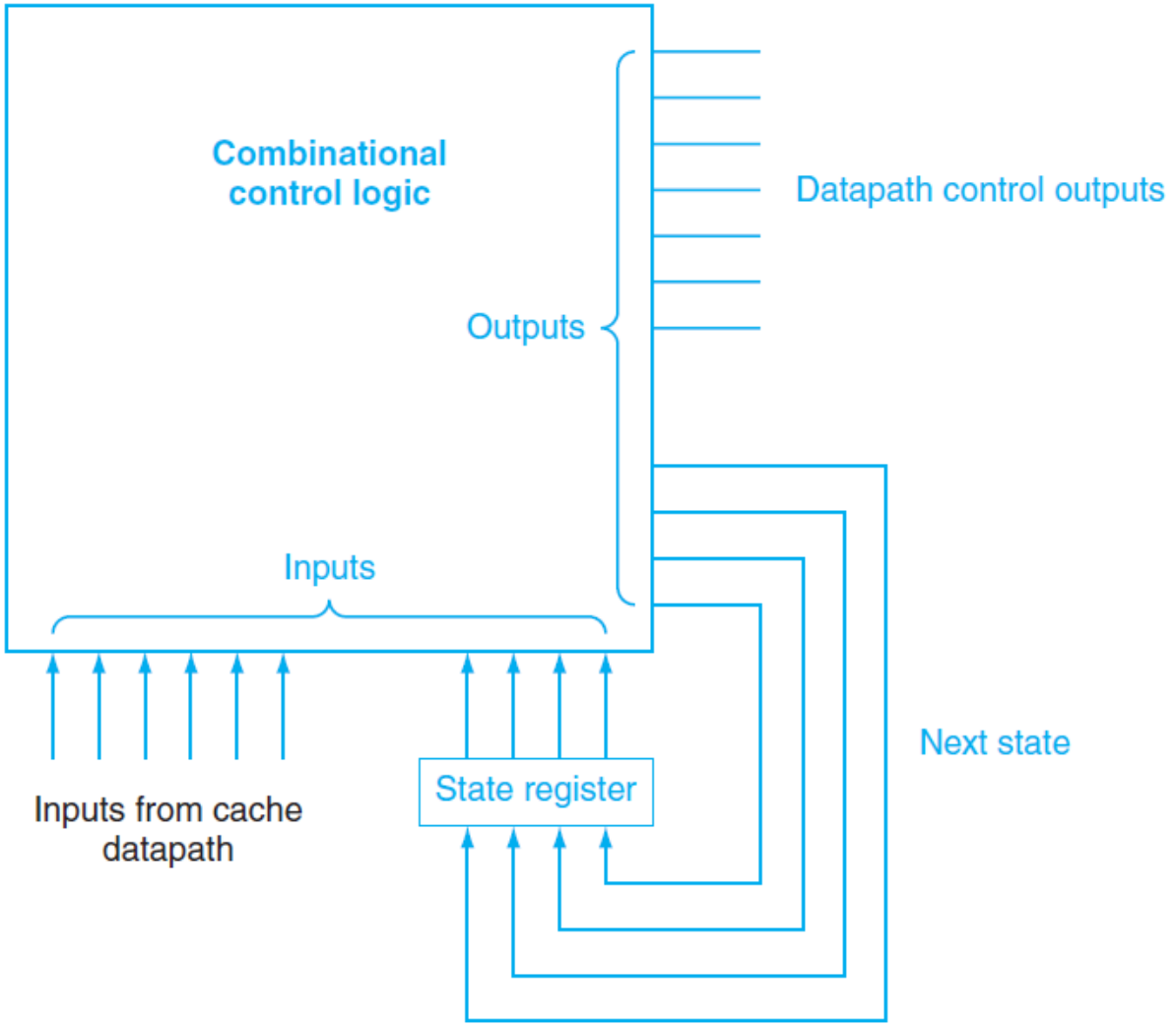
- Small, fast storage used to improve average access time to slow memory.
- In computer architecture, almost everything is a cache!
  - Registers “a cache” on variables – software managed
  - First-level cache a cache on second-level cache
  - Second-level cache a cache on memory
  - Memory a cache on disk (virtual memory)
  - TLB a cache on page table
  - Branch-prediction a cache on prediction information?



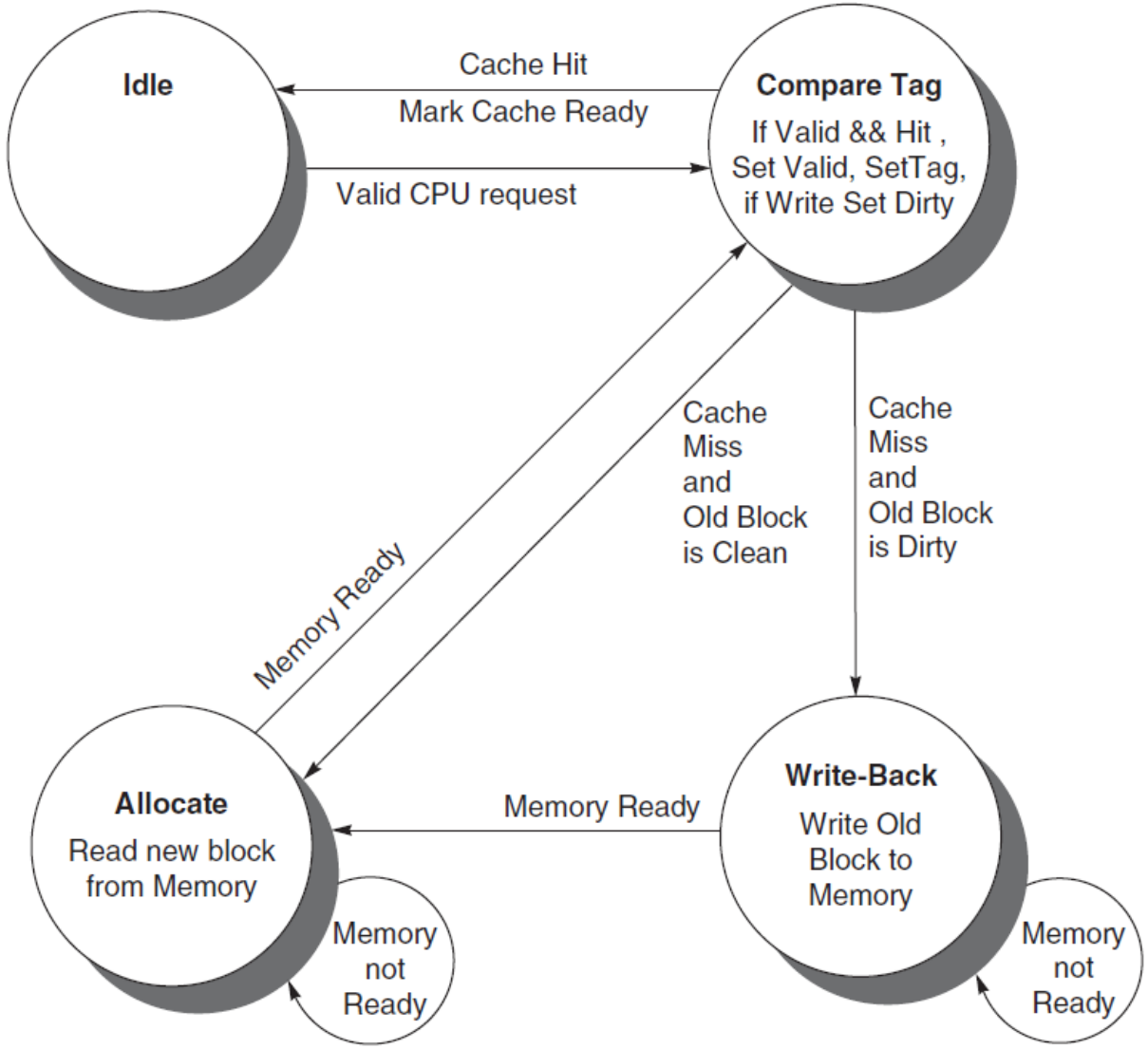
Using a Finite-State Machine to Control a Simple Cache

finite-state machine
A sequential logic function consisting of a set of inputs and outputs, next-state function that maps the current state and the inputs to a new state, and an output function that maps the current state and possibly the inputs to a set of asserted outputs.

next-state function
A combinational function that, given the inputs and the current state, determines the next state of a finite-state machine.

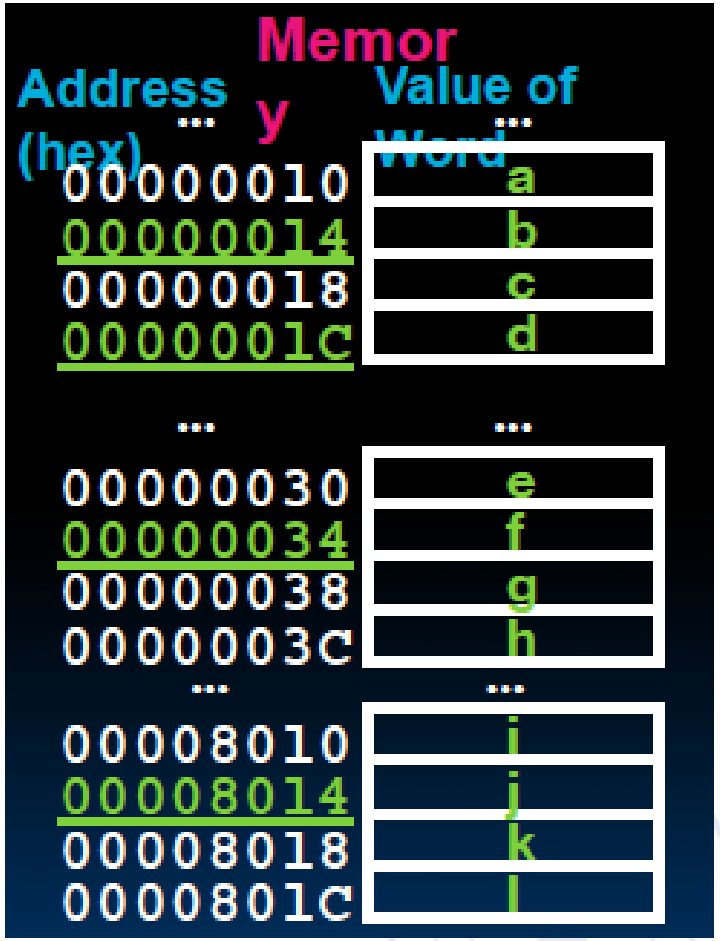


Using a Finite-State Machine to Control a Simple Cache



Access data in a direct mapped cache

- 16KB of data, direct-mapped, 4 words blocks
- Read 4 addresses:
  - 0x00000014
  - 0x0000001C
  - 0x00000034
  - 0x00008014
- Memory values here:

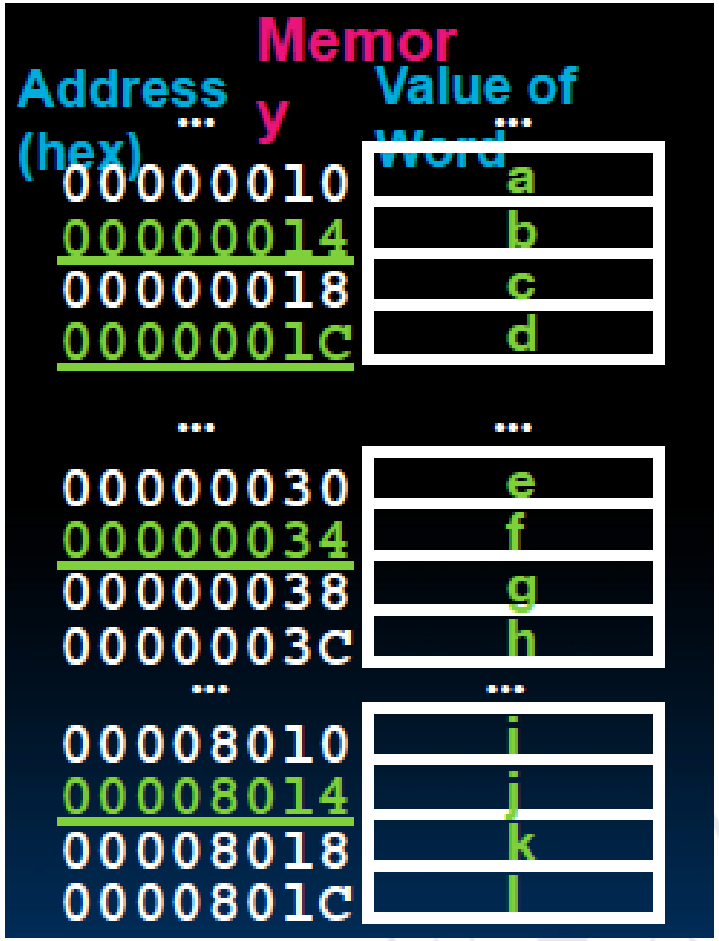


Access data in a direct mapped cache

- 16KB of data, direct-mapped, 4 words blocks
- Read 4 addresses:
  - 0x00000014
  - 0x0000001C
  - 0x00000034
  - 0x00008014

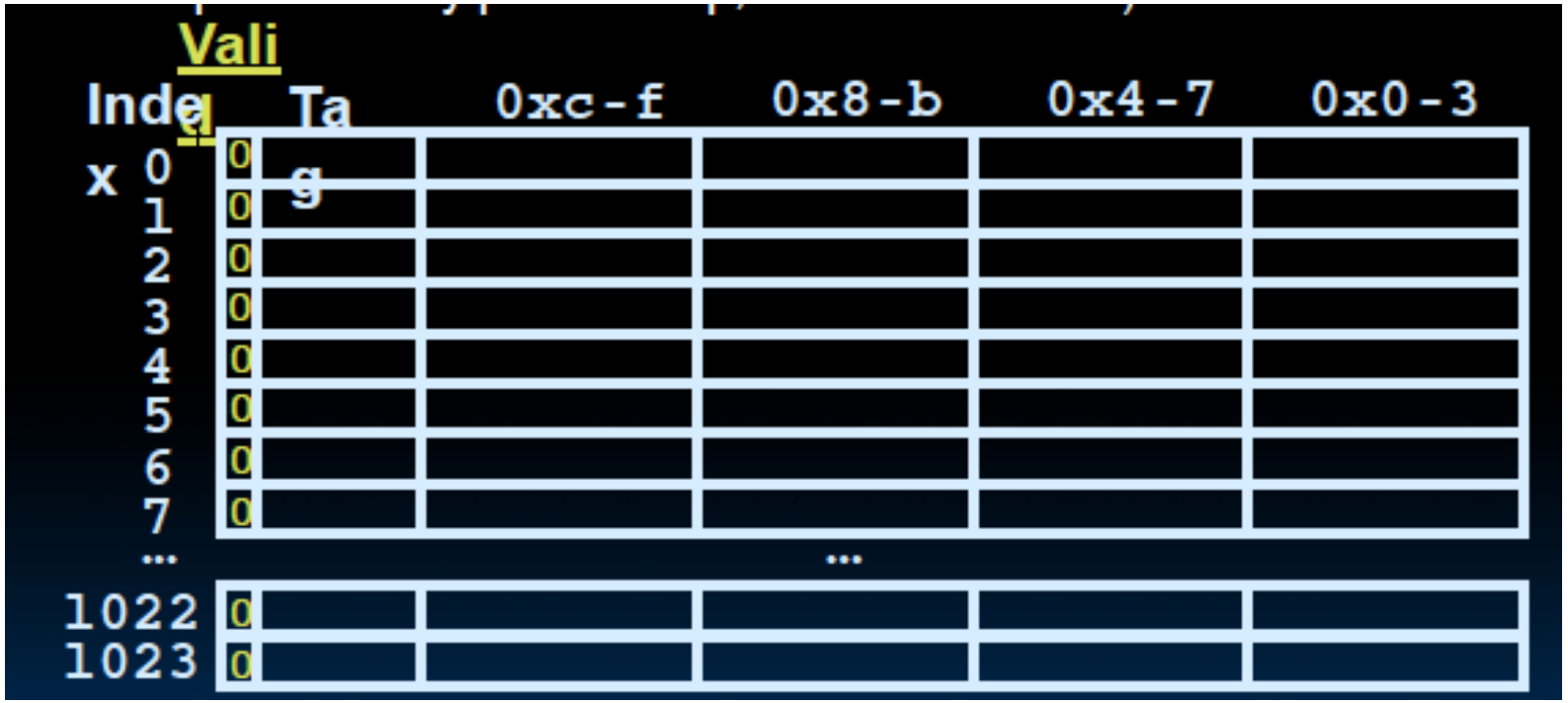
```
00000000000000000 0000000001 0100
00000000000000000 0000000001 1100
00000000000000000 0000000011 0100
00000000000000010 0000000001 0100
```

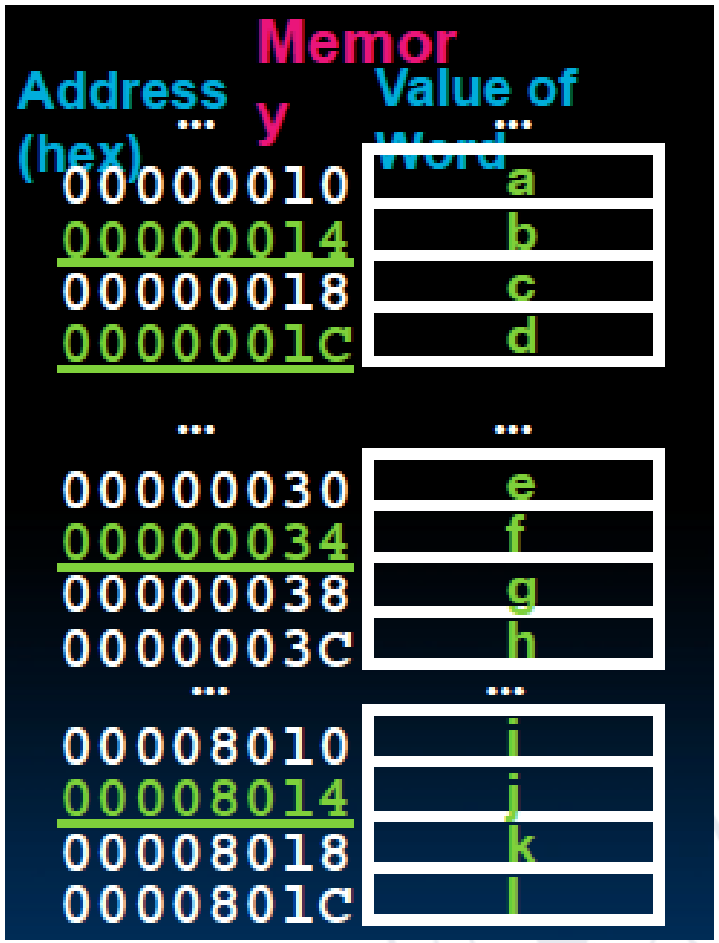
- Memory values here:
- 4 addresses divided into Tag, Index, Byte Offset



Access data in a direct mapped cache

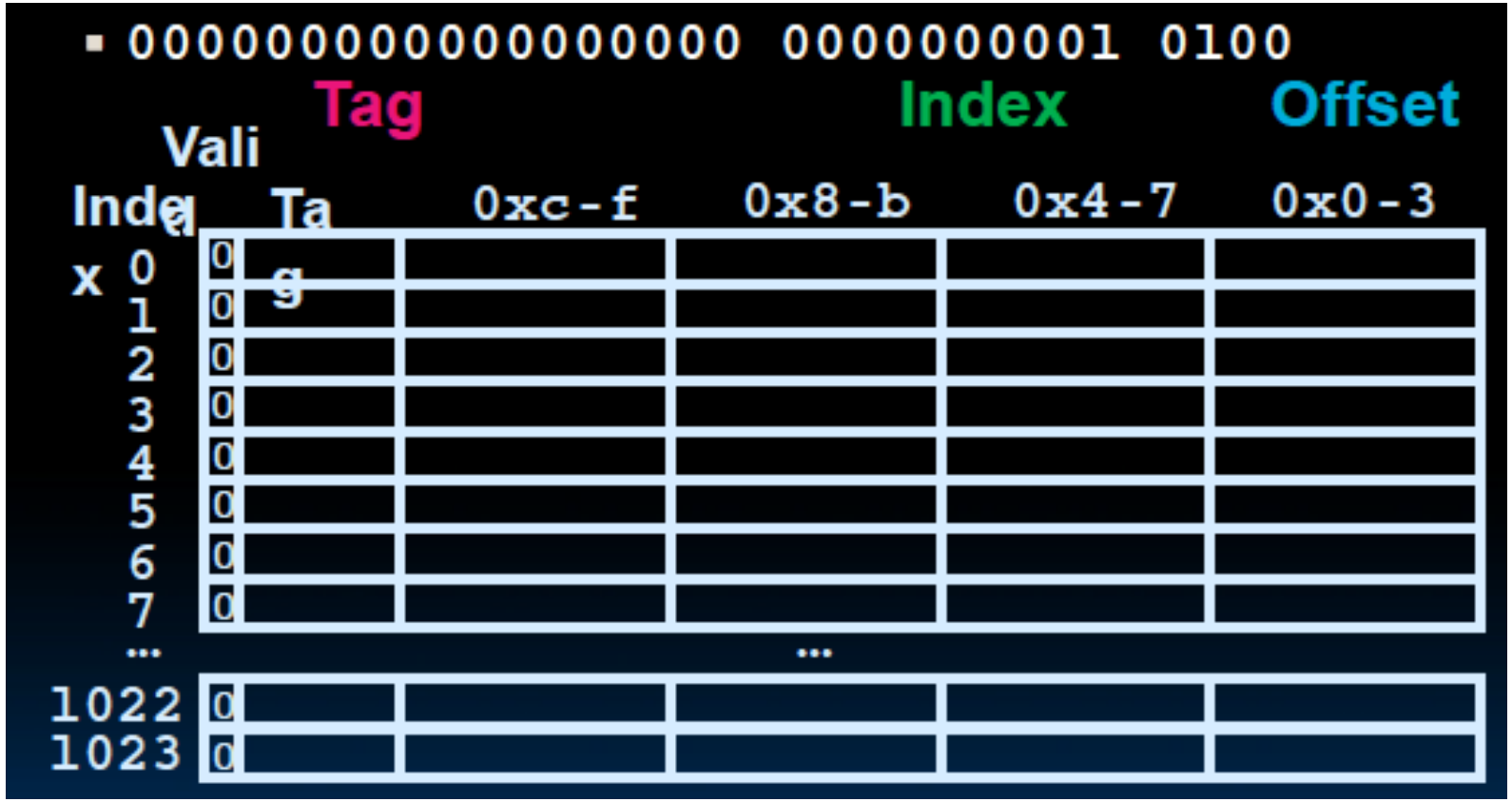
- Valid bit: determines whether anything is stored in that row (when computer initially powered up, all entries invalid)

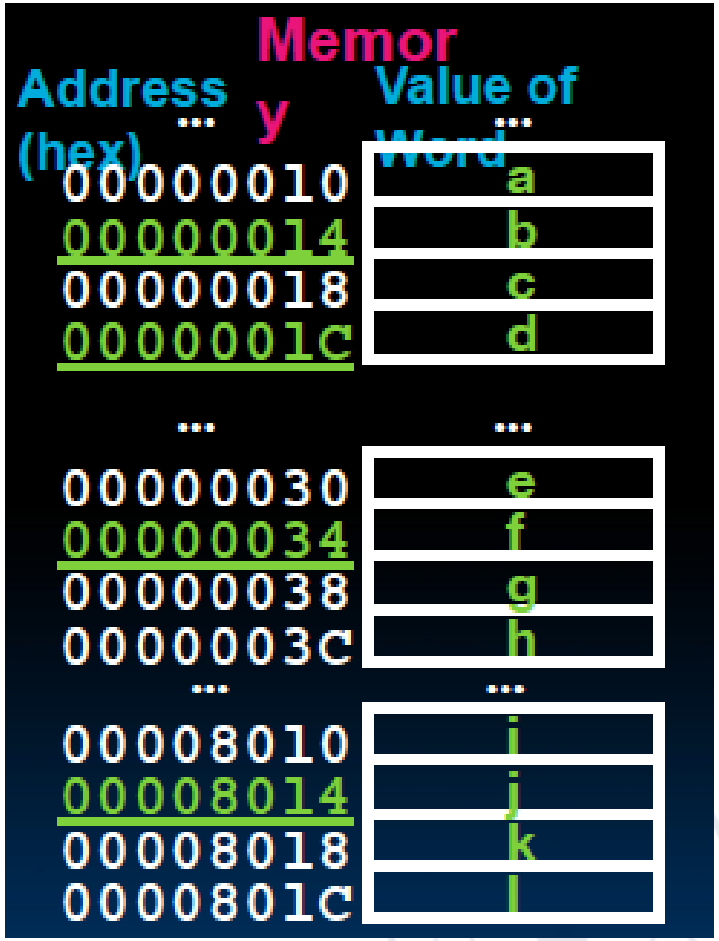




Access data in a direct mapped cache

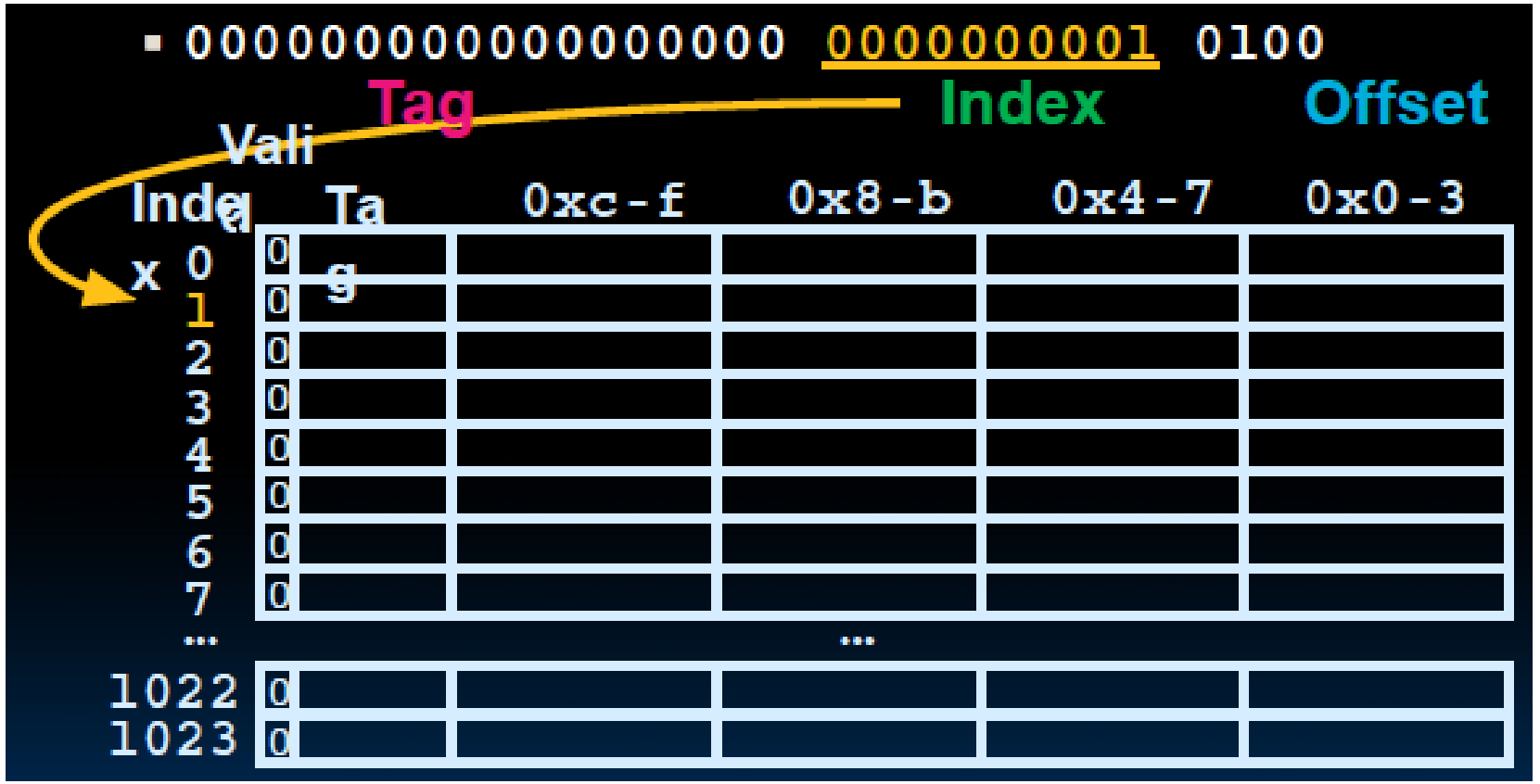
- Read 0x00000014

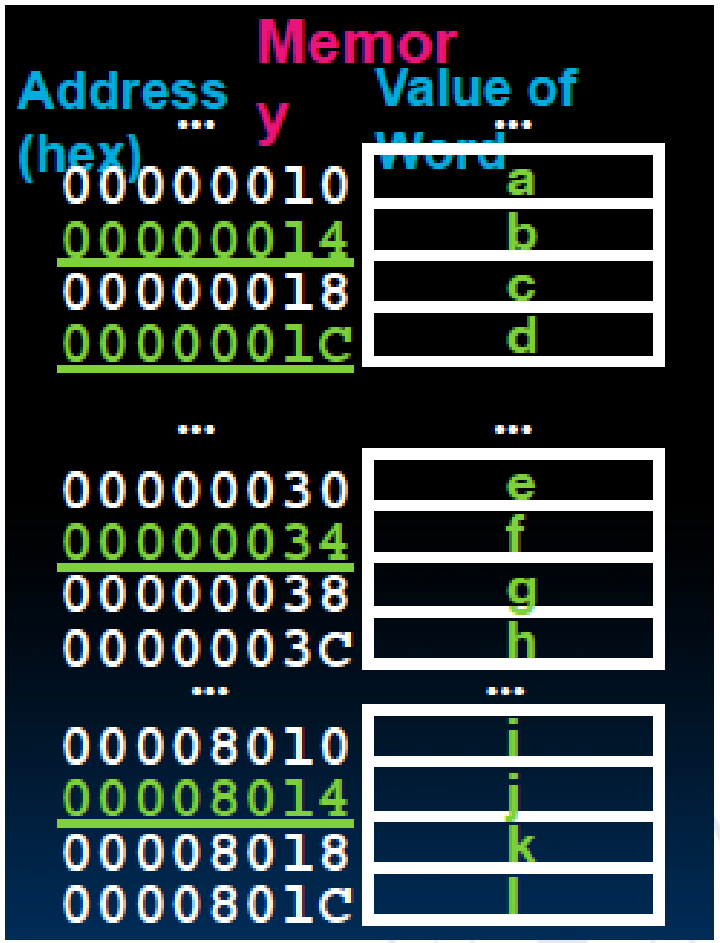




Access data in a direct mapped cache

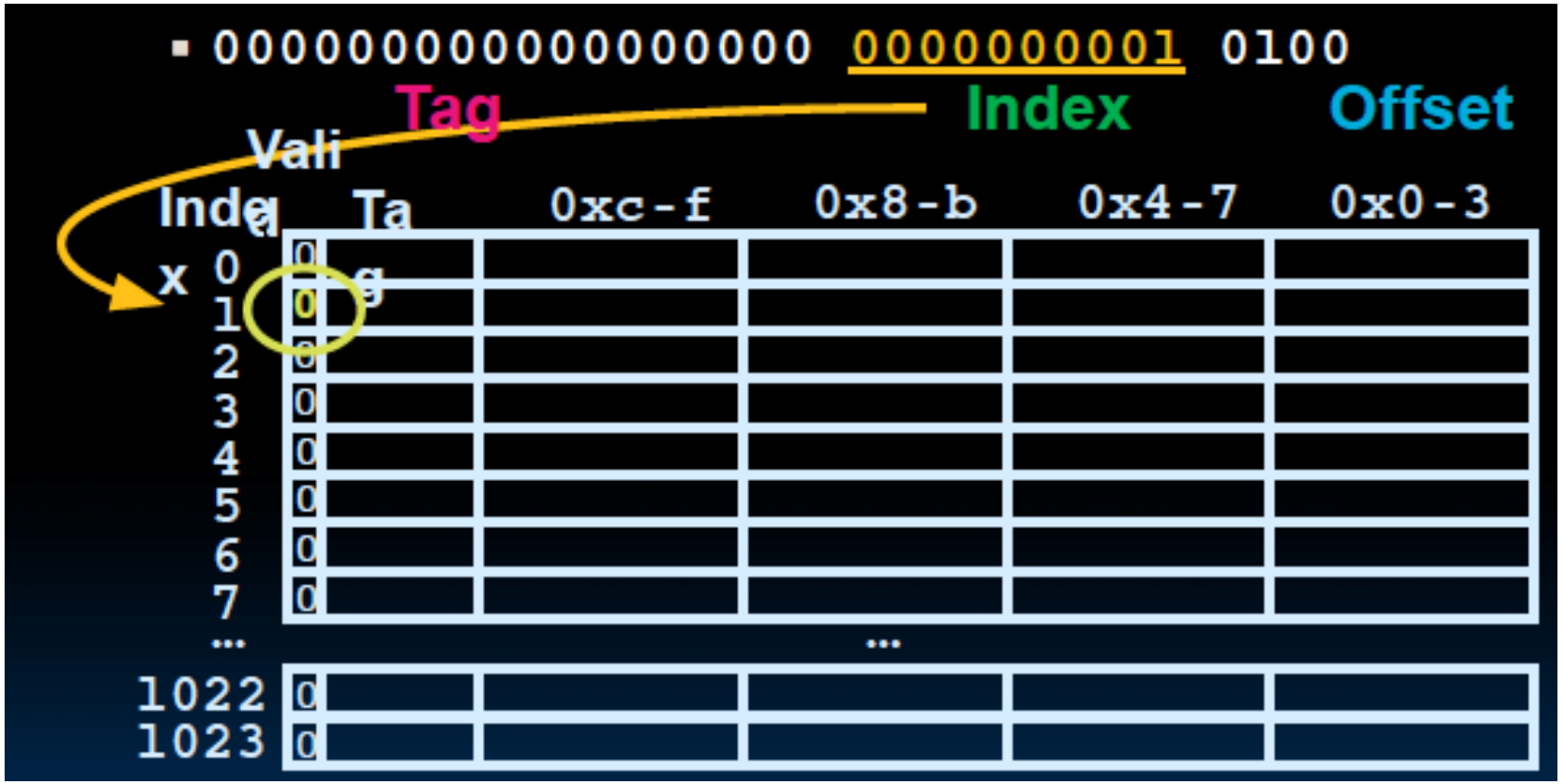
- Read 0x00000014, read block 1 (0000000001)

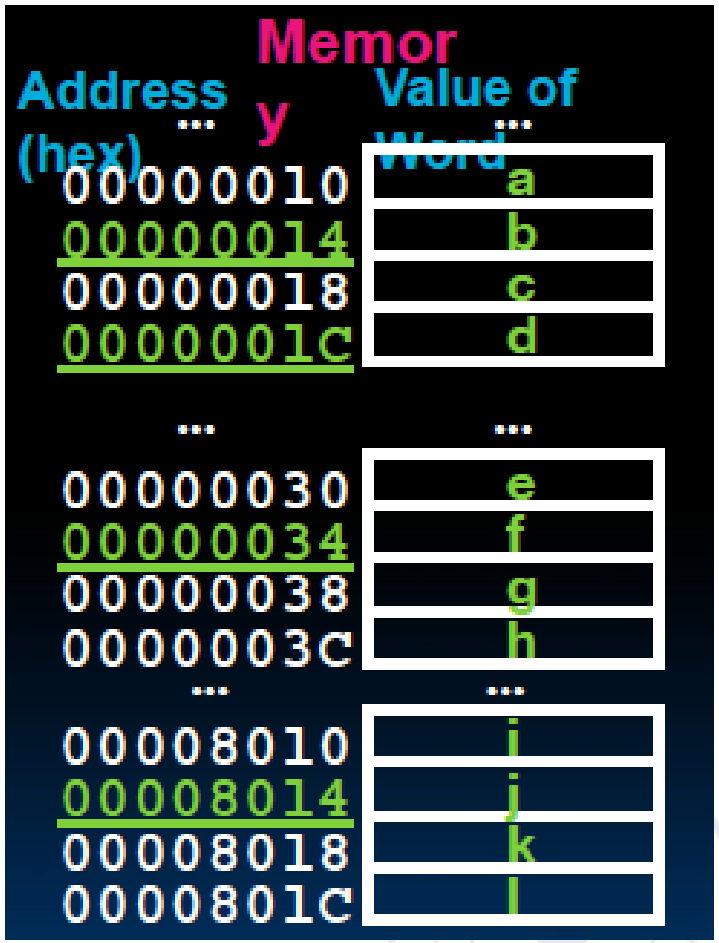




Access data in a direct mapped cache

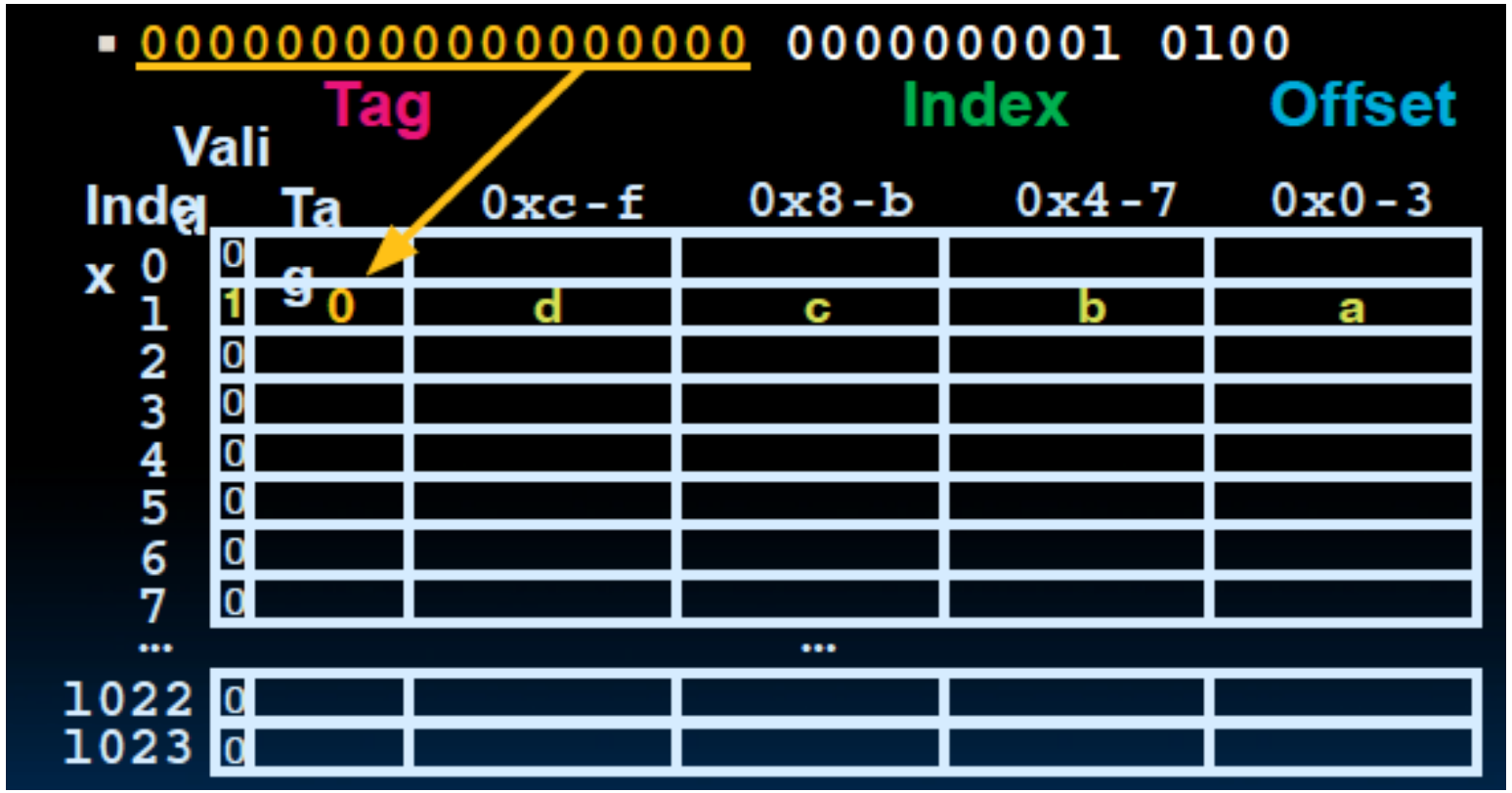
- Read 0x00000014, no valid data

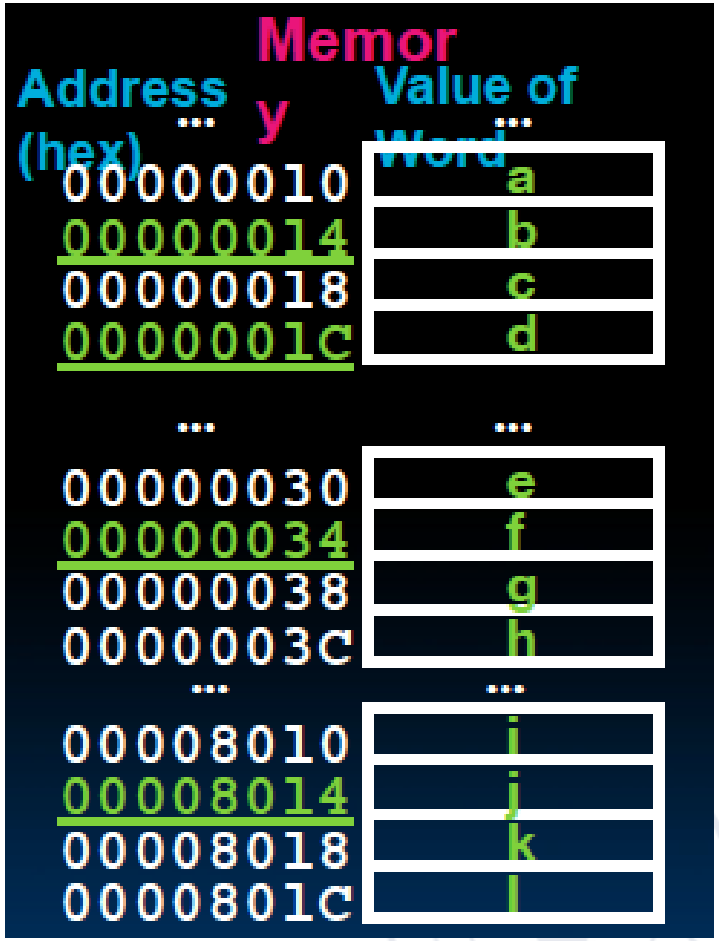




Access data in a direct mapped cache

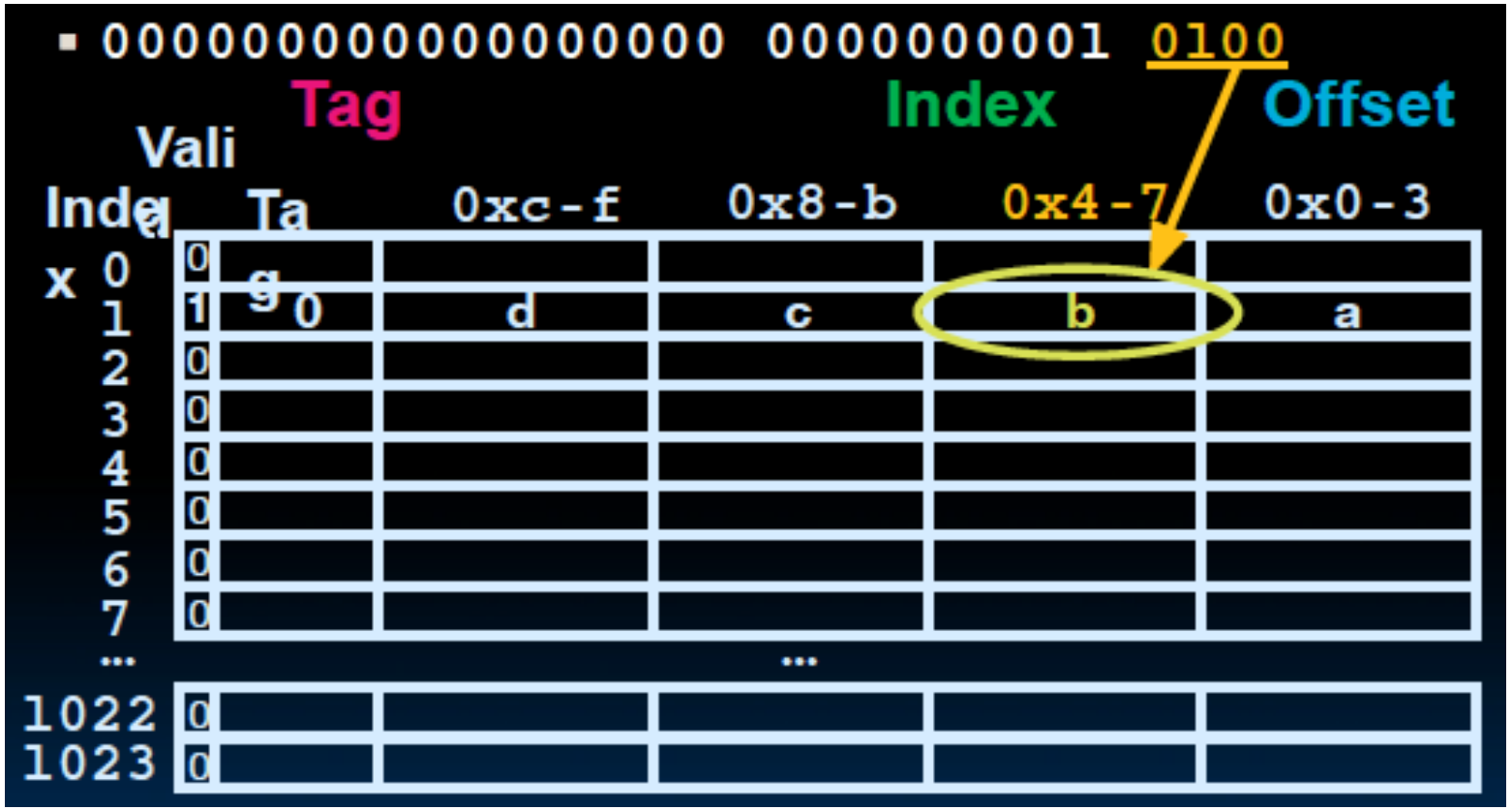
- Read 0x00000014, load that data into cache, setting tag, valid

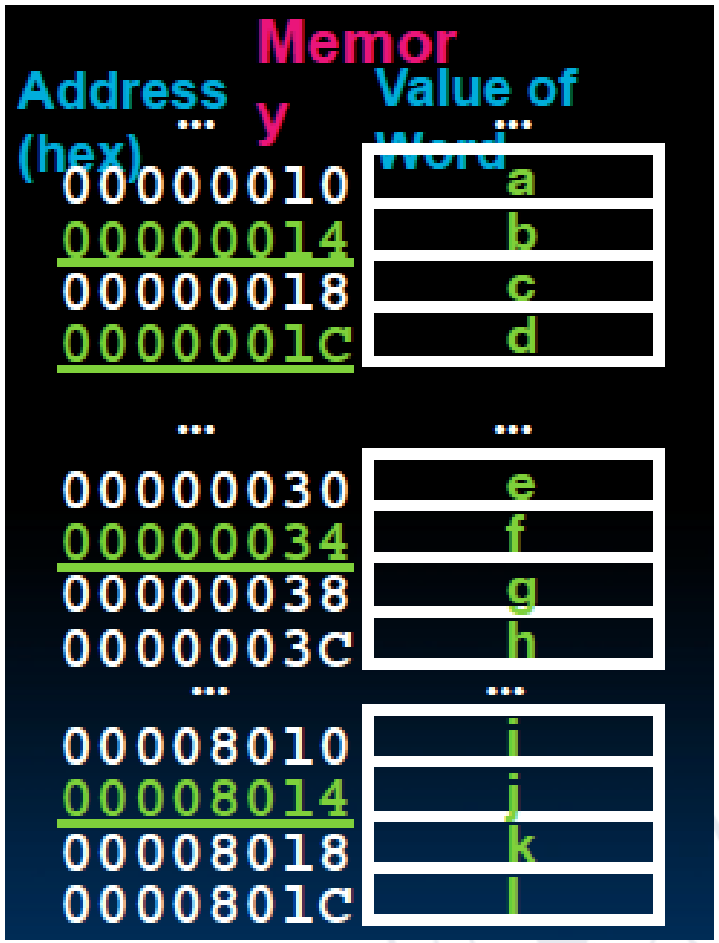




Access data in a direct mapped cache

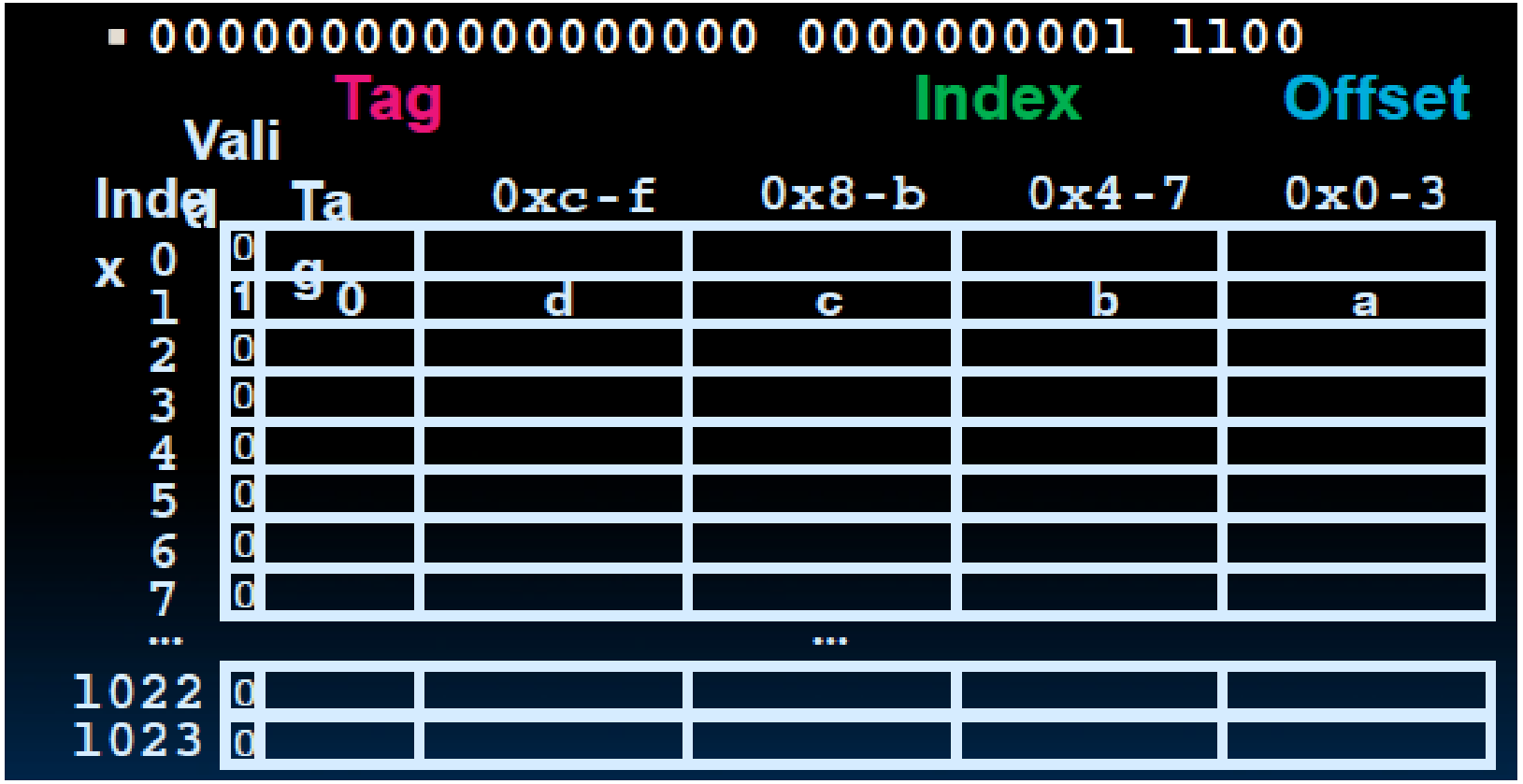
- Read 0x00000014, read from cache at offset, return word b

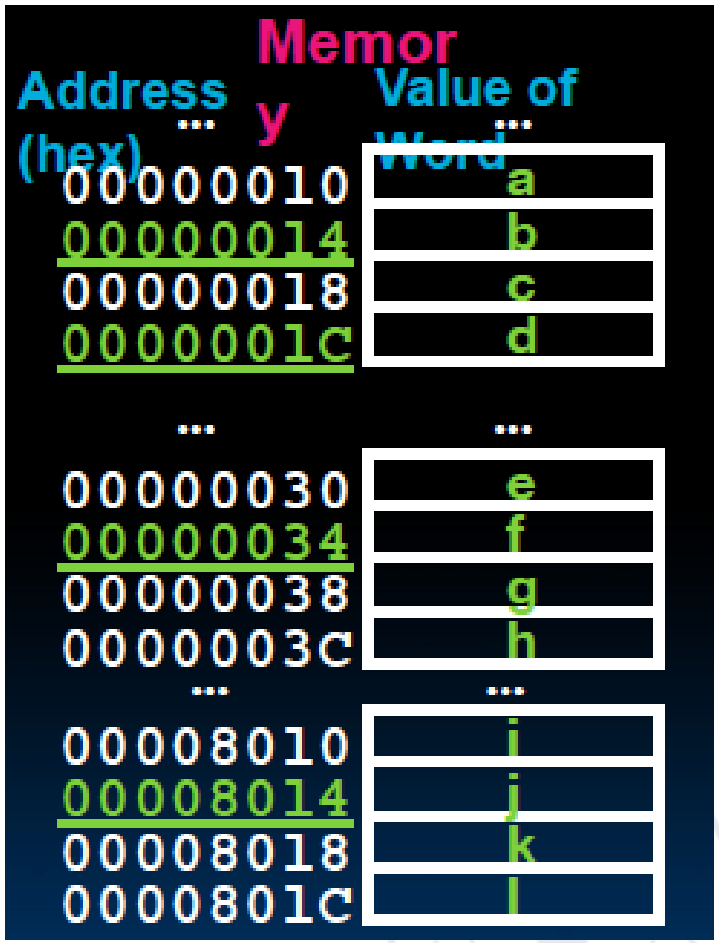




Access data in a direct mapped cache

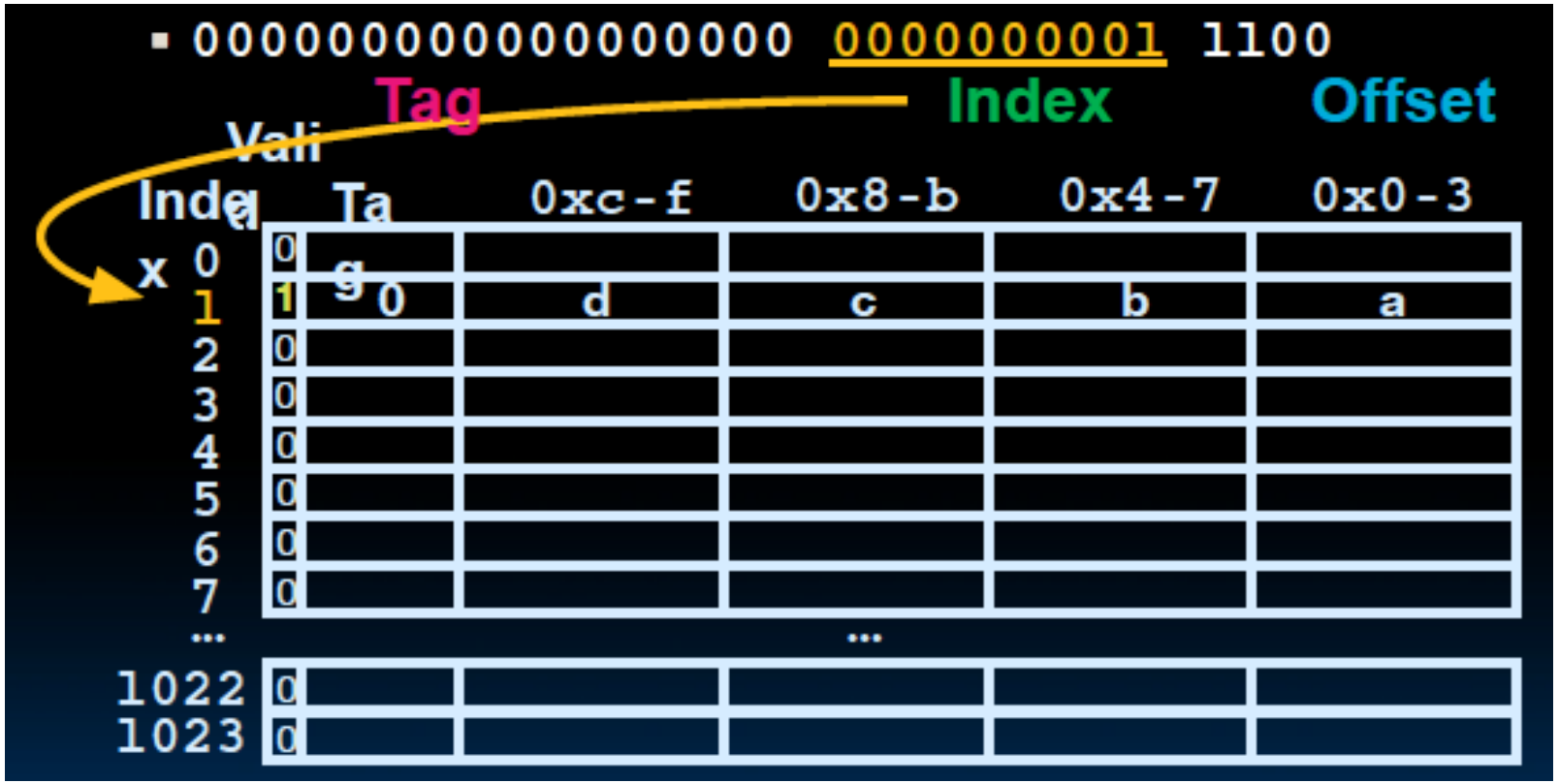
- Read 0x0000001C

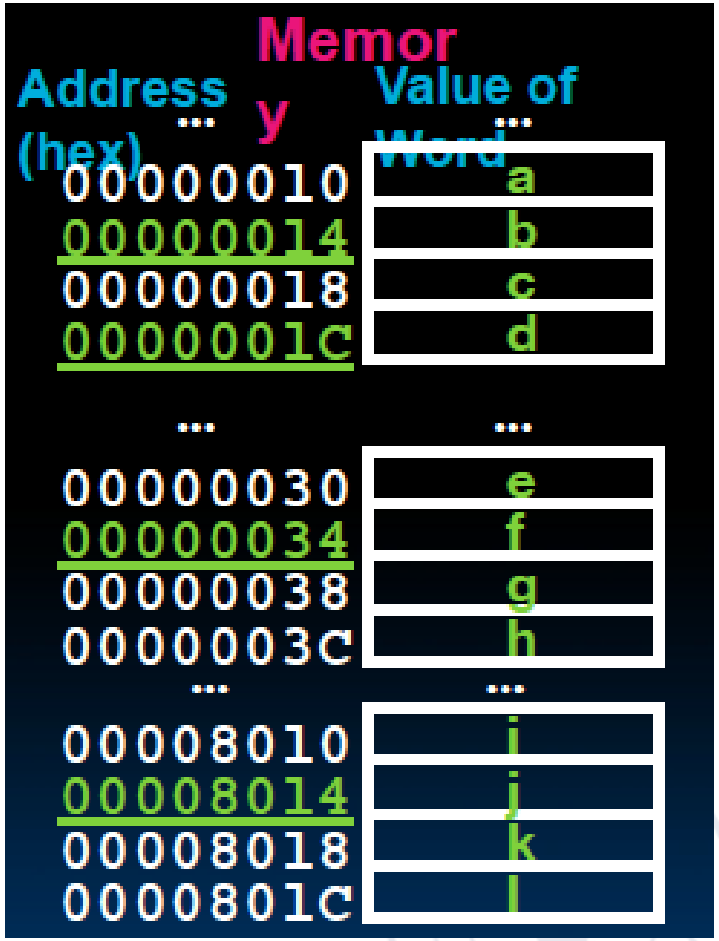




Access data in a direct mapped cache

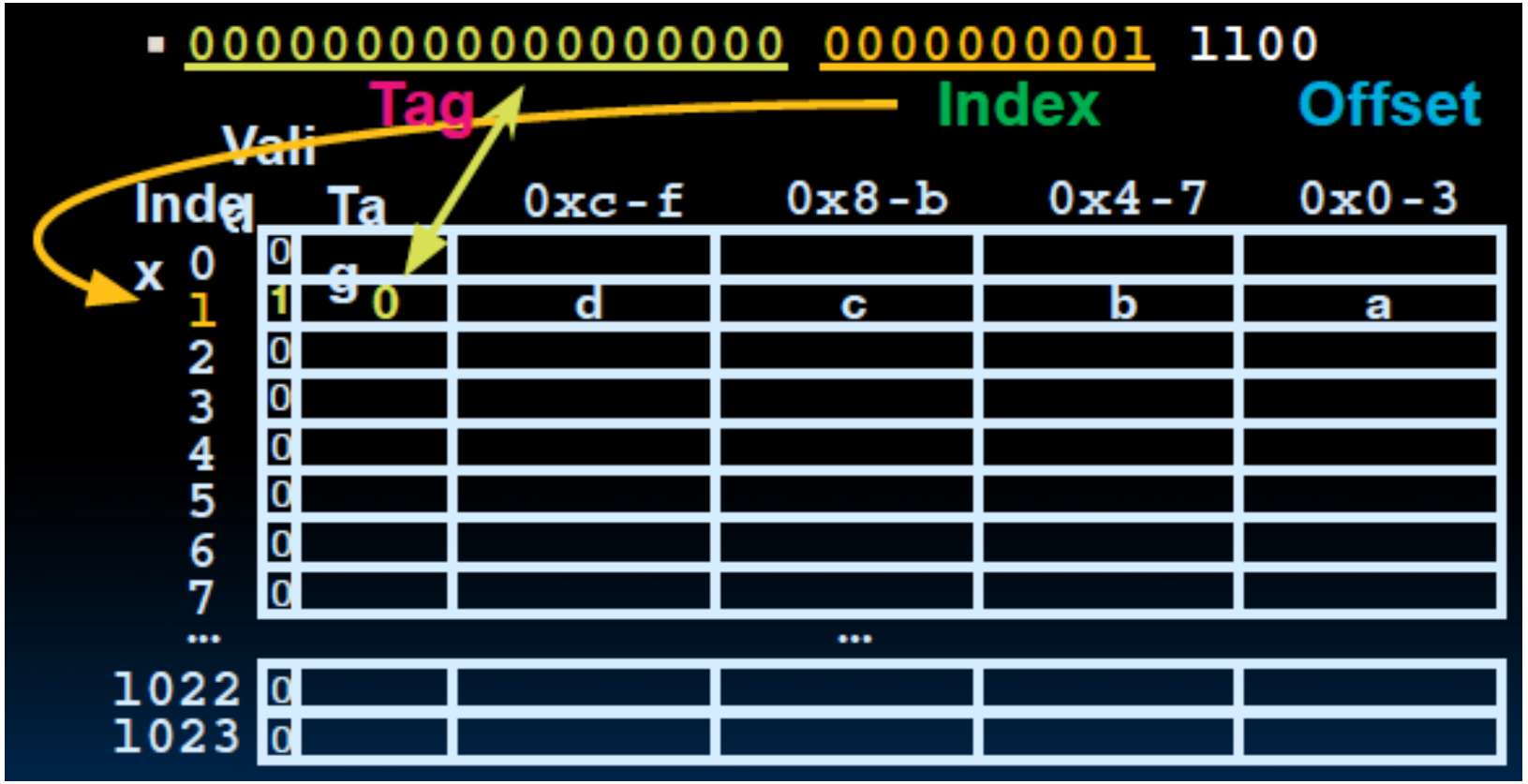
- Read 0x0000001C, index is valid

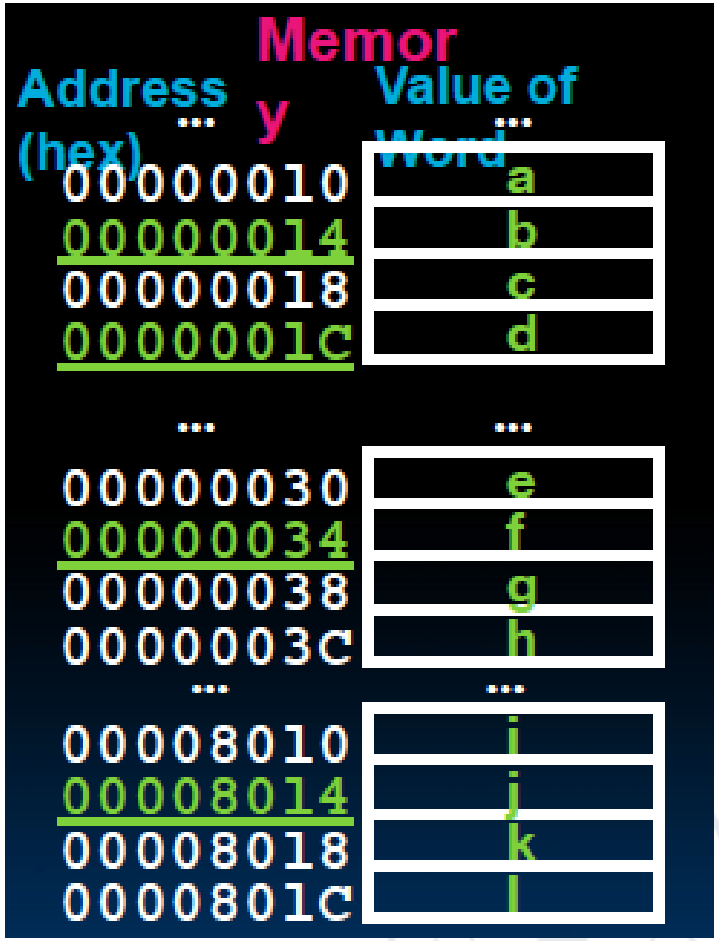




Access data in a direct mapped cache

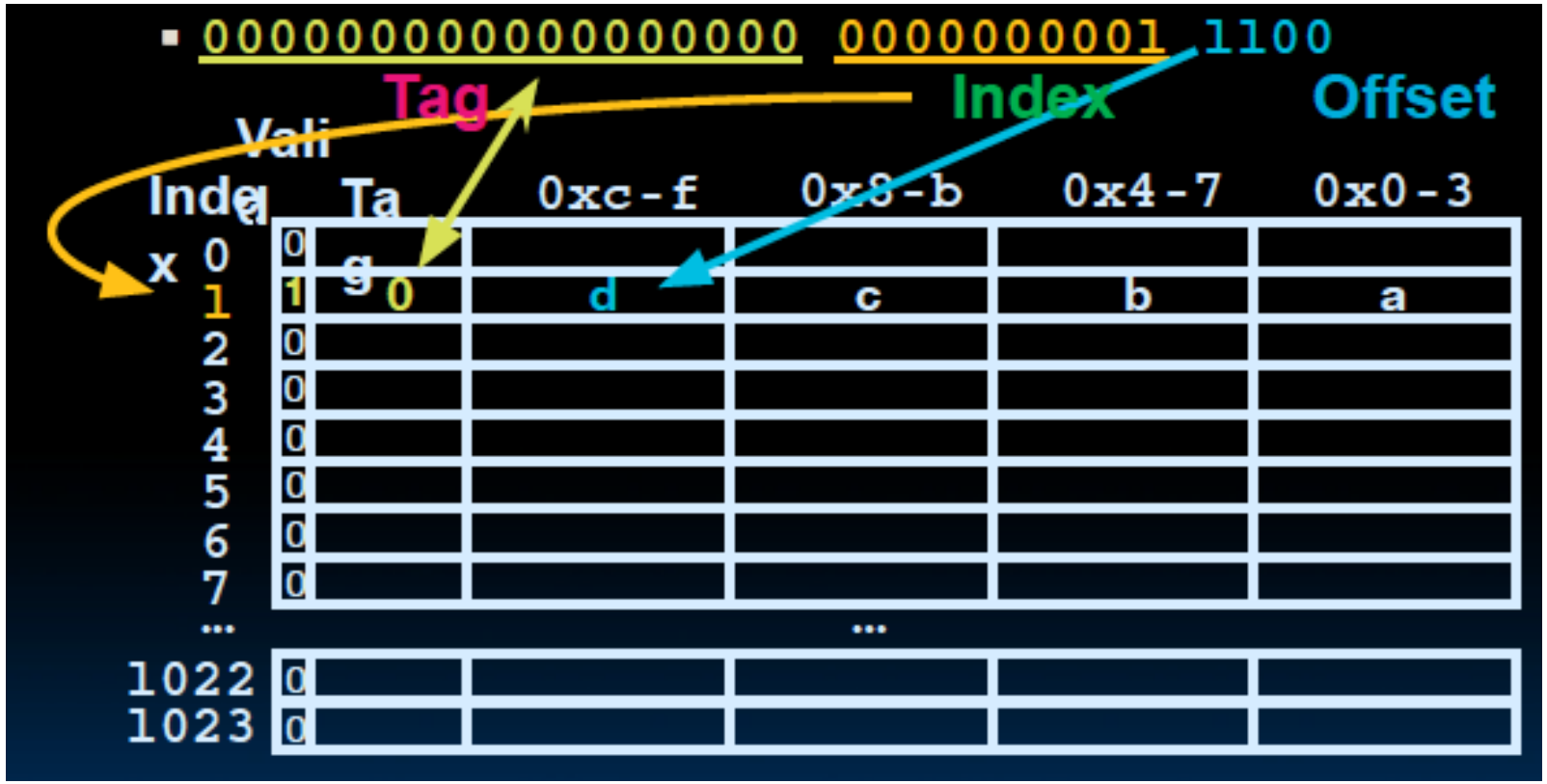
- Read 0x0000001C, index is valid, tag matches

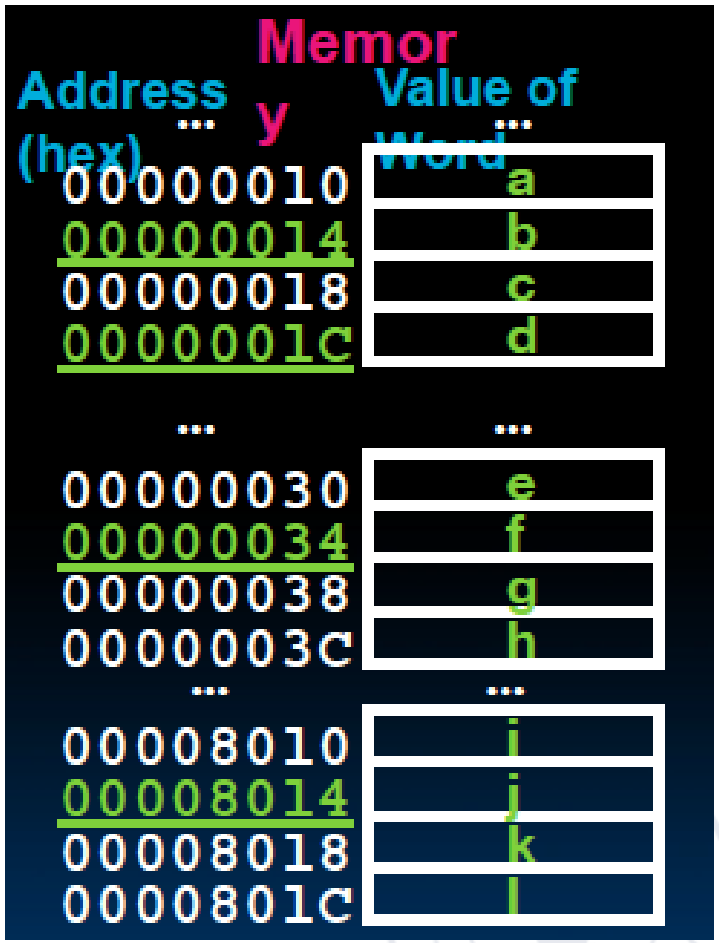




Access data in a direct mapped cache

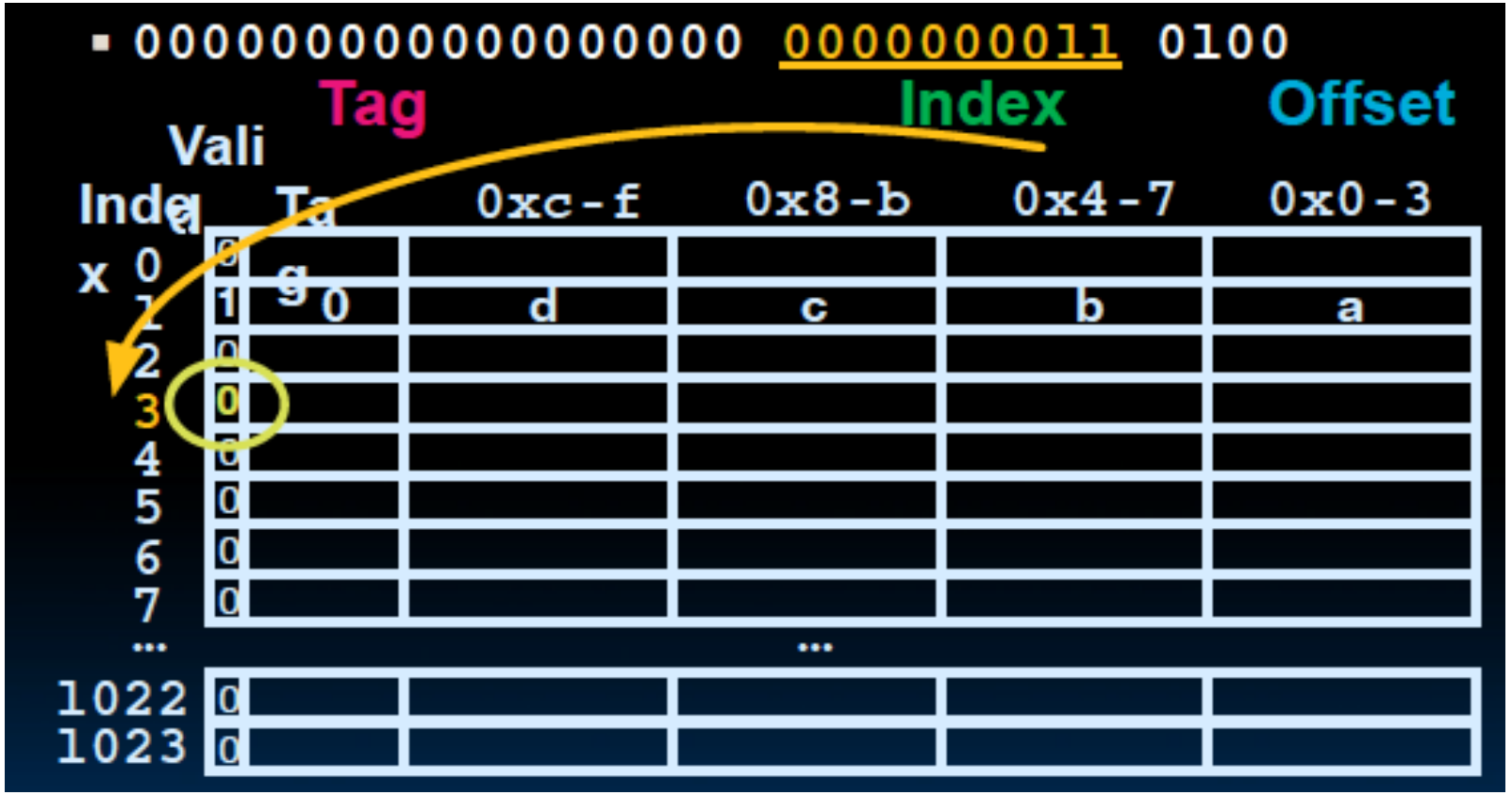
- Read 0x0000001C, index is valid, tag matches, return d

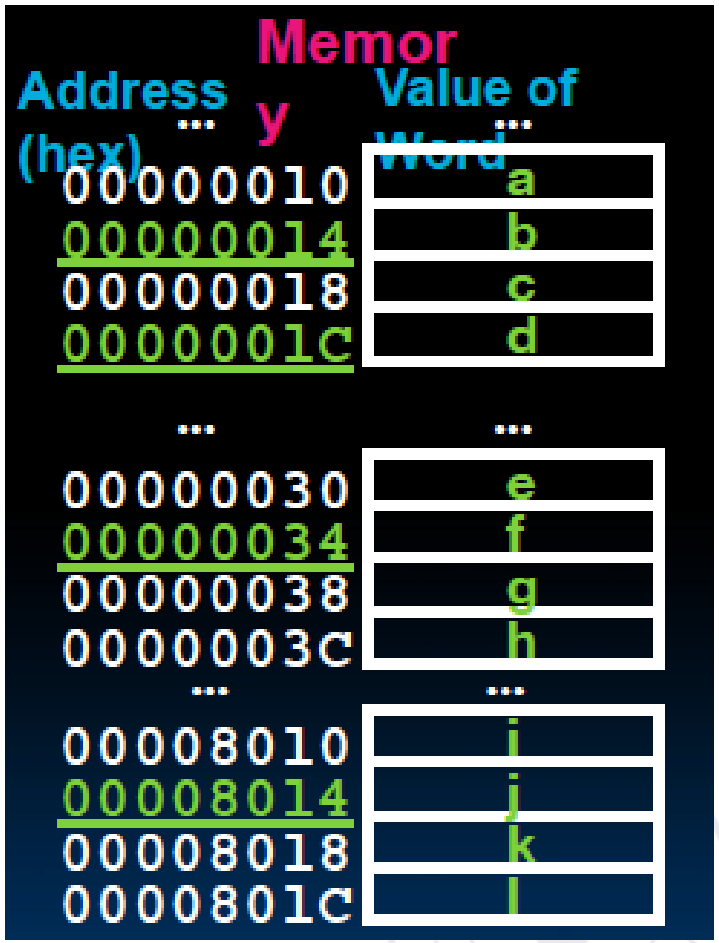




Access data in a direct mapped cache

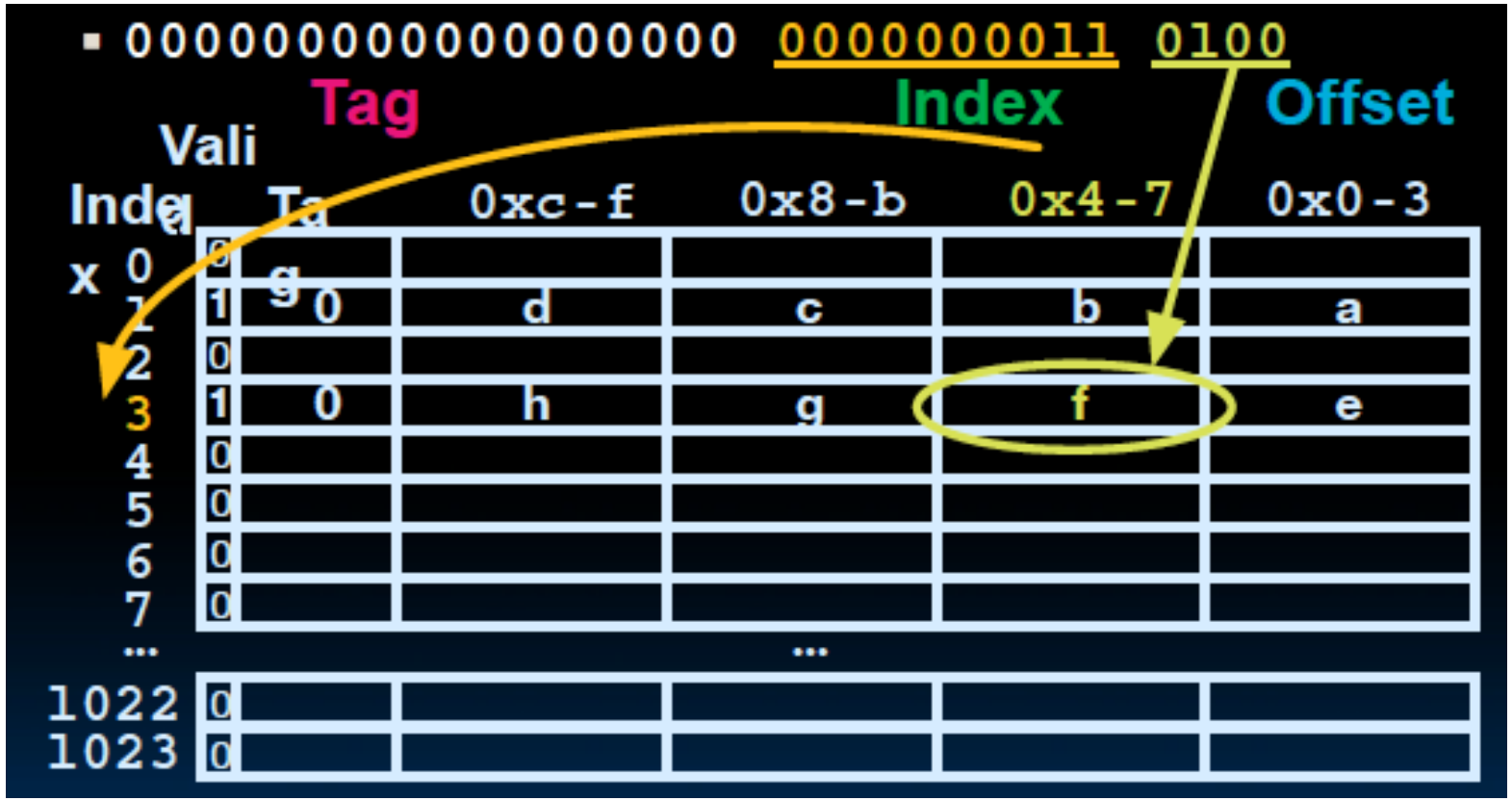
- Read 0x00000034, read block 3, no valid data

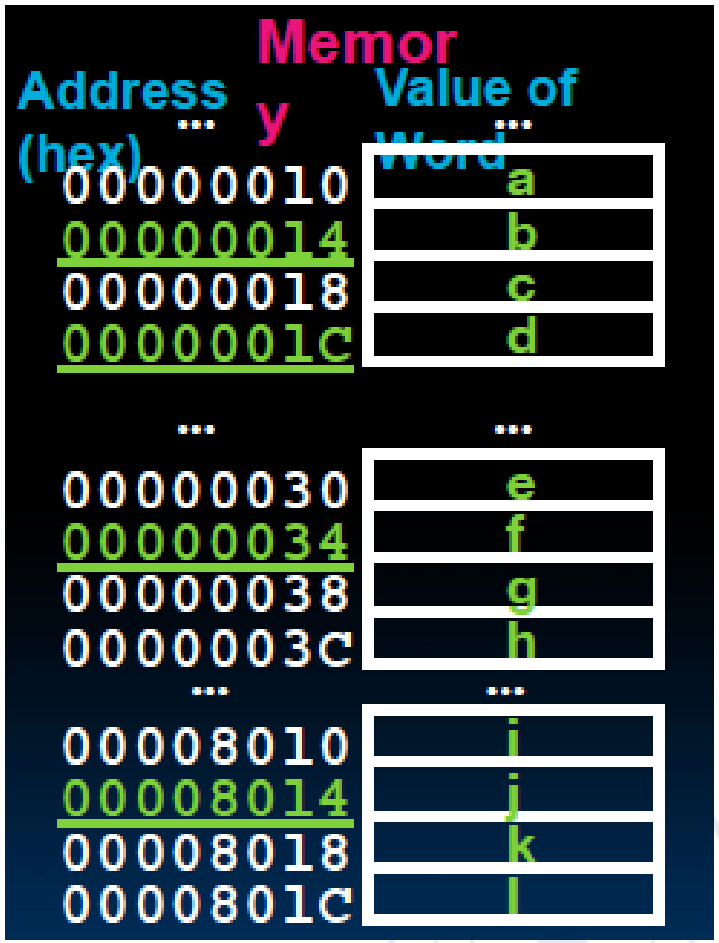




Access data in a direct mapped cache

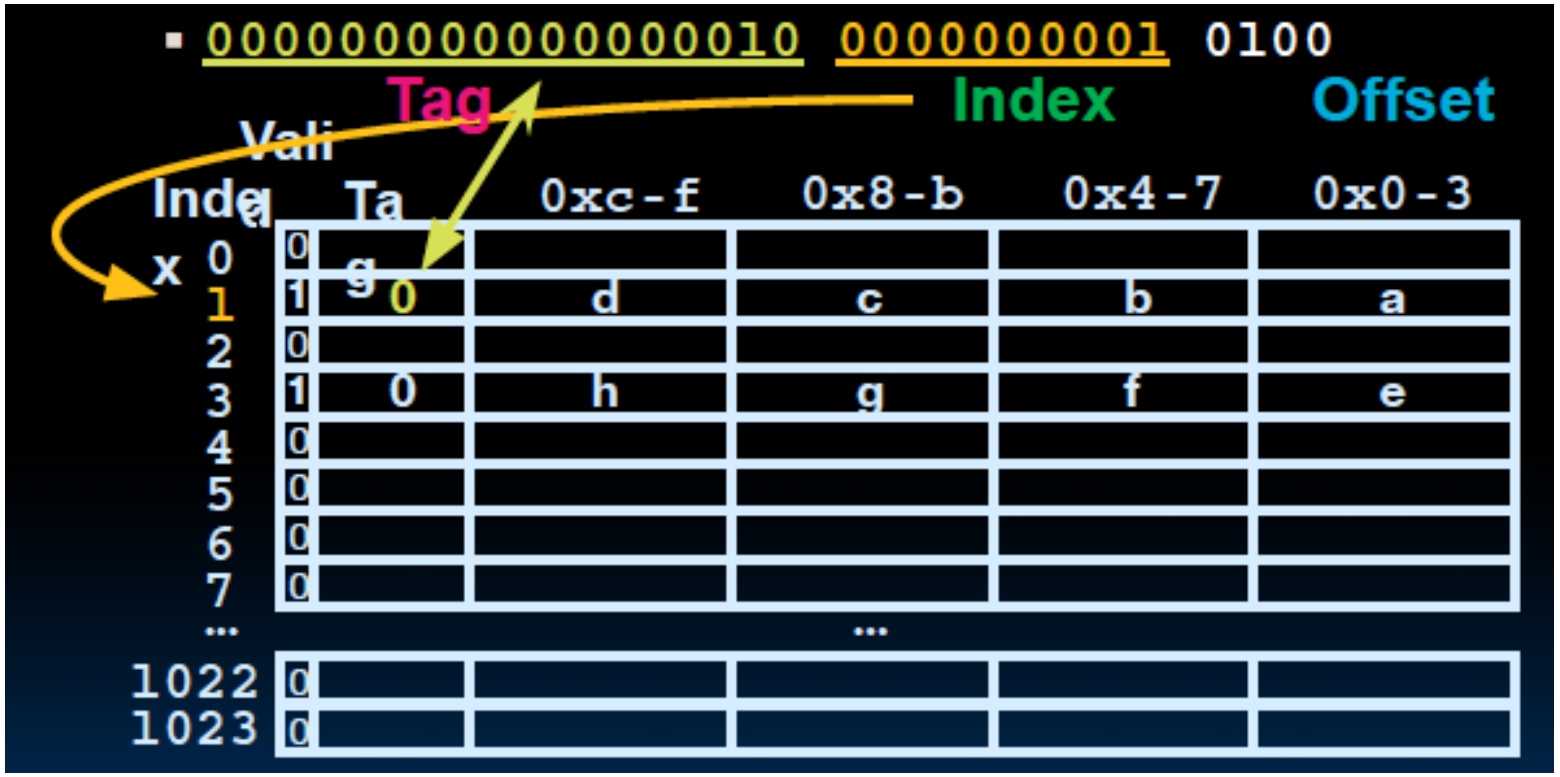
- Read 0x00000034, load that cache block, return word f

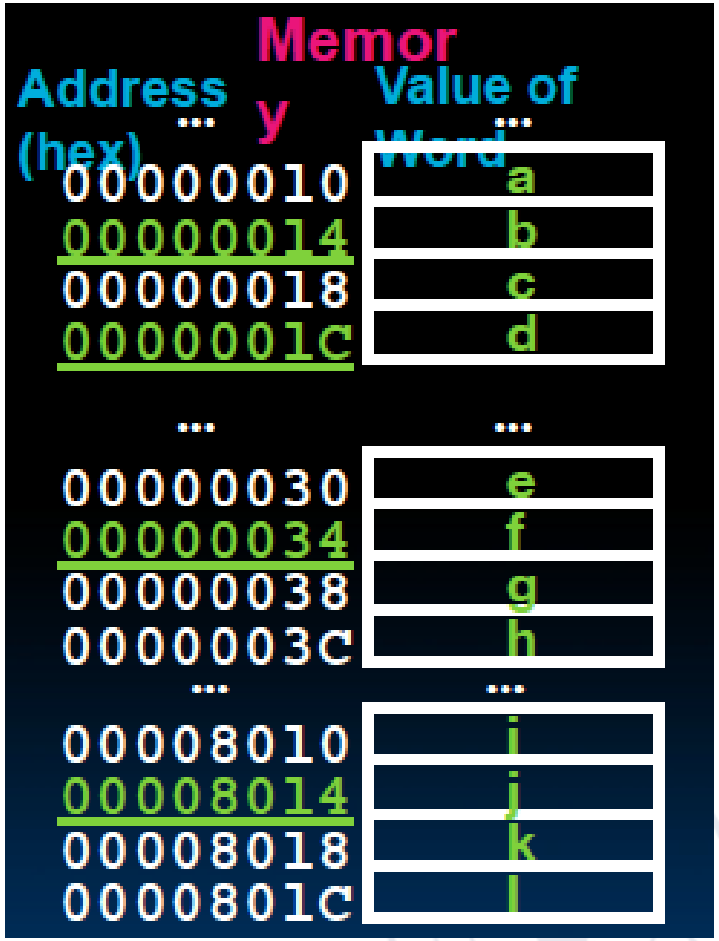




Access data in a direct mapped cache

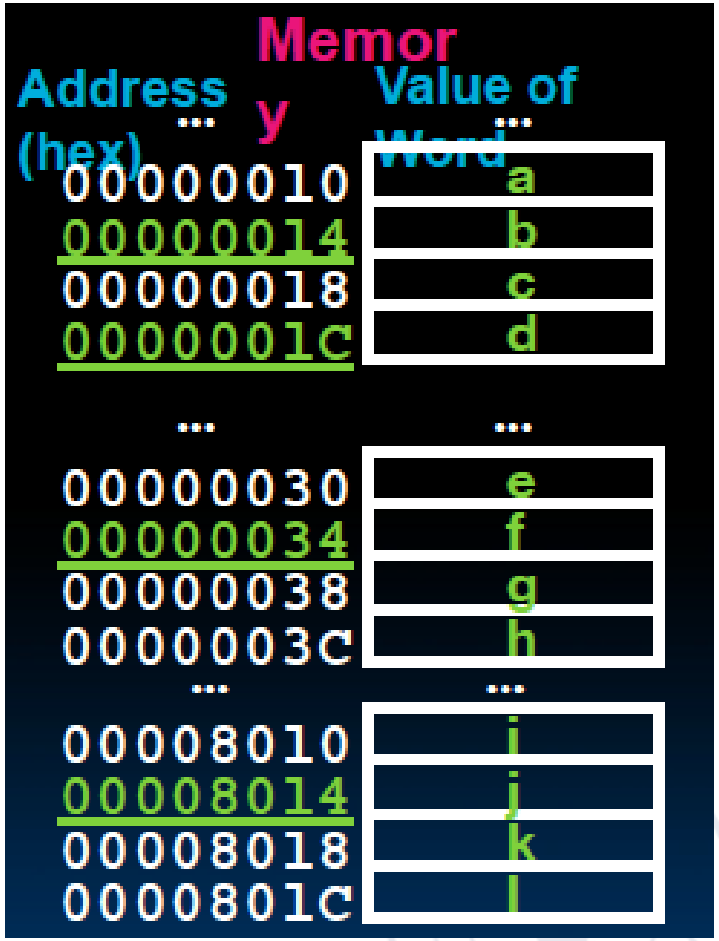
- Read 0x00008014, read cache block 1, data is valid, cache block 1 tag does not match (0!=2)





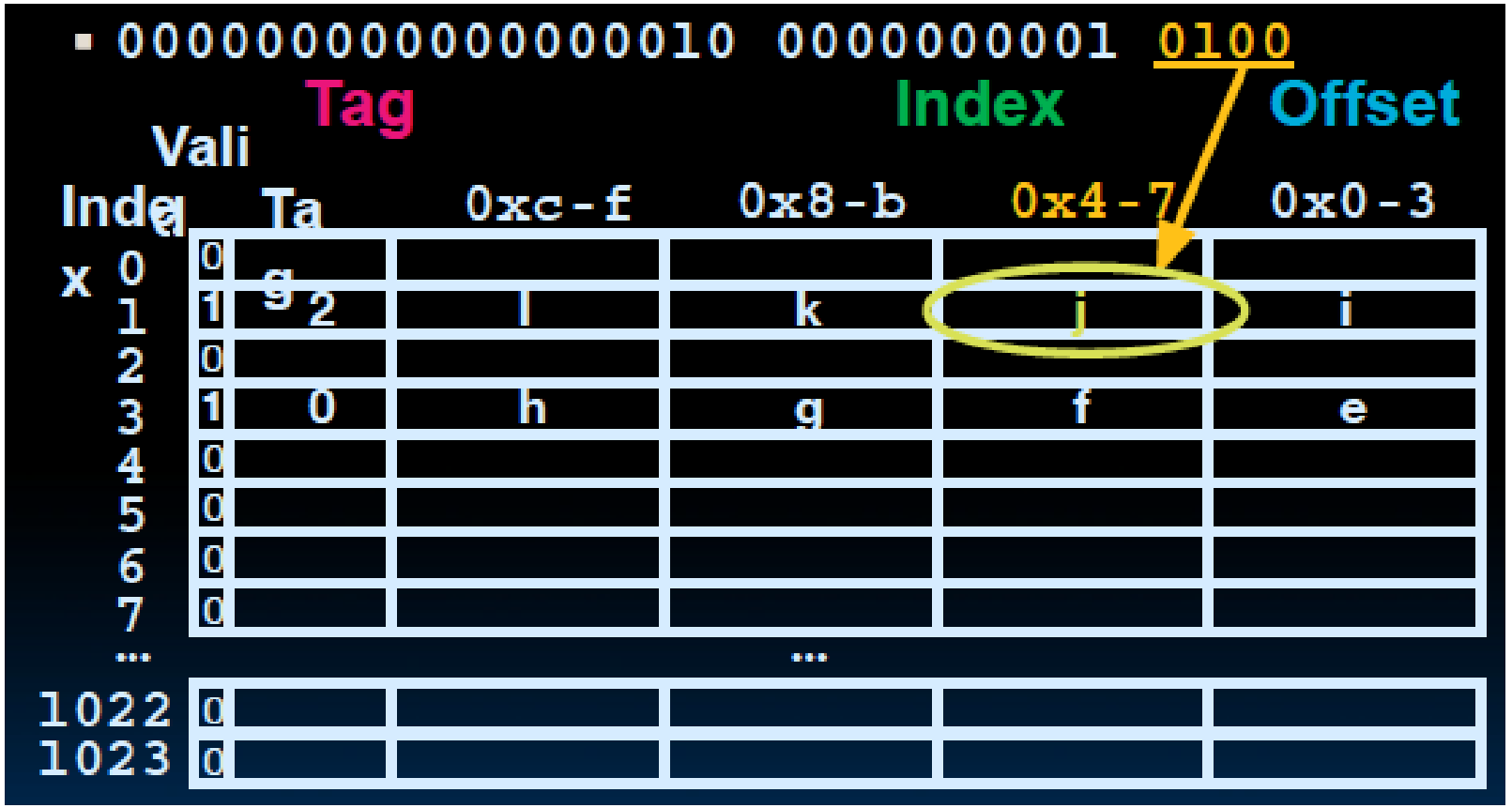
Access data in a direct mapped cache

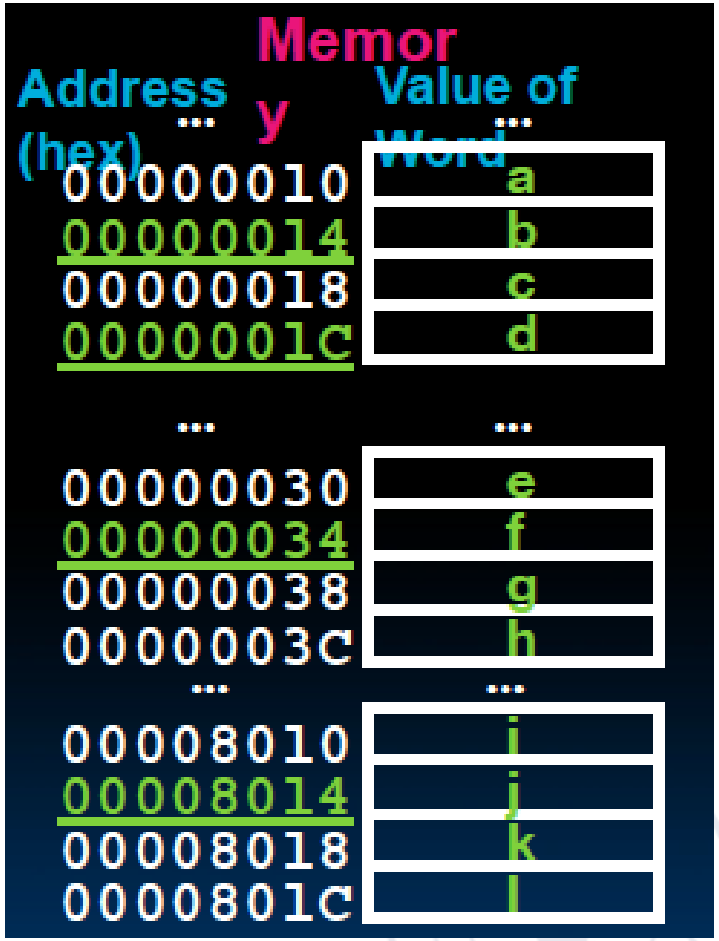
- Read 0x00008014, miss, replace block 1 with new data & tag



Access data in a direct mapped cache

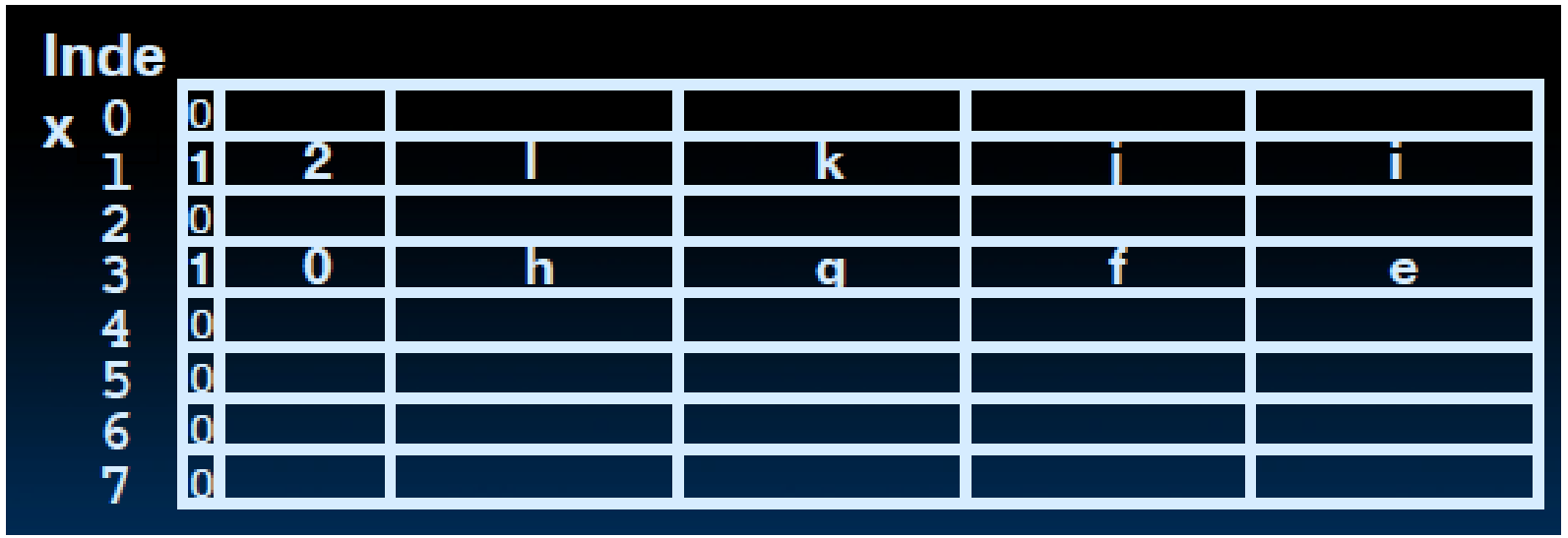
- Read 0x00008014, return word j

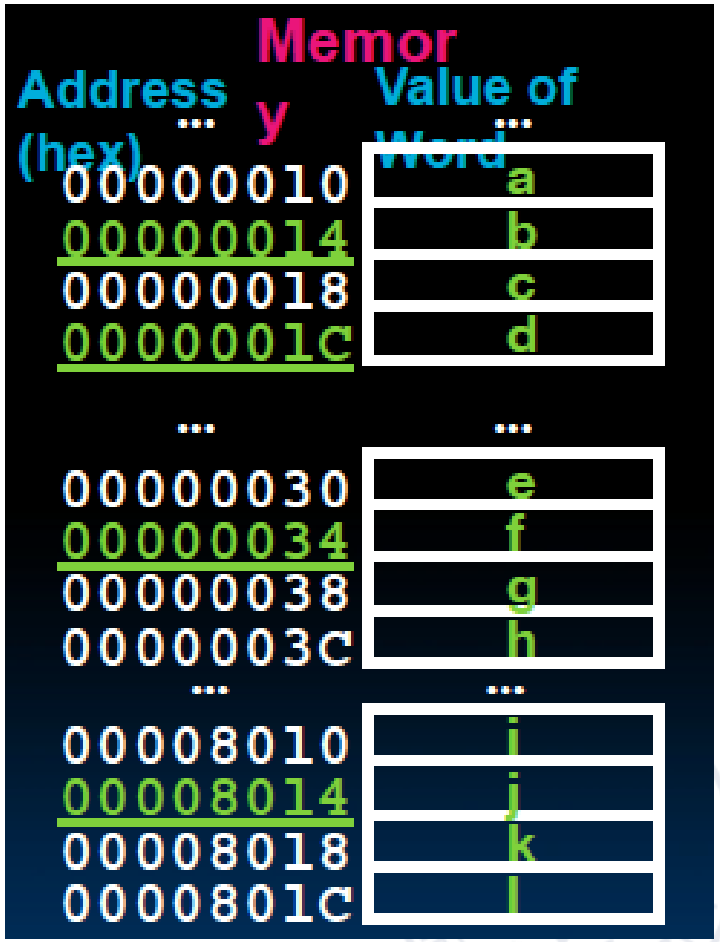




Homework

Cache 16KB of data, direct-mapped, 4 words blocks Memory





(1) Read address 0x00000030, what happens?

(2) Read address 0x0000001c, what happens?

Please present cache hit or miss, the index, tag match or mismatch, offset, and the return value.

Four Questions for Cache Designers

Caching is a general concept used in processors, operating systems, file systems, and applications.

There are **Four Questions** for Cache/Memory Hierarchy Designers

- Q1: Where can a block be placed in the upper level/main memory?

  (Block placement)

  - Fully Associative, Set Associative, Direct Mapped
- Q2: How is a block found if it is in the upper level/main memory?

  (Block identification)

  - Tag/Block
- Q3: Which block should be replaced on a Cache/main memory miss?

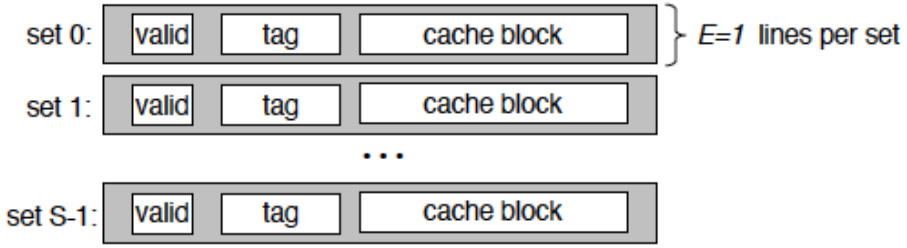
  (Block replacement)

  - Random, LRU,FIFO
- Q4: What happens on a write?

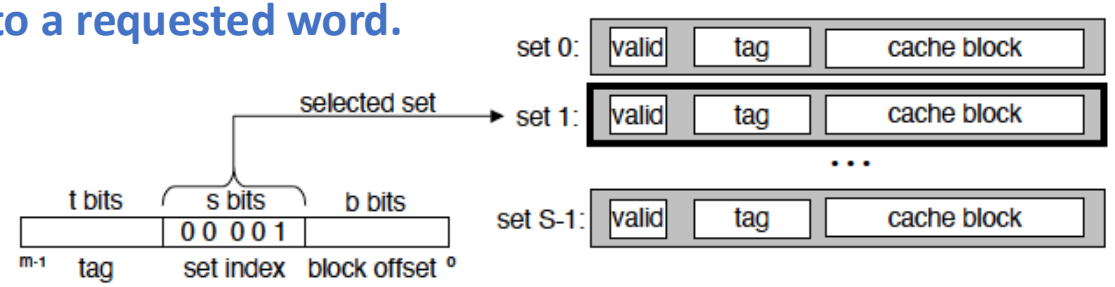
  (Write strategy)

  - Write Back or Write Through (with Write Buffer)

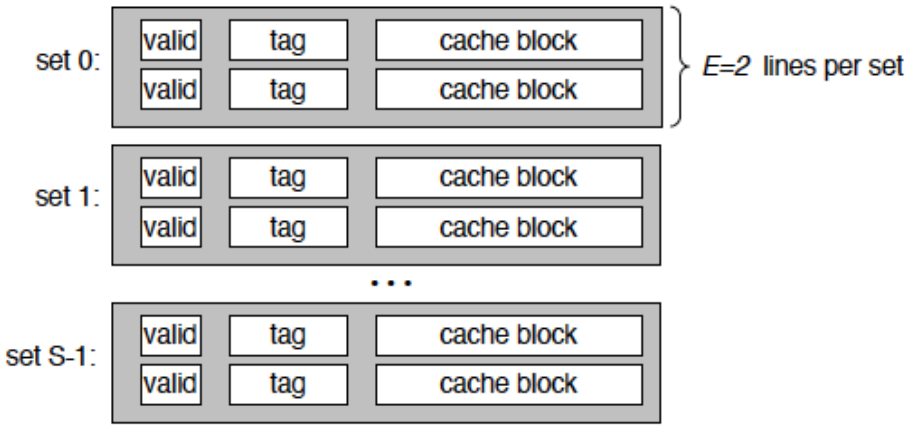
tag: A field in a table used for a memory hierarchy that contains the address information required to identify whether the associated block in the hierarchy corresponds to a requested word.



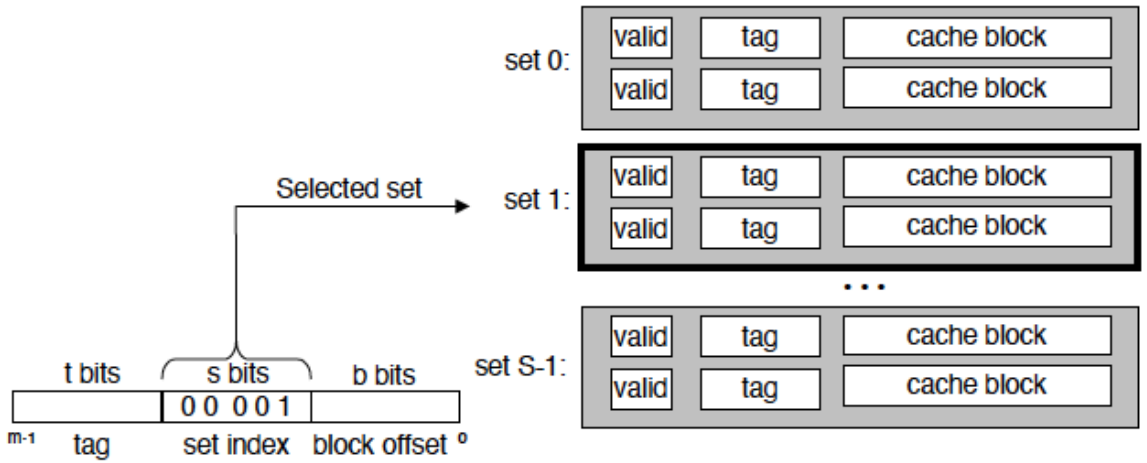
Direct-mapped cache



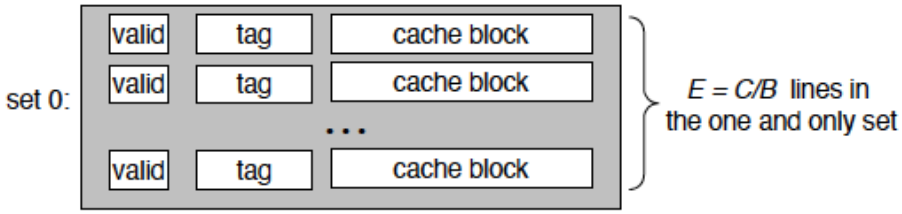
Set selection in a direct-mapped cache



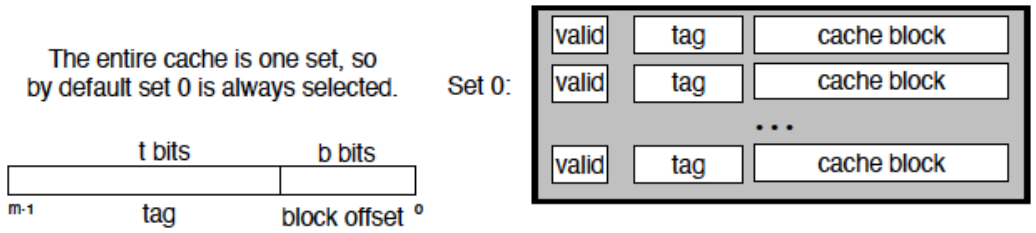
Set associative cache



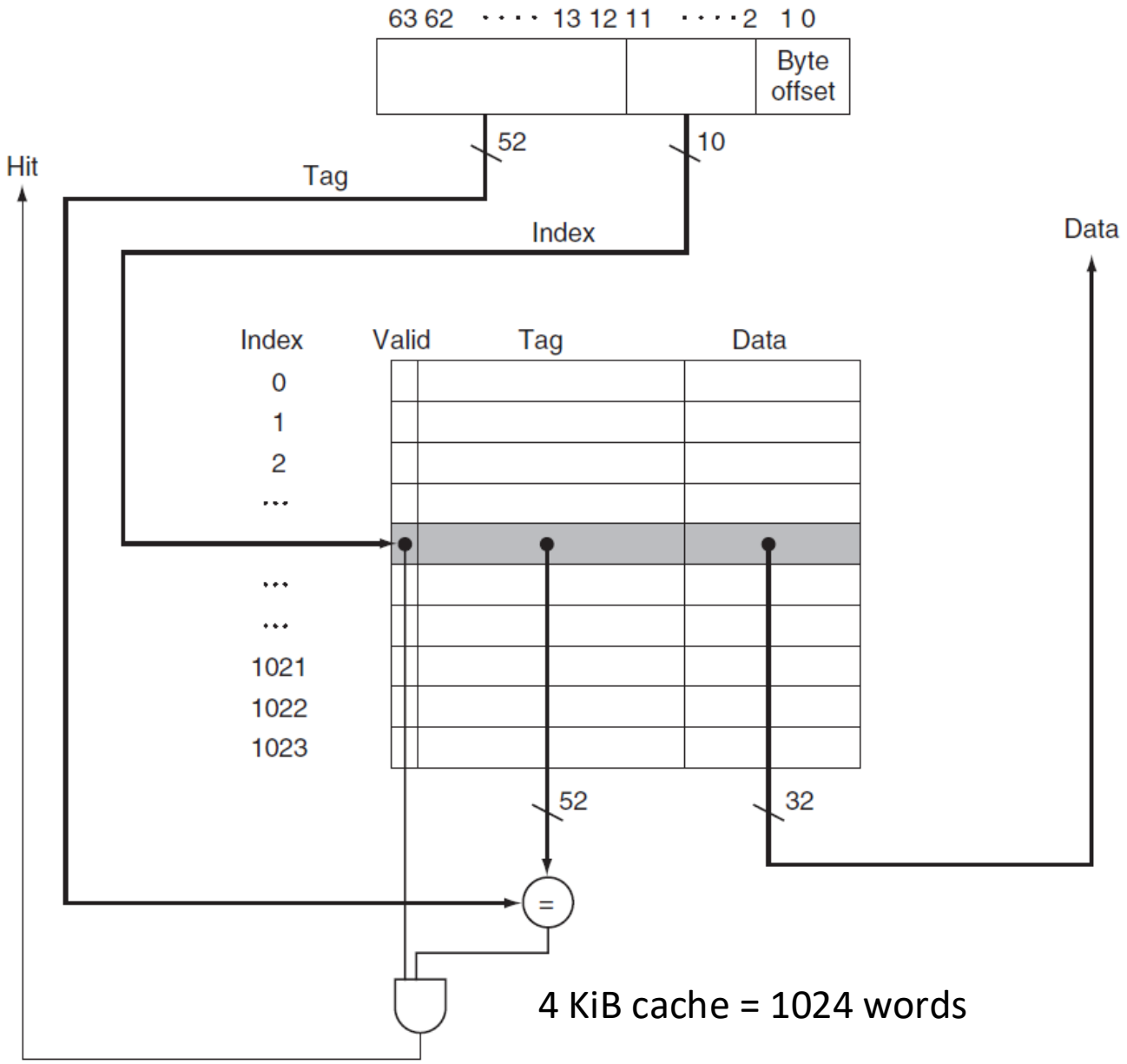
Set selection in a set associative cache



Fully set associative cache



Set selection in a fully associative cache



4 KiB cache = 1024 words

The size of the block was one word (4 bytes).

- 64-bit addresses
- A direct-mapped cache
- The cache size is 2^n blocks, so n bits are used for the index
- The block size is
  - ✓ 2^m words (2^{m+2} bytes = 2^{m+5} bits)
  - ✓ m bits are used for the word within the block, and two bits are used for the byte part of the address

tag $64 - (n + m + 2)$

cache $2^n \times$(block size+ tag size + Valid size).

$= 2^n \times (2^m \times 32 + (64 - n - m - 2) + 1)$

$= 2^n \times (2^m \times 32 + 63 - n - m)$.

actual size/complete cache size

Bits in a Cache

Example 1: How many total bits are required for a direct-mapped cache with 16 KiB of data and four-word blocks, assuming a 64-bit address?

Answers:

16 KiB = 4096 words $=2^{12}$ words

The block size is 4 words = 2^2 words, so m=2

$2^{12} / 2^2$ = 1024 blocks, so n=10

Each block has 4*32 =128 bits of data plus a tag, plus a valid bit

tag = 64-(n+m+2) = 64-(10+2+2) =50 bits

Cache size = $2^n \times$(block size+ tag size + Valid size)

$= 2^{10} \times (4*32 + 50 + 1) = 179$ Kib =22.375 KiB

Mapping an Address to a Multiword Cache Block

Example 2: Consider a cache with 64 blocks and a block size of 16 bytes. To what block number does byte address 1200 map?

Answers:

The cache block number

= (Block address) modulo (Number of blocks in the cache)

Block address = Byte address/ Bytes per block = 1200 /16 =75

75 modulo 64 =11

In fact, this block maps all addresses between 1200 and 1215.

Four Questions for Cache Designers

Caching is a general concept used in processors, operating systems, file systems, and applications.

There are **Four Questions** for Cache/Memory Hierarchy Designers

- Q1: Where can a block be placed in the upper level/main memory?

  (Block placement)

  - Fully Associative, Set Associative, Direct Mapped
- Q2: How is a block found if it is in the upper level/main memory?

  (Block identification)

  - Tag/Block
- Q3: Which block should be replaced on a Cache/main memory miss?

  (Block replacement)

  - Random, LRU,FIFO
- Q4: What happens on a write?

  (Write strategy)

  - Write Back or Write Through (with Write Buffer)

Q3: Block Replacement

- In a direct-mapped cache, there is only one block that can be replaced

§ 3.3 Four Questions for Cache Designers

Binary address of reference	Assigned cache block (where found or placed)
10110_{two}	(10110_{two} mod 8) = 110_{two}
11010_{two}	(11010_{two} mod 8) = 010_{two}
10110_{two}	(10110_{two} mod 8) = 110_{two}
11010_{two}	(11010_{two} mod 8) = 010_{two}
10000_{two}	(10000_{two} mod 8) = 000_{two}
00011_{two}	(00011_{two} mod 8) = 011_{two}
10000_{two}	(10000_{two} mod 8) = 000_{two}
10010_{two}	(10010_{two} mod 8) = 010_{two}
10000_{two}	(10000_{two} mod 8) = 000_{two}

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	10_{two}	Memory (10010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

10110	11010	10110	11010	10000	00011	10000	10010	10000
miss	miss	hit	hit	miss	miss	hit	miss	hit

cache hit If the cache reports a hit, the computer continues using the data as if nothing happened.

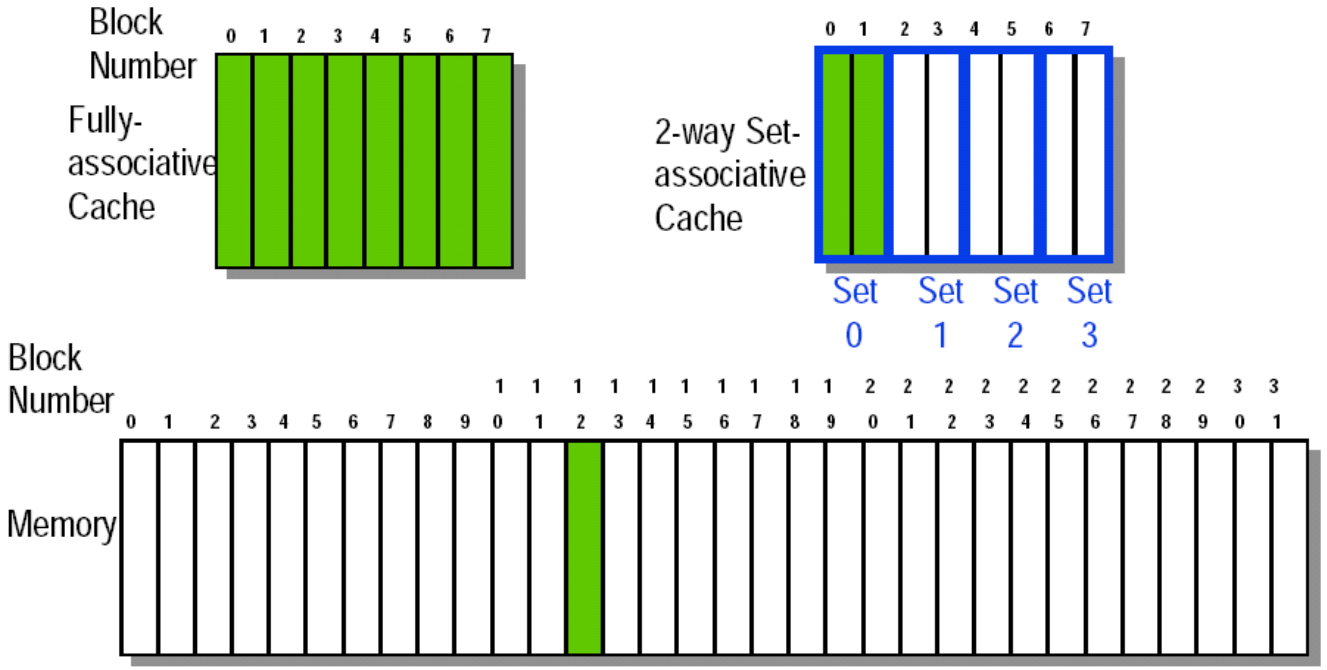
cache miss For a cache miss, we can stall the entire processor, essentially freezing the contents of the temporary and programmer-visible registers, while we wait for memory.

We can now define the steps to be taken on an instruction cache miss:

1. Send the original PC value to the memory. （PC-4）
2. Instruct main memory to perform a read and wait for the memory to complete its access.
3. Write the cache entry, putting the data from memory in the data portion of the entry, writing the upper bits of the address (from the ALU) into the tag field, and turning the valid bit on.
4. Restart the instruction execution at the first step, which will refetch the instruction, this time finding it in the cache.

Q3: Block Replacement

- In a direct-mapped cache, there is only one block that can be replaced
- In set-associative and fully-associative caches, there are N blocks (where N is the degree of associativity)



Strategy of Block Replacement

- Several different replacement policies can be used
  - Random replacement - *randomly pick any block*
    - Easy to implement in hardware, just requires a random number generator
    - Spreads allocation uniformly across cache
    - May evict a block that is about to be accessed
  - Least-Recently Used (LRU) - *pick the block in the set which was least recently accessed*
    - Assumed more recently accessed blocks more likely to be referenced again
    - This requires extra bits in the cache to keep track of accesses.
  - First In, First Out(FIFO)-*Choose a block from the set which was first came into the cache*

Strategy of Block Replacement

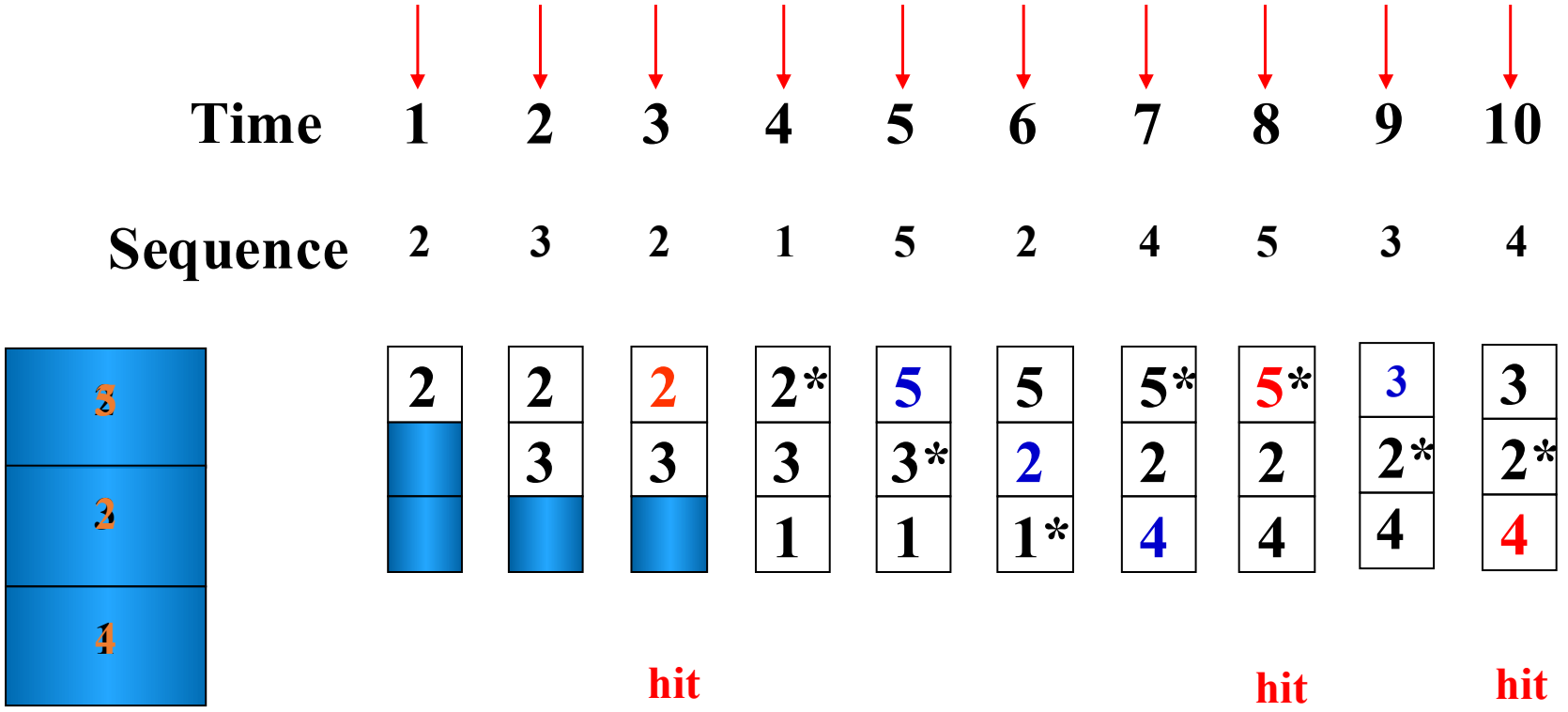
Suppose:

- Cache block size is 3, and access sequence is shown as follows.

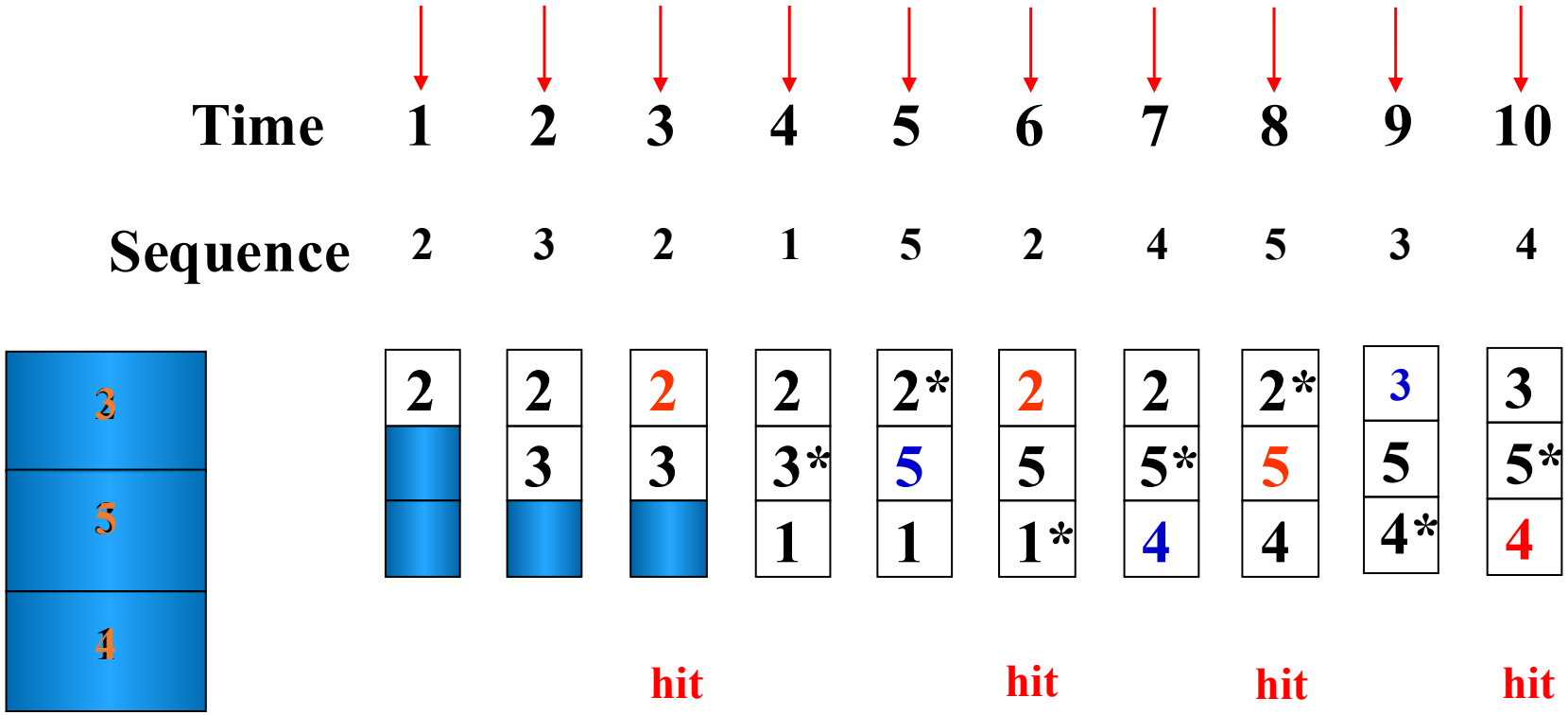
  2, 3, 2, 1, 5, 2, 4, 5, 3, 4

- FIFO, LRU and OPT are used to simulate the use and replacement of cache block.

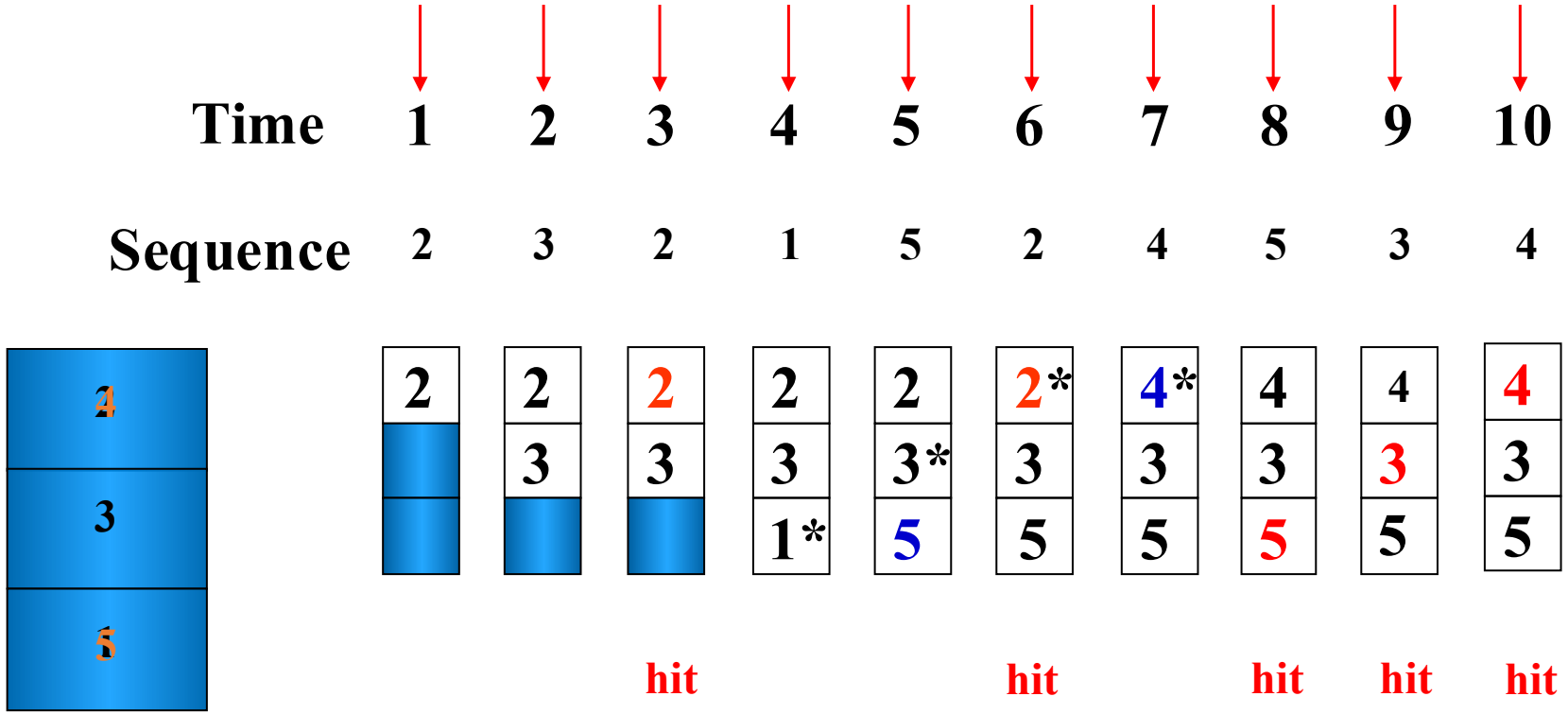
FIFO


Time
1 2 3 4 5 6 7 8 9 10
Sequence
2 3 2 1 5 2 4 5 3 4
2 2 2 2* 5 5 5* 5* 3 3
3 3 3 3* 2 2 2 2 2* 2*
1 1 1* 4 4 4
hit
hit
hit

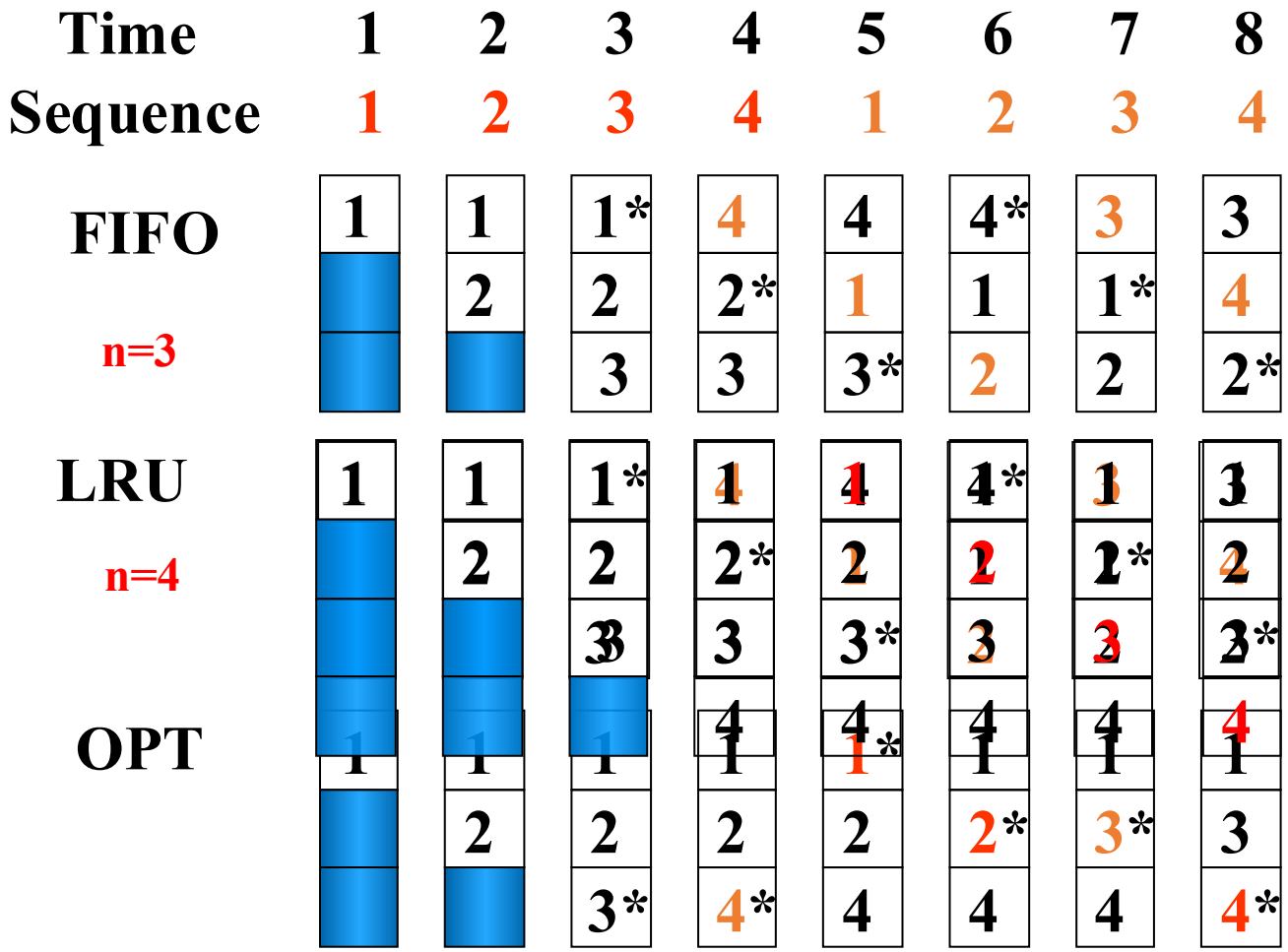
LRU


Time
1 2 3 4 5 6 7 8 9 10
Sequence
2 3 2 1 5 2 4 5 3 4
3
5
4
2 2 2 2 2* 2 2 2* 3 3
3 3 3* 5 5 5* 5 5 5*
1 1 1* 4 4 4* 4
hit
hit
hit
hit

OPT



The hit rate is related to the replacement algorithm.



thrashing

No hit

Hit 4 times

Hit 3 times

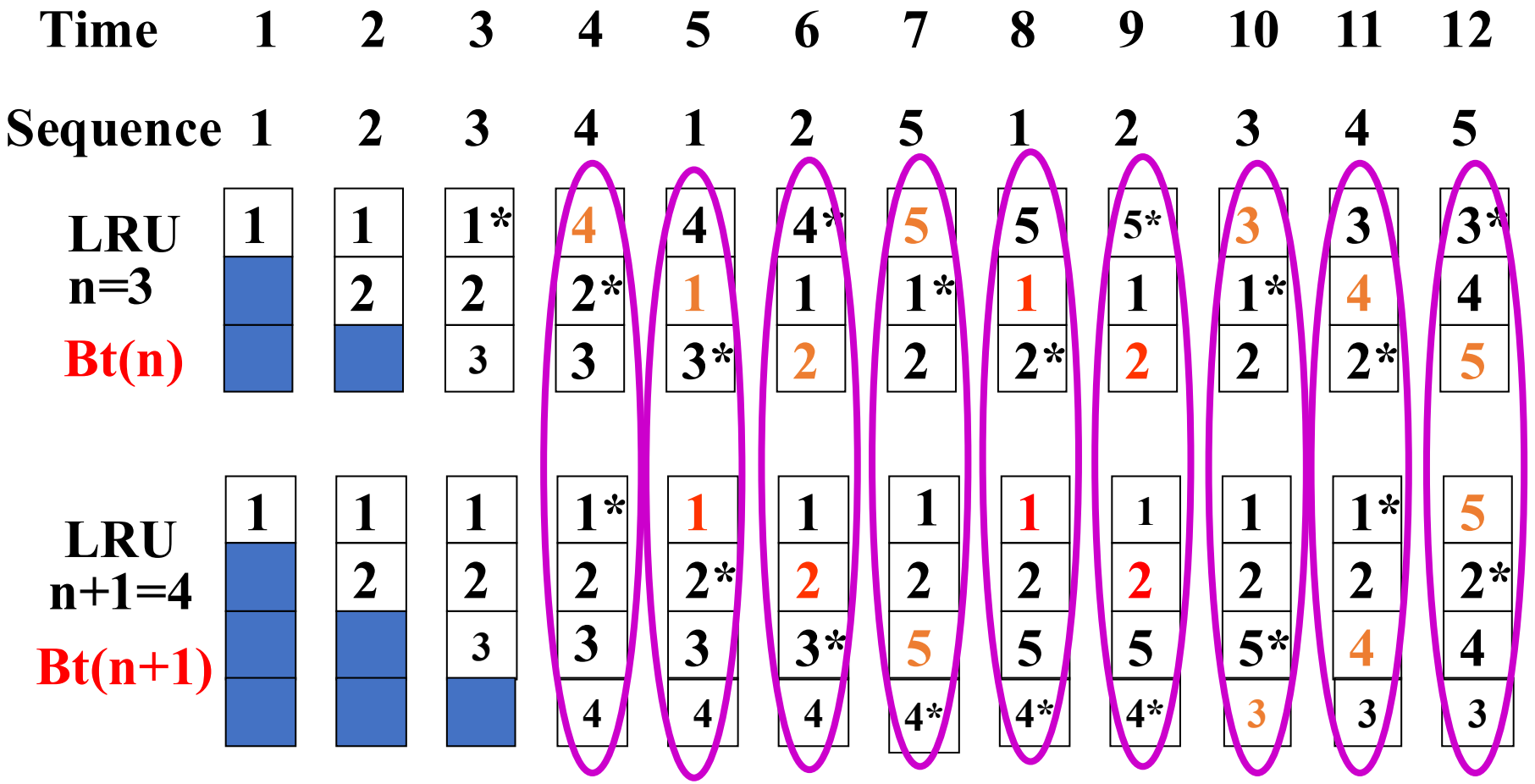
➢Hit rate is related to access sequence.

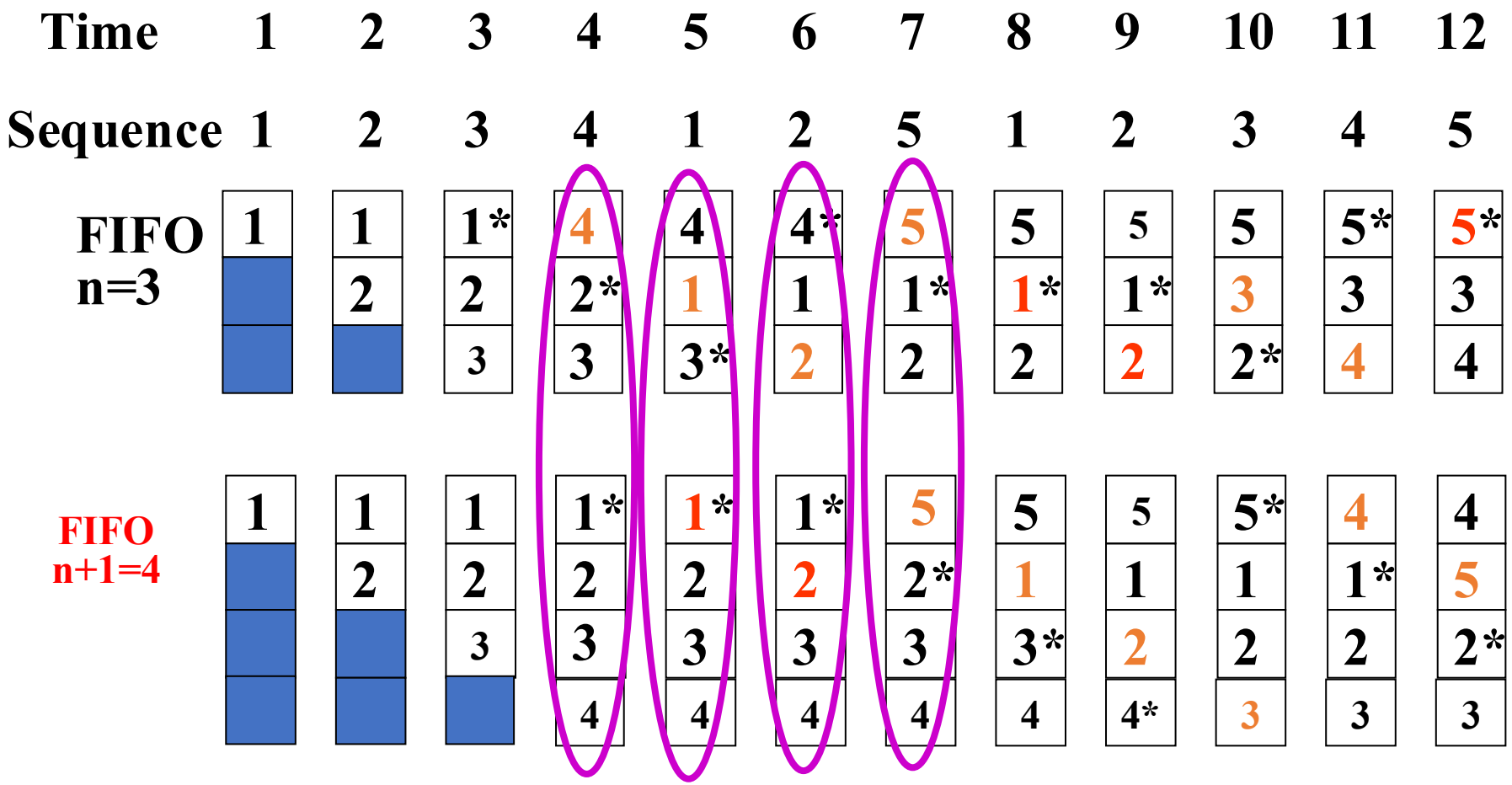
➢Hit rate is related to cache block size.

Stack replacement algorithm

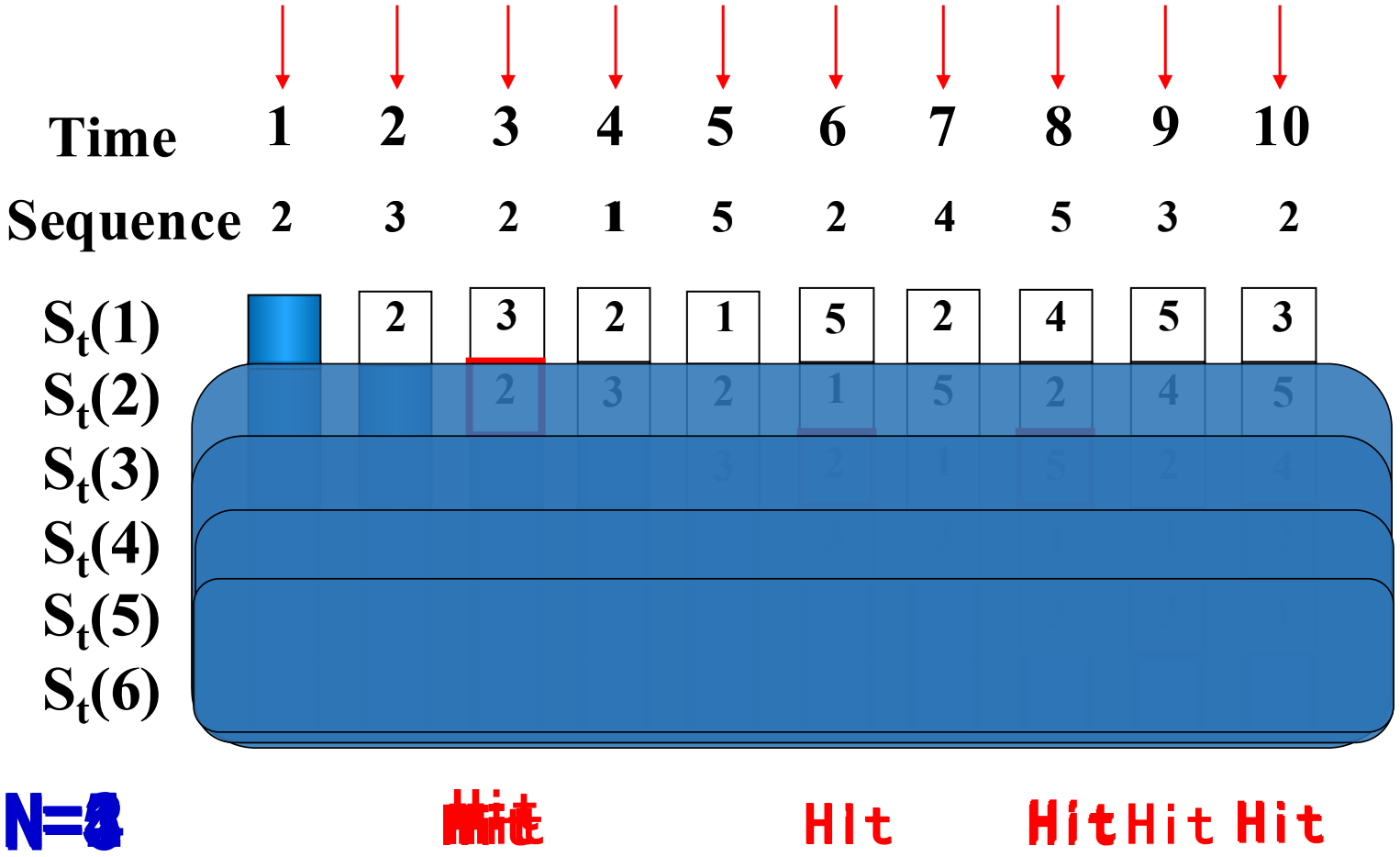
- $B_t(n)$ represents the set of access sequences contained in a cache block of size n at time t.
- $B_t(n)$ is the subset of $B_t(n+1)$.

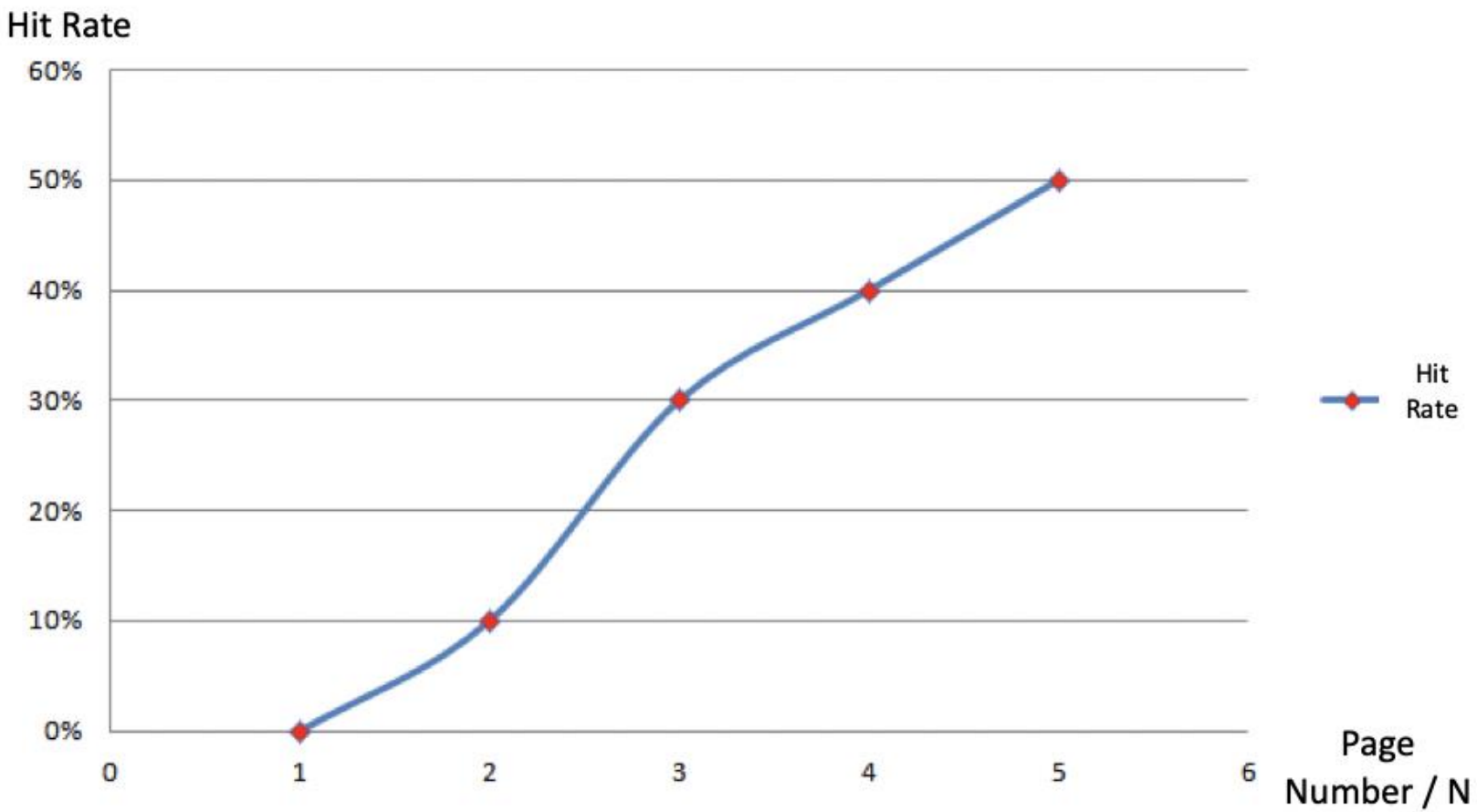
LRU replacement algorithm is stack replacement algorithm



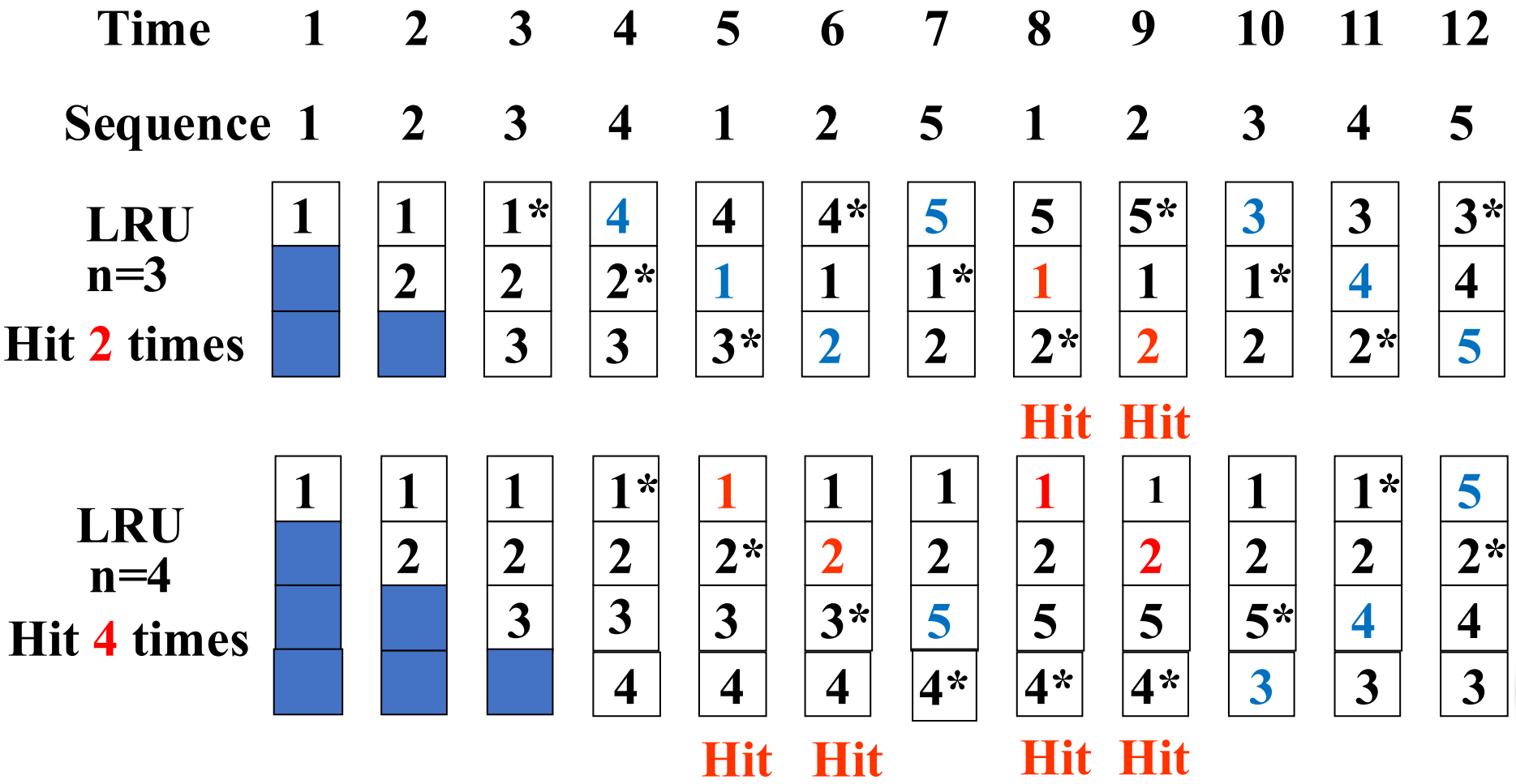


Using LRU


Time
1 2 3 4 5 6 7 8 9 10
Sequence
2 3 2 1 5 2 4 5 3 2
$S_t(1)$
$S_t(2)$
$S_t(3)$
$S_t(4)$
$S_t(5)$
$S_t(6)$
2 3 2 1 5 2 4 5 3
2 3 2 1 5 2 4 5
3 2 1 5 2 4
N=4
Hit
Hit
Hit
Hit
Hit
Hit


Hit Rate
60%
50%
40%
30%
20%
10%
0%
0
1
2
3
4
5
6
Page Number / N
Hit Rate

For LRU algorithm, the hit ratio always increases with the increase of cache block.



Belady

Time	1	2	3	4	5	6	7	8	9	10	11	12
Sequence	1	2	3	4	1	2	5	1	2	3	4	5
FIFO n=3 Hit 3 times	1	1 2	1* 2 3	4 2* 3	4 1 3*	4* 1 2	5 1* 2	5 1* 2	5 1* 2	5 3 2*	5* 3 4	5* 3 4
								Hit	Hit			Hit
FIFO n+1=4 Hit 2 times	1	1 2	1 2 3	1* 2 3 4	1* 2 3 4	1* 2 3 4	5 2* 3 4	5 1 3* 4	5 1 2 4*	5* 1 2 3	4 1* 2 3	4 5 2* 3
					Hit	Hit						

LRU

- How can I implement the LRU replacement algorithm with only ordinary gates and triggers?

- Comparison Pair Method

- Basic idea: Let each cache block be combined in pairs, use a ***comparison pair flip-flop*** to record the order in which the two cache blocks have been accessed in the comparison pair, and then use a gate circuit to combine the state of each comparison pair flip-flop, you can find the block to be replaced according to the LRU algorithm.

Example

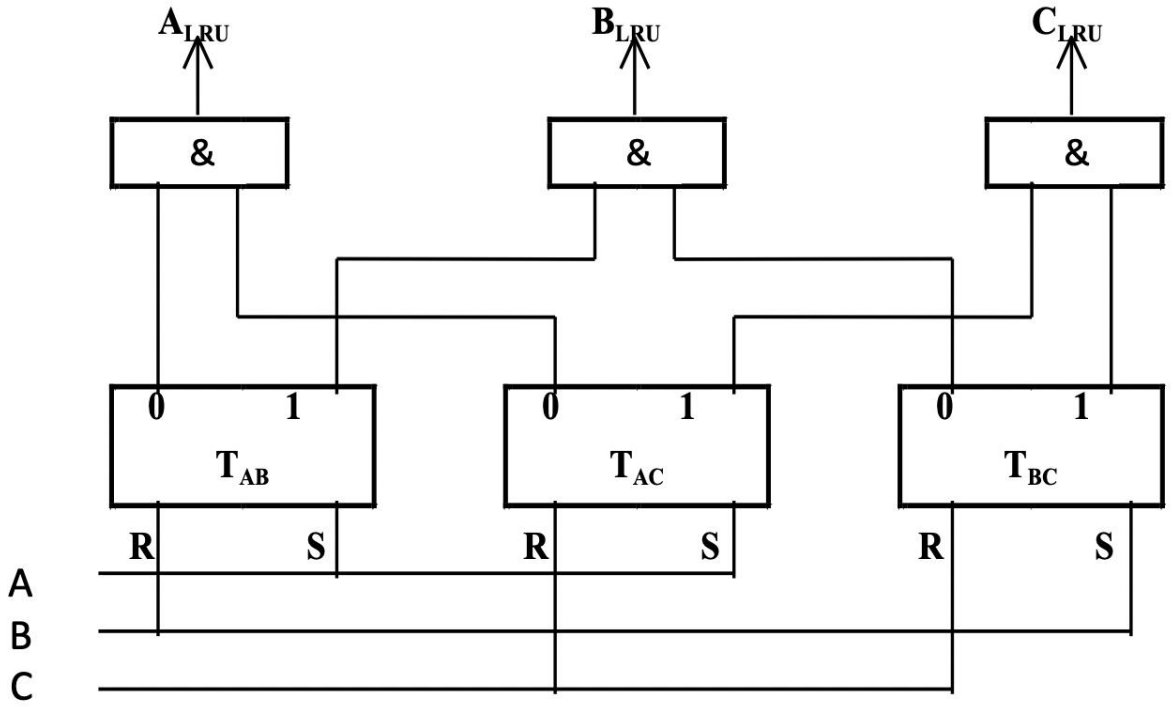
- There are three cache blocks (A, B, and C), which can be combined into 6 pairs (AB, BA, AC, CA, BC, and CB). Among them, AB and BA, AC and CA, BC and CB are repeated, so only take AB, AC, BC.

- The access sequence of each pair is represented by “comparison pair flip-flops” T_{AB}, T_{AC}, and T_{BC} respectively. T_{AB} is "1", which means that A has been accessed more recently than B; T_{AB} is "0", which means that B has been accessed more recently than A. T_{AC} and T_{BC} are similarly defined.

- If the most recently accessed block is A and C is the block that has not been accessed for the longest time, the three flip-flops' states must be respectively: T_{AB}=1, T_{AC}=1, and T_{BC}=1.
- If the most recently accessed block is B and C is the block that has not been accessed for the longest time, the three flip-flops' states must be respectively: T_{AB}=0, T_{AC}=1, and T_{BC}=1.
- Therefore, the block C that has not been accessed for the longest will be replaced. In that, the Boolean algebra expression must be:

$$C_{LRU} = T_{AB} \bullet T_{AC} \bullet T_{BC} + \overline{T_{AB}} \bullet T_{AC} \bullet T_{BC} = T_{AC} \bullet T_{BC}$$

$$B_{LRU} = T_{AB} \bullet \overline{T_{BC}} \qquad A_{LRU} = \overline{T_{AB}} \bullet \overline{T_{AC}}$$



- Change the state of the flip-flop after each access.
  - After accessing block A: T_{AB}=1, T_{AC}=1
  - After accessing block B: T_{AB}=0, T_{BC}=1
  - After accessing block C: T_{AC}=0, T_{BC}=0

Hardware usage analysis (if *p* is the number of cache blocks)

- Since each block may be replaced, its signal needs to be generated with an AND gate, so the number of AND gates will be equal to ***p***.
- Each AND gate receives inputs from its related flip-flops, for example, A_{LRU} AND gates must have inputs from T_{AB} and T_{AC}, B_{LRU} must have inputs from T_{AB} and T_{BC}, and the number of comparison pair flip-flops is the block number minus 1, so the input number of the AND gate is ***p - 1***.
- If p is the block number, for pairwise combination, the number of comparison pair flip-flops should be C_p^2, which is ***p·(p-1)/2***.

The Relationship between the Block Number and Required Hardware for Comparison Pair

Block Number	3	4	6	8	16	64	256
Number of Flip-flop	3	6	15	28	120	2016	32640
Number of And-Gate	3	4	6	8	16	64	256
Input Number of And-Gate	2	3	5	7	15	63	255

Homework

For a direct-mapped cache design with a 64-bit address, the following bits of the address are used to access the cache.

Tag	Index	Offset
63-10	9-5	4-0

(1) What is the cache block size (in words)?
(2) How many blocks does the cache have?
(3) What is the ratio between total bits required for such a cache implementation over the data storage bits?

Beginning from the power on, the following byte-addressed cache references are recorded.

Address												
Hex	00	04	10	84	E8	A0	400	1E	8C	C1C	B4	884
Dec	0	4	16	132	232	160	1024	30	140	3100	180	2180

(1) For each reference, list i) its tag, index, and offset, ii) whether it is a hit or a miss, and iii) which bytes were replaced (if any).
(2) What is the hit ratio?